Hegel Machine: A Framework for Recursive Hypothesis Formation, Testing, and Revision

Yifan Zhang ERZHU419@GMAIL.COM; 2042010480@CSU.EDU.CN
Central South University
Changsha, Hunan, China

Liang Zheng ZHENGLIANG@CSU.EDU.CN
Central South University
Changsha, Hunan, China

Abstract

An agent's hypotheses depend on what it can represent, observe and test. We present Hegel Machine, a framework for revising these conditions of research alongside predictive explanations. An unresolved supporting assumption becomes a child hypothesis; its evidence can change the parent's next experiment, representation or explanation. Research procedures and meta-priors are themselves revisable objects. A functional-analytic perspective makes the coupling explicit: available observations induce a geometry on candidate predictions, leaving some directions of disagreement invisible. Research can change the observations as well as the fitted hypothesis. Finite implementations demonstrate supporting evidence changing a pending experiment and new measurement access restoring prediction in 30/30 cases versus 0/30 without it. A language-model-authored procedure revision improves frozen transfer from 10/18 to 18/18. In an anonymous recurrent setting, an authored state representation is consolidated into a fitting procedure that solves six new-parameter targets without target-time model calls; two live developmental continuations also reuse it after failed confirmation. Preserving a refuted program improves later repair and return, with 90/90 continuation successes versus 69/90 after deletion. The evidence supports constituent mechanisms of cumulative hypothesis research. Matched flat research attains equal accuracy, and complete Code, textual-memory and Hegel agents solve 29/30, 29/30 and 26/30 targets; unique benefits of recursive organization and overall agent superiority remain open.

Keywords: recursive hypothesis research; active experimentation; program revision; executable knowledge; representation learning

1. Introduction

A useful hypothesis abstracts from details. Even a simple prediction assumes that a selected representation contains enough information to determine an outcome. A conservation law, a separability assumption and a research heuristic make different commitments, but each proposes a regularity that may carry beyond the cases from which it was formed. Learning can improve an answer within that abstraction, or expose a reason to revise the abstraction itself.

A failed prediction does not identify which commitment failed. An instrument may have changed, a formerly negligible interaction may now matter, or a response may depend on a condition absent from the current observations. Refitting can be appropriate; so can acquiring a new measurement, restricting an applicability claim, or investigating a

supporting assumption. Even an experiment that once resolved a question can lose its distinguishing power after a change in coordinates. Research therefore needs to change its questions and methods as well as its answers.

Hegel Machine treats the conditions under which hypotheses are formed and tested as further objects of hypothesis research. If a parent explanation depends on an uncertain instrument meaning, the instrument becomes a child question. Its answer returns to change the parent investigation. If the difficulty lies in a research procedure, that procedure becomes the object of revision. Acquired explanations and methods then change what later research can express and accomplish. This is a recursive organization of inquiry: supporting assumptions can themselves require support, while empirically settled subproblems can be reused.

The functional-analytic perspective is central to this formulation. A hypothesis can be viewed through its predicted responses to experiments, and an observation through the distinctions it can reveal among those predictions. Together they determine an observable geometry. Two explanations may differ internally yet coincide under every current test. Repeating those tests cannot expose the missing distinction; a different probe, measurement or representation may do so. The framework consequently pairs revision of the hypothesis space with revision of the observations used to investigate it. Section 3 makes this precise, including the conditions under which observations are continuous linear functionals.

The longer-term objective is to learn abstractions that explain both new evidence and the valid scope of earlier explanations. This involves expansion and simplification: an agent may introduce uncertainty when a deterministic description fails, or adopt a simpler effective model when the neglected detail makes no relevant difference. Across tasks, a proposed correspondence between structural roles must itself have testable consequences. A broader explanation should preserve earlier successes where warranted, while changing the assumptions responsible for failure.

Cumulative research requires separating **accepting a prediction, retaining a computation, and reusing it in a repair**. Suppose acquired components A and B support $C = A + 0.7B$. When the event component changes to B', C fails as a current predictor, yet its executable structure can participate in

$$D = C + 0.7(B' - B) = A + 0.7B'.$$

It can also be useful when its original conditions return. This preservation mechanism serves the broader research process: neither retained code nor a successful earlier argument automatically certifies a new application.

Program libraries, belief revision, active experiment design and software repair supply important precedents. Our contribution is an explicit organization of hypotheses, observations and supporting research, together with interventions that test which acquired objects change subsequent behavior. We distinguish predictor reuse from method reuse, observable equivalence from representational sufficiency, and controller operations from choices authored by a language model.

The experiments follow that argument. We first examine child-to-parent evidence and the acquisition of missing distinctions, then the revision and transfer of research procedures, and preservation through failure and return. A constrained-optimization application tests whether learned priors change downstream decisions. Finally, an anonymous recurrent

apparatus connects language-model-authored representation, method consolidation, new-parameter transfer and live reuse after a counterexample. This last study separates three consequential achievements: finding a predictive explanation, learning a procedure that can identify a new instance, and choosing to use that procedure again when an initial fit fails. Each stage is evaluated through its actual execution.

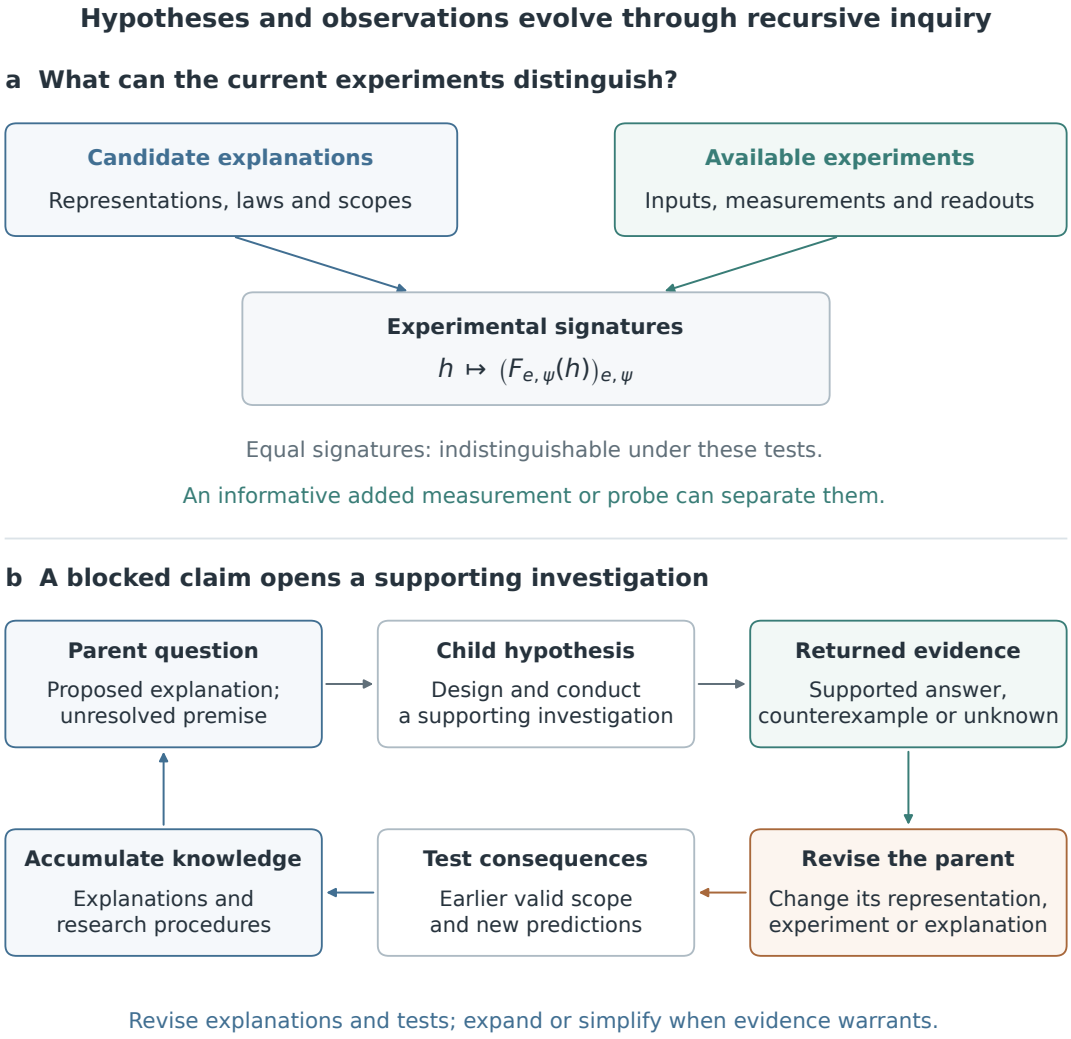


Figure 1. Hypothesis spaces and recursive research. a, Candidate explanations and available experiments jointly determine observable signatures. Equal signatures leave explanations indistinguishable under those tests; an informative added experiment can separate them. b, An unresolved premise opens a supporting investigation. Its evidence changes the parent investigation, whose revised consequences are tested in earlier valid conditions and new ones. Both explanations and research procedures accumulate and can be revised. Each reported implementation realizes a bounded subset of this organization.

2. Relation to earlier work

Program libraries and repair. DreamCoder jointly learns reusable program components and a search policy, showing how acquired abstractions make later synthesis more accessible (Ellis et al., 2021). GenProg uses executable tests to guide program repair (Le Goues et al., 2012). These are direct precedents for computational retention and test-guided revision. A new callable name is not by itself new semantic knowledge: its body may already be expressible in the base language. Here its value is measured by later predictions and research costs, including interventions that remove the acquired object.

Rules and experiments. Piriyakulkij et al. (2024) combine rule revision with active experiments. POPPER generates subordinate falsification tests for supplied hypotheses and provides a sequential statistical testing framework (Huang et al., 2025). Our finite confirmation checks serve a different operational purpose: a selected executable candidate must predict a newly acquired trajectory. The experiments examine how a supporting answer changes a parent investigation and how a failed research method is revised. Formal sequential error control is outside the present implementation.

Experience and reasoning procedures. Reflexion stores textual feedback that informs later attempts (Shinn et al., 2023). SELF-DISCOVER composes reasoning structures from available modules (Zhou et al., 2024). These motivate comparisons between text, executable procedures and ordinary code investigation. In our method-learning experiments, a model edits a Python action-selection program; frozen consumers then run it with shared numerical operators. The executable change and its subsequent effect can therefore be inspected separately from the author's explanation.

Belief and dependency revision. AGM theory distinguishes contraction and revision of commitments (Alchourrón et al., 1985), while assumption-based truth maintenance tracks the premises supporting conclusions (de Kleer, 1986). We distinguish these support relations from computational calls. A program can remain executable after the evidence for applying it changes. Support is associated with a scope, binding and version, and revised parent predictions require their own empirical test.

Table 1. Mechanism-level positioning. The final column states the additional question tested here, rather than a claim that the preceding work lacks every related capability.

Precedent	Established mechanism	Question examined here
Program library learning and repair	Reusable code; test-guided modification	Does an observed-failing response remain useful for repair and return?
Active rule learning and POPPER	Hypothesis revision; informative or subordinate tests	Does a supporting answer change the unexecuted parent experiment?
Reflexion and SELF-DISCOVER	Reusable experience; composed reasoning procedures	Does a changed executable research method improve frozen downstream execution?
Belief revision and truth maintenance	Context-dependent commitments and support	Which computational resources remain usable when a predictive commitment is withdrawn?

These mechanisms can also be implemented in an ordinary executable archive. The current evidence isolates several knowledge-availability effects, but does not isolate the entire dependency organization against one unified, equally capable archive. Section 6 states this remaining attribution gap.

3. Hypothesis spaces, observations and recursive research

3.1 A scoped abstraction

Let an experiment e specify controlled inputs, a trajectory length and requested measurements. A predictive hypothesis is $h = (\phi, p, s, \Gamma)$: a representation ϕ, executable program p, applicability scope s, and supporting premises Γ. For a deterministic response it predicts $\hat{y}_n = p(\phi(x_{\leq n}))$; an uncertainty model instead specifies conditional predictive distributions. The same statement may be a hypothesis under investigation, a working assumption in another inference, or a question reopened by a counterexample.

The research state contains representations, an executable language, experiment constructors, procedures, knowledge, evidence and remaining resources:

$$\mathcal{R} = (\Phi, \mathcal{L}, \mathcal{E}, \Pi, K, D, b).$$

Learned definitions extend the effective language available to bounded search; learned procedures change how that search proceeds. The environment interface, initial primitive families, resource ceilings and evaluation loss are supplied by the protocol. Acquired coefficients, coordinate bindings, compositions, empirical scopes and executable method edits are the learned objects. The controlled implementations do not invent unrestricted new sensors or arbitrary mathematical primitives.

We distinguish three objects by what they change. A predictive program p maps represented inputs to predicted outcomes. A research procedure $\pi(D, K)$ selects research actions from acquired evidence and knowledge. A meta-prior η biases which candidate structures or research directions are generated, ranked or reused. A learned p can serve as a prior feature in a downstream optimizer, as in Section 5.4; an authored revision of π changes executable research control flow, as in Section 5.2. Preferences among existing methods can also change with accumulated returns: in Appendix E.1, retaining return records changes method selection in 12/12 cases, although neither arm completes the subsequent demands or production tasks. This is evidence that research preferences respond to experience, with no demonstrated completion benefit in that comparison.

A procedure is itself a hypothesis about useful action selection. Its empirical consequences include which tasks it resolves, what it wrongly confirms, which earlier abilities survive, and what resources it consumes. Testing a research preference likewise requires tracing its effect on subsequent choices and outcomes. Table B2 identifies the controller, learned object and fixed interface for each implemented operation.

3.2 A functional-analytic view: what can an observation distinguish?

Fix a nonempty domain of legal physical experiments $\mathcal{E}$. A hypothesis h determines a response function f_h on that domain. More generally, an experiment and a statistic of its outcome define an observation map $F_{e,\psi}(h)$: for example, the predicted reading, mean

response or variability under a controlled trajectory. A finite family Q of such maps gives the observable signature $S_Q(h) = (F(h))_{F \in Q}$. Hypotheses with identical signatures are observationally equivalent under Q. Here Q may describe observations already acquired or a planned test family; it is distinct from the entire legal experiment domain.

A classical linear instance clarifies the geometry. Let V be a real linear subspace of the bounded scalar response functions on $\mathcal{E}$, equipped with the supremum norm. The embedded hypotheses need not themselves form a linear space. Evaluation at an experiment, $\delta_e(f) = f(e)$, is a continuous linear functional because $|\delta_e(f)| \leq \|f\|_\infty$. For a nonempty finite family $Q \subset V^*$ of continuous linear observations, define

$$p_Q(f) = \max_{q \in Q} |q(f)|, \qquad K_Q = \bigcap_{q \in Q} \ker q.$$

The seminorm p_Q measures disagreement visible to these observations. Its kernel K_Q contains the directions they cannot detect. Consequently, $d_Q(f, g) = p_Q(f - g)$ is a pseudometric on V, and $\|f + K_Q\|_Q = p_Q(f)$ is a norm on the observable quotient V/K_Q. This is the norm induced by p_Q, not necessarily the quotient norm inherited from the original supremum norm. Seminorms, continuous duals and quotient spaces are standard functional-analysis constructions (Melrose, 2020; Zsák, 2023). Here they provide an organizing formulation: observation-induced indistinguishability becomes explicit, making it possible to ask which supporting answer would change the distinctions available to an investigation.

Adding observations $Q \subseteq Q'$ shrinks the invisible subspace: $K_{Q'} \subseteq K_Q$. It reveals a previously missing distinction exactly when an added functional is nonzero on some old invisible difference. For example, on a bounded control domain the responses $f(z) = z^2$ and $g(z) = |z|$ agree at $z = -1$, whereas at $z = -0.6$ they predict 0.36 and 0.6. A new probe can expose their difference without changing either response law. Figure 2 records this situation in an executed investigation.

General experiment maps, conditional variances and research success criteria need not be linear functionals. Their signatures still define observational equivalence, and pulling back a metric on those signatures gives a pseudometric on hypotheses. Appendix A gives the linear argument and a distributional version. The numerical controllers use entropy reduction, disagreement scores and task losses for action selection; no common norm is assumed for all of them.

This formulation separates two research failures. An **observation limitation** leaves competing explanations indistinguishable under current tests. A **representation limitation** prevents the predictor from expressing a relevant difference in its inputs: if $\phi(x_1) = \phi(x_2)$ but $y_1 \neq y_2$, every deterministic predictor of ϕ incurs average squared error at least $(y_1 - y_2)^2/4$ on that pair. More fitting in the same representation cannot remove this error. A new measurement may resolve it; uncertainty modeling may instead be appropriate when the residual variation persists. Choosing between these possibilities is itself an empirical question.

3.3 Revising the space, the observations and the scope

The paired view suggests three coupled operations: change candidate explanations, change the observations that discriminate them, or change the representation and its supporting

interpretation. An instrument or coordinate hypothesis specifies how a physical intervention becomes an input to a response law. Revising that binding can therefore change which experiment is informative, even when the candidate laws stay fixed. Comparisons across bindings must refer to common physical experiments; an arbitrary change of coordinates must not count as a new empirical success. The terminal evaluation contract remains fixed while the research procedure changes.

Model expansion and simplification fit the same picture. Expansion introduces a missing variable, interaction, condition or uncertainty family. Simplification removes distinctions irrelevant to the current question while retaining their valid range. A proposed generalization may take the form $p_{\mathrm{new}}(x, c) = p_{\mathrm{old}}(x)$ on supported conditions $c \in s$, with a revised law elsewhere. Both preservation and new predictions require testing. The aim is a wider explanation of why an earlier model worked and where it fails, rather than an ever-growing list of exceptions. The implemented conditional branches and residual compositions provide finite instances; general discovery of such explanations remains a research objective.

Structural transfer adds another hypothesis: a mapping from the roles in one task to those in another preserves the relevant relation. Matching words or numerical coordinates alone cannot establish it. A proposed binding should predict the outcome of a target intervention. These correspondences are typed: a world constraint restricts possible outcomes, a complexity preference directs search, and an evidence rule governs support. They require different tests and have different consequences when revised. The present experiments test renamed, permuted and recalibrated controls, component changes and old-condition returns within supplied task families.

There are direct implementations of the paired view. The online researcher groups candidates by finite response signatures, selects experiments by predicted reduction of uncertainty, and rewrites a retained probe after supporting calibration. The growth researcher purchases candidate measurement channels and compares the representations on common acquired records. It also fits conditional variance and compares simpler response programs. These are finite operational instances of changing observable distinctions and effective model classes. Appendix B.2 maps them to the actual study sources; the framework does not require a generic symbolic implementation of a continuous dual space.

The recurrent study in Section 5.5 illustrates another consequence. A fitted history-dependent response can match every acquired short trajectory and still fail on longer trajectories. The new observations then constrain a refit within the same representation. Rejecting that fitted predictor does not alone reject the representation or the procedure that produced it. These are hypotheses at different levels: a numerical instance, a parameterized explanatory family, and a method for identifying members from evidence. The experiment records which level actually changes.

3.4 Computational dependence and empirical support

Let E_{call} contain edges $p \to q$ when p executes a call to q. Let E_{support} contain edges from a claim to the assumptions whose support it needs. The graphs answer different questions: what computation runs, and what warrants its application. A fit can be executable while one of its instrument assumptions is unresolved.

The relation Support$(p; s, D, v)$ records empirical support for program p under scope s, evidence D and version/binding v. This is a relation to particular evidence, not an assertion of universal truth. Implementations realize it through acquired records, dependencies, case-specific practices and confirmation outcomes. The notation makes explicit why support for an earlier source cannot silently certify an edited source or a new coordinate binding.

Table 2. Research-state transitions. Retention and predictive acceptance are separate decisions.

Event	Predictive/support consequence	Computational consequence
A current application is refuted	Withdraw or restrict its current support; retain the counterexample	Keep the program available for testing, composition or repair
A supporting instrument is unresolved	Suspend the dependent claim; investigate the instrument	Preserve unaffected definitions and alternative bindings
Code, scope or binding changes	Reassess the affected support relation and dependent claim	Create or select the revised executable candidate
A frozen replacement passes a new test	Record support for that candidate in the tested conditions	Make the confirmed result available to later tasks
Earlier conditions return	Compare retained predictions on current evidence	Call the old program or replay a method if supported

The first transition is the retention experiment's intervention. The instrument transition is tested by the online support experiment. Method-edit practice tests the version transition. They share a framework but use distinct controllers and protocols.

3.5 A research loop with supporting questions

The following algorithm describes the common organization. Each study supplies concrete candidate generators and stopping rules; it is not a claim that one implementation executes every operation.

```
Input: public laboratory, initial operators, acquired knowledge K,
       paid evidence D, resource and research-round ceilings
Output: a tested predictor, or the incumbent and unresolved questions;
        updated knowledge, counterexamples and resource ledger

1. Compare current fits and available retained programs on paid evidence.
2. While an eligible research action remains within its ceilings:
   a. Locate the unresolved distinction: response, representation,
      applicability condition, supporting assumption, or method.
   b. If a parent decision needs an unresolved supporting answer,
      open that child question and run its investigation.
      Return its measured answer to revise the pending parent action.
   c. Construct or adapt candidates using the shared operators.
      If current observations cannot resolve a relevant distinction,
      propose a new probe, measurement or supported representation.
      Retain useful earlier computations independently of acceptance.
   d. Choose a distinguishing experiment or a proposed confirmation;
```

```
      acquire and charge its observations and measurements.
   e. If a frozen candidate passes fresh confirmation, record the
      candidate-specific support and return the response.
   f. Otherwise retain the counterexample, revise the affected claim,
      and reopen a direction when new evidence makes it informative.
3. Stop when supported completion, a research/step limit, absence of an
   informative permitted action, or a resource limit ends the visit.
   Preserve unresolved status rather than promote a merely fitted answer.
```

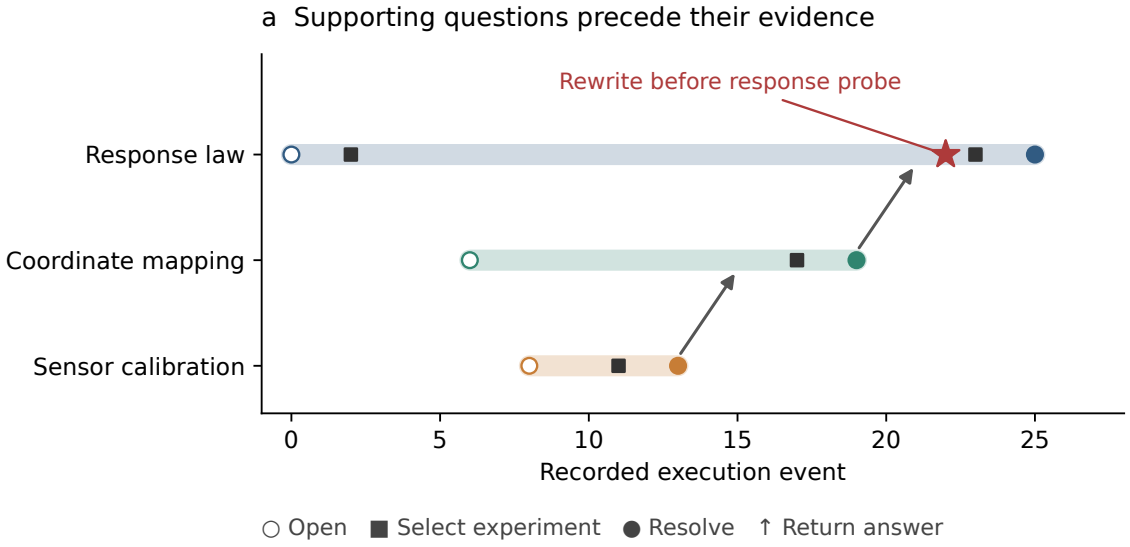
The numerical implementations include sparse and composed response fits, relation-boundary probes, instrument calibration, bounded temporal alternatives and retained definition calls. Some older nested-relation paths fit components first and then compile an answered dependency graph. Those graphs describe a decomposition of completed computation. The online laboratory instead opens a child before its decisive observation and consumes the answer before choosing the parent's next experiment.

Figure 2 shows the distinction. The parent needs a coordinate mapping; a child investigates sensor calibration. Returned calibration and coordinate evidence identifies the second control as the driver. The retained probe $(-0.6, -1)$ then aliases quadratic and absolute-value responses, both predicting 1. The method changes the probe to $(-1, -0.6)$ before acquiring the response, separating all four public alternatives. A deterministic intervention changing only the returned calibration evidence changes the parent interpretation and probe; fresh confirmation rejects the resulting incorrect branch.

For learned Python methods, the model edits `select_actions`, while the host supplies action execution, paid practice and publication checks. Frozen consumers execute the resulting program with the same numerical operators. Automatic practice after an edit and host fallback to an older source are explicit experimental conditions, not actions attributed to the model. In the autonomous recall intervention, host fallback is disabled and restoring a source must be selected by the model.

4. Evaluation design and resource interpretation

We report interventions within fixed cohorts. An apparatus or complete source chain is the statistical unit where its tasks share learned knowledge. Repeated author attempts on one state are not independent environments, and running two programs on the same targets does not double the number of target contracts. Terminal evaluation follows research and is not returned for model selection. Fresh confirmation is a research-time check after freezing the candidate; threshold crossing and independently correct prediction are reported separately.



b Returned binding changes a useful test

Child answer: the second control is the driver

Candidate	Old probe	Revised probe
Linear	-1.00	-0.60
Quadratic	1.00	0.36
Absolute	1.00	0.60
Threshold	0.00	0.00

Old: (−0.6, −1.0) Revised: (−1.0, −0.6)

Old probe aliases quadratic and absolute responses.
Revised probe separates all four remaining laws.

Figure 2. An executed child-to-parent update. Chain 79101013 opens supporting questions before their resolving observations. Returned evidence changes the response experiment before execution. The candidate predictions use the recorded language and acquired binding. This trace demonstrates the mechanism; matched-control performance is reported separately in Section 5.1.

Table 3. Evidence map. All numerical studies are offline except the explicitly model-authored phases. Table B2 separates who initiates each operation, what changes and what remains fixed. The appendices and artifact index identify each frozen protocol and retained failure record.

Evidence package	Controller and learned object	Independent unit and status	Question answered
Observation and model-class revision	Fixed growth researcher; acquired view and response/variance model	30 apparatus sequences with paired access and model controls	Value of missing measurements; expansion and simplification
Retention after refutation	Fixed numerical researcher; acquired response library	30 new apparatus sequences under a fixed protocol	Effect of retained computation; conditional marking ablation
Procedure availability	Fixed growth researcher; acquired research steps	30 shared-prefix paired successors	Marginal value of the procedure book
Method revision	LLM edits; generic Python frozen execution	Separate 18-, 32- and 56-target cohorts after source development	Effect of each frozen control-flow revision
Online support	Fixed recursive and flat controllers	30 new source chains; a separate 30-pair temporal cohort	Child-to-parent operation; matched flat performance
Complete agents	Same model, tools and allowances	30 source–target chains per arm	Overall completion and predictive outcomes
Autonomous recall	LLM selects or edits archived methods	20 attempts on one regressed state	Local selection/publication effect of supported history
Executable hypothesis delegation	LLM proposes a basis; fixed researcher tests and revises it	Six continuations of one paid development failure	Adoption under two research assignments; physical completion
Source-condition consolidation	Fixed coefficient programs; rebuilt source applicability intervals	12 paired successor conditions from three related development branches; offline	Transfer after an old interval fails; costs of invalid reuse
SC-KG application	Numerical prior search and a frozen optimizer	Nine historical cases; retrospective matched rerun	Downstream decision effect under the historical contract
Anonymous recurrent method	LLM-authored representation and fitting procedure; frozen and live consumers	Two source settings, one method author, six new-parameter targets, and a repeated two-target live comparison	Whether an acquired method transfers and is reused after a failed fit

The object studies evaluate four independent 64-row trajectories per task, using terminal MSE below 10^{-6} and research confirmation below 10^{-7}. The initial task allowance is 384 observations, 4,608 measurement units, 360,000 charged computation units and six research rounds. Retention continuations use separate one-task sessions in every arm. The online finite-language study permits 24 observations and uses MSE below 10^{-10} for both fresh and terminal tests. Temporal and growth studies retain their own stated thresholds and ceilings in the appendices; counts from these contracts are not pooled.

Resource use is a vector: acquired response observations, requested measurement units, charged program computation, measured CPU or execution time, and model calls/tokens where applicable. Charged computation is an implementation-specific ledger, not FLOPs or elapsed time. Candidate evaluation, fitting and screening are charged according to the frozen runner's operations; Python orchestration, lookup and some host work are represented only in timing. For example, response fitting charges candidate-row and fitted-column work. A learned call actually executes its body, but its main scoring charge is a candidate-row pass, not the body's expanded primitive count. Scoring recalled bindings is charged; pure library lookup and general host bookkeeping are not uniformly priced. Growth studies separately report terminal primitive steps. The precise contracts and code locations are given in Appendix B.1. Research costs and terminal evaluation costs remain separate, and unsuccessful attempts remain in total expenditures.

A transfer saving does not imply that learning has already paid for itself. For a resource with acquisition cost C and comparable positive per-task saving Δc, a break-even count would be $C/\Delta c$. This calculation is meaningful only within the same resource and workload. We do not trade model tokens against observation rows or infer end-to-end runtime from charged operations. The shared-prefix interventions include the same acquisition cost on both sides; their absolute downstream differences survive that inclusion. In the selective-method study, both programs solve 56/56 targets. The two relevant author phases, including the unsuccessful predecessor and new counterexample, cost 118,797 charged units; mean subsequent saving is 7,407.21 per target. Repeating that fixed task mixture would cover these incremental charged units after about 17 targets. Observation savings are negative, measurement savings are zero, and transfer uses no model tokens in either arm. This conditional projection therefore supplies no overall or full-history break-even claim.

5. Results

5.1 Restoring distinctions and returning supporting evidence

Online support and strong flat controls. In 30 source chains, three researchers receive the same acquired state for four separate continuations: same or changed response structure crossed with same or changed surface. Surface changes rename and permute controls and alter calibration. All four response primitives remain public. Recursive, flat active and frozen-method-with-fallback research each solve 120/120 targets, with no false confirmation. The recursive method rewrites 47 experiments before execution and uses retained definitions on the 60 same-structure targets, as do both controls.

The resource tradeoff is mixed. Recursive and flat research use 852 versus 647 observations, and 388,496 versus 735,048 charged computations. The former buys more evidence while scoring fewer candidate-action combinations. In a separate temporal comparison,

matched instrument candidates, response fits, probes and ceilings give 60/60 correct targets for both separately confirmed child research and a flat controller. For recurrent cases, flat uses 159 rather than 163 observations and 45 rather than 57 measurement units per target, but fits three instrument-conditioned responses rather than one. These are strong local null results for an accuracy advantage from explicit child organization.

Measurement and model-class revision. In a separate joint-growth intervention, access to an informative measurement channel gives correct terminal predictions in 30/30 cases, versus 0/30 for controls alone. The latter still produce 30 internal confirmations of stochastic descriptions, with mean terminal MSE 0.1442. Thus a researcher can confirm an uncertainty description while missing an available explanatory variable. The added channel costs a mean 47.2 observations, 240 measurement units and 50,656 charged computation units per target. This contrast tests the value of observable information, rather than a different fit to identical inputs.

Scope revision provides a related test: learned conditions yield 29/30 terminal successes versus 0/30 for the frozen old response. In the 27 cases with an identifiable conditional branch, applying the new branch everywhere also fails every case. The successful explanations need both the earlier behavior and its revised scope. One learned scope passes internal confirmation but fails terminal evaluation.

The same campaign also exercises expansion and simplification. On the stochastic task, all 30 runs introduce a variance model and 19 select a conditional one; variance mean absolute error falls from 0.01461 for the constant model to 0.01103. When the task's true response becomes linear, both simpler and dense models solve 30/30, but terminal prediction uses 69,120 versus 194,560 primitive steps. All 30 subsequent nonlinear returns restore a supported, correct response. The latter comparison concerns execution of the final predictors, not total research cost. Interval calibration and the limits of uncertainty identification are reported in Section 6 and Appendix H.3.7.

Complete common-model comparison. Ordinary Code/ReAct, textual memory and Hegel share a 24-observation, 16-call and 300,000-reported-token allowance per task, the same public laboratory and the requested provider model alias `gpt-6-luna` at medium reasoning. Token limits are checked before the next call and can be exceeded by that call. Model identity here is the logged request alias, rather than an independently verified backend architecture.

Table 4. Complete agent outcomes. Each arm contains thirty registered source–target chains. Service and budget failures remain in the denominator.

Knowledge condition	Source success	Target success
Ordinary Code/ReAct	29/30	29/30
Textual memory	30/30	29/30
Hegel executable research knowledge	26/30	26/30

Every submitted predictor passes independent terminal evaluation. Failures involve service interruption, token exhaustion, or consuming the observations required for confirmation. Hegel makes 36 inherited experimental-program calls, including 15 with unchanged source, but completes fewer tasks overall. The experiment uses 894 model calls and at least 20,395,181 reported tokens; 70.3% of known input tokens are cached. Table 4 is the direct

answer to whether the mechanisms improve a complete agent on this task family: they do not.

The available four-primitive environment is nearly saturated by ordinary code investigation. Harder temporal development tasks expose different failures: an agent can buy an informative instrument observation without analyzing it or revising the parent. Executable post-acquisition analysis repairs this in individual traces, while persistent-advice comparisons still tie at 4/6 attempted targets in each condition. These records are retained in Appendix F rather than combined with the 30-chain cohort.

5.2 Research procedures can be revised and transferred

Knowledge about investigation. Deleting only the acquired procedure book in the joint-growth study, while retaining the same predictive program library, leaves accuracy at 30/30 in both conditions. Keeping the procedures reduces successor observations from 5,088 to 4,028 and charged computation from 5,219,559 to 3,944,154; both use 7,200 measurement units. The paired mean observation difference is -35.33, interval $[-45.47, -25.20]$; the computation difference is $-42{,}514$, interval $[-49{,}246, -35{,}687]$. Reused steps generate fitting questions for the current instrument rather than copy its earlier index. This intervention separates knowledge about investigation from the availability of response programs.

Across the full eight-task sequence, persistent research passes fresh confirmation and independent evaluation on 232/240 tasks versus 215/240 for equally expressive task-local research. Charged computation falls 22.42%, while CPU rises 17.92%. This complete-system comparison supplies context for the procedure intervention (Appendix D).

Revising a procedural assumption. The LLM author receives an actual failure, edits a Python procedure and practices it on source conditions. Published source is frozen before evaluation on other archives. Consumers use the ordinary `code_react` Python executor and the same instrument, fitting, experiment and confirmation operators, with no target-time model calls. The intervention tests the learned control flow of an executable procedure.

Table 5. Separate method-revision cohorts. Each row has a different protocol and target set; the rows are not a single success curve or a pooled estimate.

Failed procedural assumption	Executed revision	Frozen target result
A supported instrument suffices for the response	Preserve the instrument; refit and test the parent response	Old 10/18; revised 18/18
All response records share the current mechanism	Fit on current instrument-supported records	Old 20/32; revised 32/32
New-case repair preserves prior abilities	Test old prediction, then retain or reopen its support	Initial 36/44; corrected and refit-both each 56/56

The first author makes one edit and three source practices in five calls. All eight failed response-only transfers are repaired and all ten earlier successes survive. Transfer observations fall from 2,928 to 2,544 and measurements remain 3,888, while charged computation rises from 183,048 to 200,712. The selective revision illustrates the need to test preserva-

tion: its initial program repairs twelve new cases but breaks eight older ones. The corrected method preserves all 56 target outcomes; relative to refit-both, it reduces charged computation by 35.48%, leaves measurements unchanged and adds 384 observations. Appendix F retains unsuccessful author phases and acquisition costs.

Selecting and transferring an earlier method. Twenty author attempts resume one regressed state with host fallback disabled. With supported history, all ten authors select the old source and eight publish through a six-case practice gate; without it, two of ten publish. Seven publications exactly reproduce the earlier source, and the three others contain changed code. The study uses 123 model calls and 5,405,786 reported tokens, including failed authors. It measures local selection and publication on one state; available practice histories also change the future confirmation streams.

Deduplicating those ten publications yields four distinct programs. Each frozen program and an existing fixed repair procedure receive the same 24 targets: three archived donor states crossed with eight response, instrument, combined, delayed-change and old-condition-return conditions. One donor participated in authoring and two did not. All four publications pass fresh confirmation and terminal prediction on 24/24 targets with no false confirmation; the fixed repair passes 21/24, failing after late response changes. None of the three edited sources improves accuracy over the unchanged supported program. The publications use 277,728–280,032 charged units and 33.65–35.24 CPU seconds, versus 115,893 and 18.23 for the less successful fixed repair. These are paired executions on 24 target contracts, with zero target-time model calls (Appendix F.12).

From a structural proposal to a revised experiment. A simulated crystal-control fixture makes the division of work observable. Six LLM continuations share one failed state with 184 observations, nine inherited declarations and prior refutations. Changing only the research assignment from an open explanatory request to executable-basis submission changes completion from 0/3 to 2/3. Each successful author proposes signal, squared-signal and cubed-signal terms in one model call. The first fitted prediction, 7,715.405 Hz, is physically refuted. The existing numerical researcher then selects cubic-only support, revises coefficients and the activation interval, and requests 7,905 Hz. Activation occurs at 7,906 Hz during adjustment: the task completes, but the requested point remains unconfirmed. Across all three attempts per assignment, open versus executable delegation uses 30 versus 11 model calls, 120 versus 225 world actions and 106,084 versus 933,930 charged units. A separate zero-call replay reusing earlier family exclusions preserves completion while reducing actions from 99 to 35 (Appendix F.13).

Autonomous initiation is tested separately from that assigned continuation. Starting from zero observations and the same nine-declaration archive, both Hegel and Code/ReAct independently adopt the shared program. Changed-mechanism completion is 3/3 versus 2/3, and both confirm a newly proposed target point in 2/3. Hegel proposes a cubic-only basis in all three runs, using 24 model calls per success; successful Code runs each use two calls and a composite basis that the shared researcher simplifies. Both fail all three aliased-measurement controls (Appendix F.14).

Limits of autonomous procedure use. Follow-up live cohorts fail to establish a stable Hegel advantage: changed-mechanism completion is subsequently 1/3 versus 2/3, then 2/3 versus 2/3 for Hegel and Code. Connecting author continuation, retained exclusions and physical trials also regresses Hegel completion from 3/3 to 1/3 in a separate compari-

son, despite gaining one requested-point confirmation. Restoring the research assignment on continuation later increases new-basis submission from 1/3 to 3/3 and completion from 1/3 to 2/3, with one confirmed requested point. Appendix F.15 retains every intervening cohort, cost and failure. Together, the traces establish executable proposal and counterexample-driven revision; autonomous research allocation and an advantage over the shared-program Code agent remain open.

5.3 An observed-failing program can remain useful

Each apparatus first learns a conserved response A, an event response B_r and their composition $C = A + 0.7B_r$. One paid 24-observation challenge tests the frozen acquired C after the event boundary changes. All 30 challenges refute it. Before any replacement is learned, three continuations branch from the same prefix: conditional retention, retention without the challenge's conditional feedback, and removal of C, its 17 dependent abstraction/definition entries and its meta-prior. Ancestors A and B_r, generic methods and other research experience remain. All arms may subsequently learn and confirm a replacement.

The three successor tasks are the changed mechanism, the original-condition return, and the changed-condition return. The intervention therefore concerns **90 continuation tasks per arm**; another 90 prefix tasks are shared. The prespecified primary contrast is the original-condition return, not the first changed task.

Table 6. Retention after refutation. Terminal accuracy by phase, followed by complete-sequence charged resources. Shared prefix and challenge costs are included in each arm's sequence totals. Conditional and unmarked retention have identical terminal errors and charged costs in every paired continuation.

Outcome	Conditional retention	Unmarked retention	Global discard
First changed task	30/30	30/30	10/30
Original-condition return (primary)	30/30	30/30	30/30
Changed-condition return	30/30	30/30	29/30
All continuation tasks	90/90	90/90	69/90
Full sequence, including shared prefix	180/180	180/180	159/180
Research observations	17,987	17,987	22,484
Measurement units	73,680	73,680	120,864
Charged computation	8,553,189	8,553,189	11,651,123

All three arms succeed on the original return, but retained responses use 24 observations per return versus 40 after deletion. Return computation totals are 30,930 versus 138,476; the paired mean difference is −3,584.9 units, with a 95% apparatus-bootstrap interval of $[-3{,}868.0, -3{,}273.9]$. Thus the primary endpoint supports easier recovery at equal accuracy. The accuracy difference occurs mainly on the first revision. Of its 20 deletion failures, 16 remain unresolved and four pass internal confirmation but fail terminal evaluation. These failures retain substantial numerical budgets; the stopping analysis below examines their actual control-flow cause.

A replay of all twenty first-revision failures with the original frozen runtime reproduces their statuses, costs, confirmation losses and terminal errors. The sixteen unresolved cases stop at `relation_ambiguous`: fourteen after the first confirmation attempt and two after the second, below the six-round limit. Rival relation candidates still agree with the short confirmation. Fifteen runs fail to generate a predicted distinguishing probe; one exhausts a local 6,000-unit probe-generation cap while retaining global resources. The four false confirmations instead terminate normally after accepting a candidate that later fails terminal evaluation. No session revisit occurs. Thus the loss is mediated by the available relation search and stopping policy, rather than depletion of the overall allowance. Another random acquisition and candidate comparison would fit the remaining budgets, but its ability to repair these cases has not been tested. Appendix H.3.4 records the replay.

Across the complete sequence, retention reduces observations by 20.00%, measurements by 39.04% and charged computation by 26.59%. The computation difference per sequence is −103,264.5, interval [−132,874.2, −78,700.7]. Conditional-retention CPU time is nearly equal to deletion: the paired difference is −0.62 seconds, interval [−12.93, 10.79]. These intervals resample complete apparatuses, stratified by the two generator profiles, with 10,000 paired resamples.

The conditional marking adds no measured benefit over unmarked retention. Moreover, unmarked retention uses the same researcher and library machinery; it is not a general archive baseline with the entire support organization removed. The result identifies a benefit from retaining executable knowledge under this controller, not a unique benefit from attaching conditional semantics.

Three checks distinguish inheritance from a textual or syntactic trace. We inspect exact called definitions, test the repaired response on independent trajectories, and intervene on availability. A direct call proves that code was executed; an algebraically expanded equivalent could preserve the same meaning without that call. In a separate component-change study, a gain change replays a method using A and B without calling C, whereas a boundary change retains C inside a residual correction. Both and the unchanged condition solve all 60 successor/return tasks in each branch. These observations establish usable preservation without treating a particular syntax as indispensable.

The complete four-algorithm composition study is supporting evidence: full research solves 180/180 versus 179/180 for task-local research, with 76.25% less charged computation. Memory and unused sequence-resource allocation both change in that contrast, so it is a complete-system comparison. The dense fixed-basis control's 0/180 result is an implementation diagnostic: it fits 185 coefficients from 48 observations and even misses a representable linear task. It is not the principal competitive baseline. Appendix H.3 gives the full comparisons and failures.

Revising a condition while retaining its response program. A final offline intervention tests the corresponding distinction between a fixed numerical law and its activation interval. Reconstructing intervals from all final paid source observations preserves 15 of 21 saved coefficient programs across three development branches; six remain inconsistent. Compatible old models are retained, and target observations do not enter source reconstruction. Two new object pairs crossed with unchanged or 1.01-times source gain produce twelve paired successor tasks, with the same controller and inherited archive in both arms. Adding the reconstructed source models leaves completion at 12/12 and reduces research

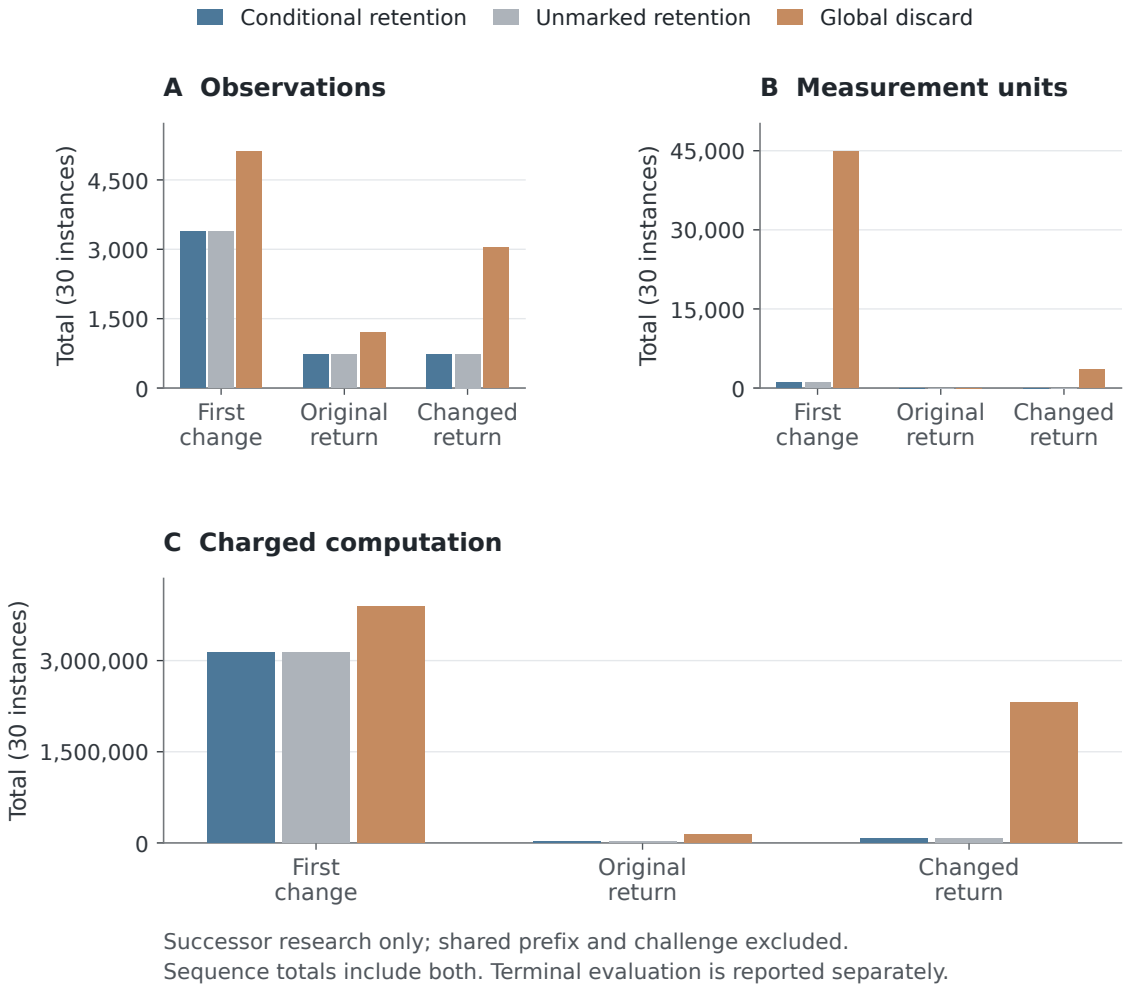


Figure 3. Where retention changes research expenditure. Each panel shows the three successor stages across 30 apparatuses. The shared acquisition prefix and common challenge are excluded from these bars and included in Table 6. Retention and unmarked retention coincide. The primary original-return accuracy is tied; its observations and charged computation decrease.

computation from 798,735 to 702,817 (12.0%); actions change from 775 to 774. Requested-point confirmations for the two successive target crystals, C4 and C5, change from 10 to 10 and from 8 to 9, respectively, with no unsupported positive verdict. Including source publication and terminal diagnostic exports gives a net reduction of 96,044 charged units.

The effect is conditional. C4's unchanged total contains one case-level gain and one loss. In the six changed-gain successors, the new knowledge yields no confirmed request, is eventually rejected during all six successor runs, and adds five actions and 10,928 research units. The added C5 confirmation occurs at unchanged gain, after a retained program guides a further interval revision and a successful requested experiment. This zero-model-call comparison tests source-condition consolidation on three development branches; it does not establish new structural discovery or reliable autonomous applicability selection (Appendix F.17).

5.4 Downstream application: learned priors for constrained optimization

SC-KG supplies a decision-level application. A learned numerical program becomes a GP feature: objective and constraint surrogates can use $(1, q)$ and $(1, p)$ instead of a fixed coordinate-quadratic basis. Residual discrepancies can reopen research. This is bounded numerical program search, using a three-dimensional optimization domain, rather than a language-model agent or a test of the original very-high-dimensional motivation.

We rerun nine historical cases against the same historical SC-KG backend and 32-observation prefix, followed by 32 optimization calls. The shared prefix was originally selected by the Hegel researcher. The controls are native SC-KG and its full configuration combining low-frequency structure, orthogonality, sparsity and additivity. The original model-selection gate is retained. Frozen Hegel v0.53 research supplies the third system.

Table 7. Historical common-prefix SC-KG comparison. Lower terminal regret is better. Every final recommendation is feasible. The paired cases were already used in the historical Hegel evaluation, so this is a retrospective comparator addition.

Optimizer prior system	Mean terminal regret	Feasible recommendations	Hegel paired wins
Native SC-KG	0.100000	9/9	9/9
Historical full four-property configuration	0.129478	9/9	9/9
Frozen Hegel v0.53	0.017944	9/9	—

Native and Hegel exactly reproduce their historical endpoints. The full control selects its learned coordinate model at both recorded initial and final gates in every case; enabling four properties does not mean that all four were deployed. The three mean regrets cover stable conditions, a moving optimum and joint center and objective-interaction change. Optimizer wall times are 24.40, 624.82 and 26.56 seconds respectively; the full control adds 8.45 seconds of preparation. Hegel's historical prefix-research cost is not included in these rerun times, so they are not end-to-end learning-time comparisons.

Appendix G preserves two important boundaries. An earlier same-paid-data feature comparison found no independent gain from the learned prior, and a later component revi-

sion improved calibration without changing recommendations. The current nine-case win is a complete-system result on a low-dimensional, shared-prefix contract. It does not isolate recursion, or reproduce the full prior's original independent-source high-dimensional training setting.

5.5 From an authored representation to live method reuse

From explanation to an executable method. An anonymous apparatus exposes resettable control trajectories and optional measurements. Agents receive raw histories and an ordinary Python interface. In four source investigations on two recurrent parameter settings, Hegel solves 1/2 and Code 0/2. The successful Hegel acquires measurements, identifies a recurrent intermediate variable, combines it with a quadratic response, repairs an omitted input, and submits a predictor using controls and their history alone. Its terminal MSE is 1.52×10^{-30}. The exported representation predicts the held-out trajectories accurately; its latent coordinates need not be the simulator's unique physical state.

A separate Luna author receives all four investigations, including failures, and writes a reusable fitting procedure. Seven author calls and eight cumulative practices yield a published source that passes all four required practice settings. The program first evaluates archived predictors against the current evidence. When none fits, it estimates a latent recurrence and a quadratic readout from the new apparatus's paid observations. What transfers is a procedure for identifying a model, with target fitting costs, rather than source coefficients assumed to apply everywhere.

Frozen transfer and live use. The unchanged published procedure passes confirmation and terminal prediction on six new parameter settings without target-time LLM calls. Every target follows the same informative sequence: a model fitted to 72 short-path rows fails a 96-row long-path confirmation; refitting the same family on the accumulated 168 rows then passes a new 96-row confirmation. Thus each target uses 264 response observations and no purchased instrument readings. The fixed confirmation contract supplies these long trajectories. During frozen execution the procedure executor resumes the program; during live inquiry the failure returns to Luna, which must choose the next action.

Table 8. Frozen recurrent-method transfer and live adoption. Passed means both fresh confirmation and terminal prediction pass. Original live and development rerun use the same two target settings and learned program; the rerun restores registered method information after failed submission. Costs include each episode's initial 72 rows and unsuccessful work. These are distinct execution conditions, not independent replications or a pooled success trend.

Execution	Agent	Passed	LLM calls	Response rows	Python seconds
Frozen targets	Published procedure	6/6	0	1,584	631.4
Original live	Code	2/2	10	528	216.7
Original live	Hegel	1/2	28	840	204.3
Development rerun	Code	1/2	13	624	866.4
Development rerun	Hegel	2/2	4	528	670.4

The original live comparison exposes the difference between executable competence and use: Hegel abandons the available method after one failed fit in one target, whereas Code reuses it and succeeds. A trace audit finds that Hegel's temporary submission manifest loses the registered invocation limit and source-practice record after failure. Reusing the existing metadata merge restores them in the live decision context; feedback also distinguishes predictive rejection from unscored execution failure. In the resulting developmental repeat, both Hegel episodes explicitly invoke the unchanged program again, refit on the counterexamples and pass terminal evaluation, using two Luna calls each. This is an observed continuation decision by the model, with no run-time human intervention or automatic live retry.

Code's unsuccessful repeat first obtains a paid-data fit of 6.85×10^{-30} but only prints its parameters. After additional acquisition, its second method invocation exhausts the 600-second Python allowance before producing another candidate or confirmation. This is an execution-censored outcome, not a second predictive refutation. Appendix I reports the complete source, authoring, transfer and continuation costs. The connected record shows a learned representation becoming a reusable identification method and subsequently guiding live repair; the interpretation of organization and generalization is addressed in Section 6.

6. Discussion and limitations

The framework connects abstraction to the experiments that give it empirical meaning. A hypothesis specifies which differences matter; an observation determines which of those differences can be exposed; recursive supporting research revises the connection when it is uncertain. The experiments instantiate parts of this cycle: calibration changes an unexecuted probe, measurement access resolves missing information, and procedure revision changes what later research does. The recurrent study connects another pair of levels: an acquired representation supports a method for identifying new instances, and counterexamples to an instance become inputs to that method. Retention makes these revisions cumulative by preserving earlier computations without preserving their refuted applications.

This perspective also connects several familiar research settings. For a reinforcement-learning agent, a state representation and transition model are hypotheses about what suffices to predict consequences. Robust modeling expands the considered family of consequences; simplifying assumptions restrict it. Causal inquiry asks which interventions distinguish competing explanations, and representation learning changes the variables in which those explanations are expressed. These are connections within the proposed framework, rather than additional evaluated domains. Appendix H.1 develops them and the philosophical motivation: a more general explanation can preserve the insight of earlier ones by identifying their valid conditions.

Attribution. The retention intervention identifies value in executable knowledge availability. Conditional marking ties unmarked retention; strong flat research ties explicit recursion in accuracy; and frozen Python method transfer already runs in an ordinary code archive. The recurrent follow-up supplies both live arms with the same executable archive and learned method, but the source agents do not adopt its optional child/component interfaces. Its successful source explanation is composed in a monolithic Python predictor. The two-target continuation repeat changes method-information plumbing and failure

feedback together on already studied cases. It neither isolates either repair nor estimates an advantage from recursive organization; the developer's repair is not an autonomous agent improvement. A prospective comparison across independent donors that differs only in support and recursive organization remains absent. The present result is therefore a framework with demonstrated mechanisms, not proof that its organizational vocabulary is uniquely necessary.

Finite discovery and transfer. The finite numerical studies supply operators, candidate families and measurement interfaces. In the anonymous Python study, Luna authors the recurrent representation and response program within a supplied experimental interface and numerical library; the author receives prior investigations but no host-provided candidate menu for the new apparatus. The functional-analytic construction organizes the hypothesis–observation relation; it does not replace the study-specific search procedures or provide a general discovery guarantee. The implementation does not learn arbitrary infinite-dimensional spaces, new sensor primitives or a general topology from data. Some dependency graphs are compiled after fitting, and only the online experiments establish pre-action support return. New parameterized programs and changed action-selection code extend accessible research within those interfaces. A useful abstraction can be expressible in the old primitive language while changing what bounded research can accomplish. Learned models used as prior features, revised procedures and experience-dependent method preferences do not by themselves establish invention of a new cross-task structural principle. Unrestricted experimental-primitive invention, cross-domain abstraction discovery and general recursive self-improvement remain open. The recall cohort has one authoring state; additional target executions do not turn it into multiple independent method discoveries. Likewise, the recurrent method has one author and six new-parameter targets in the same simulator family. Its latent parameterization is not uniquely identified, and successful parameter refits do not establish when the method family itself should be rewritten. Source discovery, instructed method consolidation, frozen execution and live continuation are separately initiated stages, not a single autonomous agent launching its own successive learning phases. The executable-delegation study also reuses one previously studied state: its two rediscoveries and model-free execution repair establish local adoption and completion, while automatic initiation, independently confirmed response laws and unseen-task discovery remain untested by that comparison.

Task-dependent simplification. The simplification study changes the true response to a linear law and later restores the nonlinear law. It tests adaptation to a simpler response and recovery of an earlier model. It does not test coarse-graining one unchanged complex environment because a different task or scale makes some distinctions irrelevant. Learning that cheaper effective model together with an empirical boundary for its approximation would more directly address task-dependent idealization.

Statistical support and confirmation. Fresh trajectories separate candidate selection from that individual check, but thresholds are not calibrated significance levels. Repeated revision and testing have no formal sequential type-I error guarantee here. False confirmations in the deletion, passive and growth studies show that limited confirmation coverage matters. In particular, improved variance prediction does not establish calibrated uncertainty: nominal 95% intervals cover 91.09% of fresh stochastic responses, below the constant model's 92.81%. Controls-only confirmations also fail to detect missing explana-

tory measurements. The current uncertainty results do not establish robust decision guarantees. Bootstrap intervals concern the stated apparatus distribution; they cannot turn related tasks or repeated authors into independent samples. Method-development cohorts are reported separately from frozen transfer, and the historical SC-KG comparison is retrospective. The recurrent continuation repeat reuses two already opened targets and changes the interface between runs; its 2/2 Hegel result is descriptive development evidence. Its Code timeout is retained as a failure to finish under the execution contract, rather than as a measured predictive error.

Cost and autonomous control. Charged computation omits host work and can fall while CPU rises. Learning and unsuccessful author attempts must be paid for before claiming long-run savings. No stable open-ended net-benefit claim follows from these sequences. Complete agents still fail to use available procedures or to finish valid confirmation, and the main common-model comparison favors the ordinary controls. Model-request aliases and service outcomes also limit extrapolation to other providers or architectures.

The next discriminating experiment is a prospective method-transplant comparison across genuinely different donor failures, retaining failed donors and holding initial archive contents, numerical operators, experiment permissions and budgets fixed. It should compare support-organized research, a strong ordinary executable archive, and a condition with frozen research methods but the same predictive knowledge. Each researcher may choose different experiments: equality concerns access and evaluation, not the observations it decides to acquire. Tasks should include surface changes that preserve structure and superficially familiar changes that invalidate it. The decisive trace links new supporting evidence to an actual change of research action and then to independent target outcomes or a measured resource tradeoff. That experiment would test the remaining organizational claim.

7. Conclusion

Hegel Machine treats predictive explanations, their supporting assumptions and the procedures used to investigate them as revisable hypotheses. Its functional-analytic perspective pairs a space of possible explanations with the observations that make their differences visible. Recursive research can revise either side: a child answer changes the parent's experiment, a new measurement enriches an inadequate representation, and a failed method becomes a candidate for revision and transfer. Controlled studies demonstrate finite instances and benefits from preserving earlier executable knowledge. An authored recurrent representation also becomes a transferable fitting procedure, and live agents can reuse it to revise a failed prediction with newly acquired evidence. General abstraction invention, a unique advantage of recursive organization and overall autonomous-agent superiority remain open. The resulting framework makes those broader claims concrete enough to test.

Experimental materials

Appendices A–I retain complete protocols, intermediate failures, detailed costs and the mathematical derivations in this document. The artifact index maps each study to its saved source snapshot, protocol, results and analysis commands, and distinguishes numerical replay from reauthoring with a language model. The distribution includes the English

Markdown, editable TeX, a unified PDF and a selected evidence archive. Tables 4–8 are regenerated directly from the included result JSON during manuscript construction, without model calls. The archive also supplies a runnable frozen numerical study and the four published programs with their 24 transfer targets. Replay dependencies for the other studies are identified in the index. The local review revision is `paper-2026-10-12-v3`; it adds the recurrent method study to the preserved `paper-2026-10-11-v2` evidence. Its selected records and source snapshots preserve the reported study protocols. Public archival deposition remains a publication step.

Appendix A. Observable geometry and representation sufficiency

A.1 Additional experiments refine observational equivalence

Let E be a nonempty finite experiment set. For each e, hypotheses h and g predict probability distributions P_h^e and P_g^e on the same outcome space. Write $h \sim_E g$ when these distributions agree for every $e \in E$. For weights $w_e > 0$, define

$$d_E(h, g) = \sum_{e \in E} w_e \, \mathrm{TV}(P_h^e, P_g^e).$$

Each total-variation term is nonnegative, symmetric, and satisfies the triangle inequality. Their positively weighted sum is therefore a pseudometric. It is zero exactly when the predicted distributions agree for every experiment in E, so it defines a metric on the quotient by $\sim_E$.

If $E \subseteq E'$, then $h \sim_{E'} g$ implies $h \sim_E g$ by restriction. Thus every E'-equivalence class is contained in an E-equivalence class. The refinement is strict if some newly available experiment distinguishes a pair that was equivalent under E. This statement concerns predicted distributions and available tests; finding a useful additional experiment remains a research decision.

For deterministic point predictions, $\mathrm{TV}(\delta_a, \delta_b) = \mathbf{1}[a \neq b]$. Thus this pseudometric encodes whether an experiment can distinguish two predictions, not the magnitude of their difference. The numerical disagreement and MSE scores used by the controllers are separate quantities.

A.2 Representation aliasing creates a fitting error floor

Suppose two deterministic cases have the same representation, $\phi(x_1) = \phi(x_2)$, but responses $y_1 \neq y_2$. Any predictor depending only on that representation must output the same value a on both. Its average squared error satisfies

$$\frac{(a - y_1)^2 + (a - y_2)^2}{2} = \left(a - \frac{y_1 + y_2}{2}\right)^2 + \frac{(y_1 - y_2)^2}{4} \geq \frac{(y_1 - y_2)^2}{4}.$$

Refitting such a deterministic predictor cannot remove the error. A representation containing a relevant distinguishing observation could do so. In a stochastic setting, repeated outcomes can instead support an uncertainty model; the pair alone does not identify the cause. These elementary observations motivate alternative research actions without supplying a general discovery guarantee.

A.3 Continuous observations, invisible directions and the observable quotient

Let $\mathcal{E}$ be a nonempty experiment domain and $V \subseteq B(\mathcal{E}, \mathbb{R})$ a real linear subspace with the supremum norm. Each point evaluation δ_e is linear and satisfies $|\delta_e(f)| \leq \|f\|_\infty$, hence belongs to the continuous dual V^*. More general bounded linear observations can also be used. No completeness assumption on V is needed here. The functional-analysis terminology follows Melrose (2020) and Zsák (2023).

For nonempty finite $Q \subset V^*$, define $p_Q(f) = \max_{q \in Q} |q(f)|$ and $K_Q = \bigcap_{q \in Q} \ker q$. Linearity implies absolute homogeneity and subadditivity:

$$p_Q(af) = |a|p_Q(f), \qquad p_Q(f+g) \le p_Q(f) + p_Q(g).$$

Moreover $p_Q(f) \le (\max_{q\in Q} \|q\|)\|f\|_\infty$, so it is a continuous seminorm with kernel exactly K_Q. If $k \in K_Q$, every $q(f+k) = q(f)$, and therefore $p_Q(f+k) = p_Q(f)$. The induced quantity $\|f + K_Q\|_Q = p_Q(f)$ is well-defined on V/K_Q and is a norm: its only zero class is K_Q. Equivalently, the evaluation map $T_Q f = (q(f))_{q\in Q}$ identifies this quotient with its image equipped with the maximum norm. This is the norm induced by the observations; it need not equal the usual quotient norm from $\|\cdot\|_\infty$.

For $Q \subseteq Q'$, intersection of the additional kernels gives $K_{Q'} \subseteq K_Q$. Inclusion is strict precisely when some $v \in K_Q$ has $q'(v) \neq 0$ for an added observation. This proves that an informative new observation can resolve an old invisible direction. If the hypothesis set $\mathcal{H}$ is nonlinear, its embedded response functions can still be compared through $p_Q(f_h - f_g)$ in the ambient space. Strict refinement of V/K_Q need not separate a pair in the particular current candidate set: that requires the invisible direction to be a difference $f_h - f_g$ for those candidates.

For arbitrary nonlinear observation maps F, define the signature $S_Q(h) = (F(h))_{F\in Q}$. Equality of signatures is an equivalence relation. Pulling back a metric ρ gives $d_Q(h,g) = \rho(S_Q(h), S_Q(g))$, a pseudometric whose quotient identifies equal signatures. A numerical tolerance relation $d_Q(h,g) \le \epsilon$ need not be transitive and is not an exact equivalence relation. Squared prediction error, conditional variance and arbitrary constraint violations are not automatically norms or linear functionals.

The construction is an organizing model of experimental distinguishability, not a theorem of discovery or a claim that every current score implements functional-analysis operations. Adding an experiment supplies evidence; changing an instrument interpretation additionally changes the hypothesis-to-observation map. Such comparisons require common physical experiment semantics, as in the calibration trace in main-text Figure 2. Internal research choices may change, while the independently specified terminal evaluation remains fixed.

Appendix B. Implementation and experimental material

The current implementation is organized around executable programs, persistent knowledge, research procedures, the research controller, and task sequences and revisits. The object laboratory provides the composition study's interface.

The SC-KG bridge uses program compilation and optimizer initialization and objective research. A compact evidence extraction connects Appendix G to the frozen predictive, same-data optimization, decision-confirmation and deployment-return records. Its version labels identify those experimental snapshots; current source contains subsequent changes.

The following material contains the exact protocols, per-instance outcomes and frozen runtime snapshots. The first three studies are supporting mechanism cohorts. Later cohorts were registered before execution; the final product-removal intervention is explicitly post-hoc on previously observed continuation cases.

Study	Protocol	Recorded results
Single source	Protocol	Summary
Two sources	Protocol	Summary
Composition and local revision	Protocol	Summary
Four-arm composition comparison	Protocol	Summary
Retention after an observed failure	Protocol	Summary
Different component changes and no change	Protocol	Summary
Joint research-state growth	Protocol	Summary, derived joint outcomes
Procedure integration	Protocol	Summary
Demand-conditioned procedure choice	Protocol	Summary
Complete research budgets	Protocol	Summary
Paid-demand continuation	Protocol	Summary
New-product contribution (post-hoc)	Protocol	Summary

The joint-growth frozen source bundle contains the actual runner and runtime used for the 30-instance campaign. Current entry points are joint research, expression search, procedure compilation, and variance learning. The separate post-run analysis derives confirmation/evaluation intersections and prefix-inclusive costs directly from the saved case files; it does not alter the frozen protocol or fitted models.

The composition study's procedural failure is analyzed in the instance 26107 diagnostic. Earlier development cohorts remain available in the corresponding artifact directories. The prospective experiments are specified separately in the research plan.

B.1 Cost contracts and incremental amortization

Charged computation is a proxy defined by the frozen operators. In the retention runtime, scoring charges candidate-row passes; `_fit` charges $n(k+2)$ for n rows and k fitted terms. Growth also charges validation rows. Proposal generation, role binding, dependency screening and planning consume their explicitly assigned search-budget units. Sub-budgets return their actual expenditure. These rules do not count every linear-algebra instruction or every Python operation.

A learned program call executes its body, but the main prediction score charges a row pass rather than the expanded body size. `Program.size` expands calls for complexity comparison; it is not a universal billing multiplier. Separate terminal execution diagnostics count primitive steps and call/cache events. Library lookup, sorting, serialization and general host work have no blanket charge; evaluating recalled bindings does consume charged row passes, while fit-cache reuse avoids another fit. Process CPU, Python elapsed execution and provider wall time remain separate measurements.

Model calls include attempted requests that return errors. Reported token totals add input and output, with cached input a subset of input rather than a subtraction. Unknown usage is not treated as zero. Artifact records identify where totals are lower bounds. Object/growth CPU uses process time, whereas method execution seconds measure monotonic

elapsed time. These contracts explain why a charged-computation reduction can coexist with higher CPU.

The selective-preservation comparison has equal 56/56 success. The unsuccessful predecessor-author phase, successful revision phase and newly purchased counterexample together cost 118,797 charged units. The revised frozen program saves 414,804 units over the 56 targets, or 7,407.21 per target. Under repeated use of the same task mixture, these incremental units would be covered after 16.04 target equivalents, hence 17 whole targets. This is a conditional projection, not a measured long-run repayment trajectory. Earlier common source acquisition and older development remain separately recorded sunk costs. Observations increase by 6.86 per target; measurements and target model calls/tokens do not decrease, so those resources have no demonstrated break-even. Timing observations do not establish end-to-end payback.

The cost-contract audit provides original source locations and the resource-specific calculation. The artifact index separates table regeneration, numerical replay and new model authoring.

B.2 Where the hypothesis–observation relation is implemented

The functional-analytic perspective has finite operational counterparts in the existing code. The online study uses its frozen researcher; the growth study uses its frozen researcher, response models and uncertainty models. These are the study implementations, rather than retrospective replacements with current source.

Table B1. Finite implementations of observable distinctions.

Operation	Executed mechanism	Interpretation
Identify indistinguishable candidates	`response_keys` groups predictions over the online laboratory's complete supplied response-action grid	A finite observable signature; this grouping uses the entire supplied response grid, not only measurements already acquired or every executable continuous probe
Choose a distinguishing experiment	`action_gain` partitions predicted outcomes and scores reduction in uncertainty over candidate classes	Experimental value depends on which distinctions the candidate action exposes
Revise a pending parent probe	`adaptive_method_action` consumes calibration and coordinate answers before selecting the response action	Revising a supporting interpretation changes the useful observation
Change an input representation	The growth researcher purchases candidate channels and compares projected pilot records and validation fits	Acquire information absent from controls alone, then test its predictive use
Expand or simplify a response class	Repeated controls motivate variance models; supported complex programs compete with simpler affine fits	Finite selection among deterministic, conditional and uncertainty descriptions

In chain 79101013, returned calibration and coordinate answers make the old probe $(-0.6, -1)$ evaluate the relevant response at $z = -1$. Quadratic and absolute responses then both predict 1. The revised probe $(-1, -0.6)$ exposes $z = -0.6$, at which they predict 0.36 and 0.6; the four supplied response alternatives all differ there. Predicted information gain increases from 1.03972 to 1.38629 before acquisition. This trace supplies a concrete instance of an observation recovering a missing distinction, with the child evidence actually consumed by the parent.

Related temporal code constructs probes from competing instrument predictions and realized trajectories: instrument hypotheses, temporal instrument research, and response experiments. Its squared disagreement scores are useful selection criteria, without implying that every score is a metric. These mechanisms precede the paper's mathematical restatement; no new general-purpose dual-space solver or additional experiment is introduced by this exposition.

Table B2. Control and learning in the reported implementations. The protocol supplies the initial interfaces; the controller or author below makes the recorded choices. These rows locate responsibility within each study.

Operation	**Who initiates or selects it**	**Object acquired or revised**	**Fixed components**
Supporting answer returned to a parent	Online numerical controller opens and resolves the child before the parent action	Instrument/ coordinate binding, surviving candidates and the next probe	Child-question types and 96-hypothesis language
Measurement revision	Growth researcher buys and compares public candidate channels	Measurement view and its fitted response program	Instrument catalogue, access prices and validation rule
Experience-dependent method preference	Numerical selector uses retained source-return evidence	Preference among existing methods for the current demand (Appendix E.1)	Matching rule, executable method repertoire and task limits
Executable method revision	LLM author edits source; host executes paid practice	Python action-selection control flow	Numerical operators, public interface and practice contract
Autonomous recall and publication	LLM selects a source or edit; host checks publication; automatic fallback disabled	Restored or revised procedure (Appendix F.11)	Six practice cases and publication criterion
Learned prior for optimization	Numerical program search supplies a feature to the historical optimizer	Response program used as a GP feature	Optimizer, model-selection gate and common-prefix contract

The three object levels in Section 3.1 have distinct evidence: p is evaluated by response prediction, π by executed research and frozen transfer, and preferences η by their effect on

candidate or method selection and subsequent outcomes. Appendix E.1 reports an actual selection change without a completion gain. Sections F and G separately establish procedure revision and the use of learned responses as optimizer priors. This mapping preserves both the changes that occurred and the outcomes they achieved.

Appendix C. Supporting mechanism studies

C.1 Single-source acquisition, failure, and return

The first sequence learns a response, reuses it, changes the mechanism, and returns to the original condition. Initial visits have caps of 96 observations, 384 measurements, 24,000 charged computation units, and one research round. The session can allocate unused resources to revisits within the summed sequence cap. All 12 instances satisfy the complete procedural criterion. Table C1 reports complete-sequence totals.

Table C1. Single-source study. Each arm is evaluated on 48 tasks. The fixed-library, passive, and memory-disabled arms are algorithm controls; source deletion is a separate intervention on a shared learned prefix.

Arm	Tasks passing	Observations	Measure-ments	Charged computation
Full	48/48	2,592	2,448	318,674
Delete learned source	48/48	2,592	2,448	323,966
Fixed library	48/48	2,304	9,216	20,736
Passive experiments	48/48	4,608	18,432	591,702
Memory disabled	48/48	2,736	1,296	549,219

Retaining the learned source saves 5,292 computation units, or 1.63% of the deletion arm's complete-sequence cost, with equal observations and measurements. The first successor uses 195 rather than 636 computation units, illustrating how a large local saving can become modest after acquisition and revision are included.

The fixed-library control provides an important comparison: it solves every task with much less computation, while requiring more measurements. It uses a predeclared quadratic and two-step-history basis. The passive control uses precommitted experiments and also changes adaptive revisiting behavior. The original memory-disabled control retains active research but sets `use_memory=False`, which also disables within-task instrument learning and changes the confirmation reserve. It is therefore a broader memory ablation. The task-local control in Appendix H.3.3 preserves these within-task capabilities. Their results establish a resource trade-off and distinguish the shared-prefix source effect from differences between entire research algorithms.

C.2 Two independent learned sources

The second sequence learns source A, learns source B, revisits A and B, changes the mechanism associated with B, and returns to A and B. The source tasks involve distinct control-history and delayed-measurement dependencies. The first two tasks naturally acquire the

separate sources. Budgets and terminal criteria match the single-source study. Four arms retain both sources, remove A, remove B, or remove both after the common two-task prefix.

Table C2. Two-source study. All arms use 5,328 observations and 5,184 measurements across 84 evaluated tasks.

Knowledge retained	**Tasks passing**	**Charged computation**
A, B	84/84	721,080
B only	84/84	727,908
A only	84/84	723,348
Neither	84/84	730,176

All 12 sequences satisfy the procedural criterion. The contributions of A and B are 6,828 and 2,268 computation units, respectively, and their sum is the observed 9,096-unit saving against deletion of both. The measured interaction term is zero. This study demonstrates separately useful retained knowledge; the next study tests two sources within the same response.

C.3 Composition and local mechanism change

The object environment exposes controllable object variables and coarse and fine measurement channels. Its response can combine a total-quantity component A with an event component B_r, where r controls an interaction radius. The combined target is

$$C = A + 0.7B_r.$$

The sequence is: learn the total component; learn the event component; learn their combined response; change the event radius to 0.45; return to the original combination; return to the changed combination. Initial visits have caps of 384 observations, 4,608 measurements, 360,000 charged computation units, and six research rounds; revisits draw on the remaining sequence budget. The source-deletion arm removes A after the shared two-task learning prefix while retaining B.

The learned combined program actually calls both sources. Under the mechanism change, the original combination encounters a measured counterexample. A successful revised expression preserves the exact definition of A and compensates for the changed event contribution, with the schematic form

$$D = C + 0.7B_{0.45} - 0.7B_r.$$

The experiment thereby tests preservation inside an executable composition, followed by return to both the original and changed conditions.

Table C3. Composition study. Resource totals cover 12 complete six-task sequences per arm, including the common source-learning prefix.

Arm	Tasks passing	Observations	Measurements	Charged computation
Retain A, B	72/72	7,002	29,484	3,415,926
Delete A	72/72	10,334	57,804	5,800,741

Both arms meet the same terminal accuracy criterion on every task. Retention reduces all three charged resources in every instance. The cumulative reductions are 32.24% for observations, 48.99% for measurements, and 41.11% for computation. The complete procedural criterion passes in 11 of 12 instances: six of six under the development profile and five of six under the confirmation profile.

The remaining instance separates program recovery from direct calling. On the final task, it uses the stored research procedure to recover exactly the same program body as the earlier revision, but does not call the new definition D directly. It uses 48 observations and 2,686 computation units; the other 11 retention instances use 24 observations and 2,575–2,583 computation units on that task. The retained transfer record and corresponding control flow indicate that a smaller response passed the short validation sample before being rejected on fresh evidence; this intermediate sequence is a reconstruction from the compact record. The subsequent method replay succeeds. This is a failure of the stipulated direct-call criterion and an instance of avoidable research cost, despite correct final prediction and actual procedure reuse.

Appendix D. Joint-growth protocol and operational scope

D.1 Response families and task changes

The joint study uses three finite response families,

$$f_1(x, y) = ax^2 + bxy, \qquad f_2(x, y) = ax^3 + bxy, \qquad f_3(x, y) = a|x| + bxy,$$

with apparatus-specific coefficients sampled in $a \in [0.8, 1.2]$ and $b \in [0.3, 0.7]$. Each apparatus belongs to one family. Thirty new seeds are balanced across the three families. The first surface implements the product directly; later surfaces implement it as $xy = ((x + y)^2 - x^2 - y^2)/2$ and use different anonymous control names, coordinate origins and units. A transfer must discover an empirically adequate binding. These are controlled algebraic analogs, rather than independent real-world scientific domains.

The eight-task sequence learns the source response, transfers it across a surface change, adds $0.8 \max(0, x)$ in one region, introduces an unobserved input, changes the instrument names and their ordering, introduces random response variation, changes the true response to $0.8x - 0.3y$, and returns to the original nonlinear law. The simplification intervention therefore changes the generating law, with the same predictor interfaces available for model comparison. In the hidden-input tasks, the response contains $z \sim \text{Uniform}[-0.6, 0.6]$. One auxiliary instrument measures an affine transformation of z and another measures an independent distractor. Both have the same per-row price. The later instrument task reverses the informative instrument's catalogue position, so replaying an old index cannot by itself

recover the appropriate view. In the stochastic task, $\epsilon \mid x \sim \mathcal{N}(0, (0.08 + 0.16|x|)^2)$ is added to the response. The researcher receives purchased observations and the public interface; family, stage, latent means, and true noise scales are available only to the experiment driver and evaluator.

Each task has ceilings of 512 observations, 1,536 measurement units, 300,000 candidate-row computation units and six rounds. The two main arms use identical expression primitives and search algorithms. The persistent arm retains its knowledge; the task-local arm starts each task without prior knowledge. Whole sequences include all source acquisition, failed search and confirmation costs. Process CPU time is measured separately from charged computation. Main-arm execution order is randomized within each seed.

All research and prediction-diagnostic fitting finish before terminal data are requested. Evaluation uses four fresh trajectories of 64 rows per task. A deterministic task passes at MSE $\leq 10^{-5}$. The stochastic task requires mean prediction MSE ≤ 0.03 and mean predicted variance between one quarter and four times the actual noise variance. The latter is a broad noise-detection criterion; normal-interval coverage, standardized squared residuals, and variance estimation error are reported separately. There are no external model calls.

D.2 Interventions and eight operational capabilities

Additional branches copy an actually acquired prefix before changing one form of knowledge or access. Program deletion removes the source concepts and their derived program knowledge while retaining the method book. Method deletion clears only the method book. The controls-only laboratory preserves the response process but exposes no auxiliary measurement channel. All branches have the same per-task resource caps. The prefix is physically acquired once and its cost is included when reporting each counterfactual whole sequence.

Capability in the formulation	**Operational test in this instantiation**
New abstractions after prior failure	Add $0.35y^3$ after source acquisition; compare retained and removed program knowledge under the same expression search.
Applicability and preservation	Learn the local change; compare its predictions with the frozen old response and, when identifiable from paid data, its new branch applied without the learned guard.
Structural correspondence	Transfer across anonymous role order, units and origin; compare retention with program deletion from the same source prefix.
Observation insufficiency	Compare access to paid instruments with a controls-only view of the same hidden-input process.
Research-procedure generation and revision	From the same measured prefix, retain or remove the procedure book before changing instrument roles; inspect and execute the generated fitting subquestions.
Uncertainty and reversible simplification	Assess a repeated-evidence variance model on fresh outcomes; compare simple-model execution with a dense fit on the same paid data, then test the nonlinear return.
Revising how hypotheses are proposed and tested	Combine the definition and procedure interventions with the complete persistent/task-local sequence.

Capability in the formulation	Operational test in this instantiation
Recovering experimental distinguishability	Test whether paid view changes and boundary-directed probes lead to fresh predictive success.

The last two rows are properties of the connected loop, not two additional independent algorithms. The static scope contrast identifies an old branch by agreement on already acquired observations; it does not consult terminal targets. When an equivalent hinge expression contains no identifiable conditional branch, that diagnostic is recorded as unavailable. The dense cubic fit in the simplification diagnostic uses the same paid observations, and its additional fitting and evaluation costs are reported. These prediction diagnostics are distinct from complete competing research algorithms.

Paired intervals resample apparatus seeds within each response family, using 10,000 bootstrap replicates and percentile endpoints at 2.5% and 97.5%. Tasks within a sequence are not treated as independent replicates. All unsuccessful research remains in the reported cohort. CPU for additional deletion branches is not order-balanced against the retained prefix continuation; the primary runtime comparison is the randomized complete-sequence comparison.

Appendix E. Learned procedures in the legacy researcher

The growth study in Appendix D uses its own research loop. A subsequent integration exposes the same procedure book through the legacy knowledge library. The legacy agent records actual acquisition, fitting and fresh confirmation steps, executes learned procedures through its existing acquisition and search functions, and returns failed procedure evidence to its observation-revision controller. Produced concepts retain their procedure origin, so later source returns can reach the procedure book. Execution feedback and downstream knowledge returns remain distinct.

The integration comparison uses 12 new apparatus seeds: six finite-history and six recurrent instruments, with six tasks per sequence. Two source tasks are acquired once; their complete session is cloned before retaining or removing only the learned procedure list. Predictive concepts, instrument models, legacy methods, experience and method-identity counters are identical at the fork. Both arms can subsequently learn new procedures. Research caps are 384 observations, 1,536 measurement units and 360,000 charged computation units per task, with the same session pooling rules. Whole-sequence costs include the common prefix in each counterfactual arm. Terminal evaluation occurs after both arms finish and uses four fresh random trajectories plus positive and negative impulse tests; trajectory length respects each task's public limit. A joint pass requires internal confirmation at MSE $\leq 10^{-7}$ and both terminal tests at MSE $\leq 10^{-6}$. Paired intervals resample apparatuses within instrument mode.

Both arms pass confirmation and terminal evaluation on 72/72 tasks. Retaining source procedures uses 4,334 observations, 9,006 measurement units and 1,038,795 charged computation units, compared with 4,092, 8,328 and 1,011,121 after removal: increases of 5.91%, 8.14% and 2.74%. Training CPU totals are 59.21 versus 54.44 seconds. Actual procedure replay occurs 26 versus 33 times; the removal arm learns new procedures and per-

forms two procedure revisions. Neither arm requires a task revisit in this cohort. All three paired resource-difference intervals include zero (per-sequence observation difference: $[-3.33, 42.83]$; measurement: $[-9.25, 120.75]$; computation: $[-8998.75, 13306.92]$). The integration therefore establishes execution through the legacy controller without demonstrating an additional benefit from retaining its source procedures. All registered cases are retained, with no execution failures or external model calls.

A separate existing continuation regression starts one task under a restricted initial computation budget. A later task supplies learned knowledge, after which the existing portfolio scheduler selects a revisit and the measured source return reaches its producing procedure. The paid comparison loss falls from 0.3338 to 6.64×10^{-20}, followed by fresh confirmation at 4.28×10^{-19}. This is an operational integration test. The producer and consumer have different measured response signatures. In this first integration revision, strict condition matching does not use that return to choose a procedure. These results therefore separate shared storage, executable reuse, receipt delivery and an economic benefit of that delivery. General object and scalar-continuation procedure execution remain outside this integration experiment. Source and raw results are preserved in `artifacts/workflow_bridge_20261006_main_12/`; the earlier paper studies retain their frozen implementations and outcomes.

E.1 Demand-conditioned procedure choice

A subsequent revision separates procedure applicability from the conditions under which its knowledge helped a consumer. Unfinished visits supply demands through their paid observations. The selector matches confirmed source returns to those demands after filtering procedures for executable steps and acquisition and confirmation budgets.

Twelve new paired cases first acquire a shared three-task prefix containing two learned procedures and a confirmed source return. An independent researcher then supplies an unfinished consumer visit under a 512-unit initial computation cap. The next production task has a different response condition. The arms differ only in retaining or deleting procedure return records; both retain predictive knowledge and can acquire new returns. Each scheduled task has caps of 192 observations, 768 measurement units and 120,000 computation units. Terminal evaluation uses two fresh 24-step trajectories after both arms finish.

Retention changes the native choice from `method_1` to `method_0` in 12/12 cases. Both arms pass the 36 shared-prefix tasks but complete neither the 12 incoming demands nor the 12 production tasks. Whole-sequence research costs are 9,888 observations and 11,808 measurement units in each arm, and 2,497,834 versus 2,497,809 computation units. The paired mean computation difference is +2.08 units (95% bootstrap interval $[-3.08, 7.33]$); the mean suffix MSE difference is -0.00474 $[-0.05159, 0.04572]$. All cases are retained. This experiment establishes that actual returns influence procedure choice, without a demonstrated research benefit. It repeats a known consumer condition and tests choice inside an already scheduled task; the historical return does not measure the marginal value of producing more knowledge. Frozen source and results are in `artifacts/workflow_demand_20261006_main_12/`.

E.2 Complete research budgets and state identification

The next revision bounds all fitting callbacks within a replay by one shared computation allowance. Observation-revision options specify enforceable packages for fitting, distinguishing observations and fresh confirmation. When a measured alternative is affordable, preliminary mechanism revision cannot consume the observations needed to execute it. An existing continuous-state identifier also enters the research-action policy with a complete quote for first- and second-order identification, executable fitting and scoring up to six candidates. This allows a small temporal hypothesis to compete before broad representation search uses its budget. Identified programs replay input-driven state without past target values; adoption still requires fresh confirmation.

A paired experiment uses 12 new seeds and the task family and caps of E.1. The frozen prior implementation acquires each shared prefix and incoming unfinished visit once. Both continuations retain identical predictive knowledge, procedures and return records; only the subsequent implementation differs. All research finishes before terminal scoring. Research costs include the common prefix in each counterfactual bill, while terminal costs are reported separately.

The revised implementation completes fresh confirmation and terminal evaluation on 12/12 production tasks, compared with 0/12 for the baseline. Mean terminal MSE on these tasks falls from 0.2344 to 6.27×10^{-18}. Both arms retain 36/36 successful shared-prefix tasks and leave all 12 incoming demands unresolved. Whole-sequence research costs are 9,600 versus 9,888 observations, 11,808 measurement units in both arms, and 2,043,146 versus 2,496,666 computation units: 2.91% fewer observations and 18.17% less charged computation. The paired mean computation difference is −37,793 units, with a 95% interval of [−40,445, −35,167] from 10,000 bootstrap resamples of the 12 cases.

At the first remeasurement, every revised run retains 104 observation rows, compared with 80 in the baseline. All revised runs execute state identification and confirm the resulting response. Each then has 32 observation rows left in the session pool. In that implementation, the revisit scheduler requires 48 rows before screening an unfinished scalar task, so neither source screening nor revisiting occurs. The remaining demands therefore expose an allocation gate before candidate evaluation; these cases do not measure whether the new knowledge could help them.

This comparison evaluates the combined allocation and execution-order revision. It does not isolate a benefit of knowledge-return feedback or extend the model language. The unsuccessful first development revision, which enabled earlier measurement without completing the response, is retained alongside the successful second development case. Protocol, frozen source and all cases are preserved in `artifacts/workflow_budget_20261006_main_12/`.

E.3 Reopening a paid demand within the remaining budget

The next comparison tests the continuation rule in Appendix H.2.5. Screening includes retained knowledge that the unfinished consumer has not previously encountered. A candidate selected on paid observations is frozen for one fresh confirmation; failed confirmation ends this package. Completed short-history fits can also be challenged by later knowledge that predicts a distinguishable longer response.

Twelve new seeds use the same task family and ceilings as E.2. The original implementation acquires the common prefix and unfinished consumer; the frozen budget repair then finishes production. Both arms resume copies of this exact post-production state, including all knowledge and costs, with 32 observation rows remaining. The runner invokes the native session controller without choosing a source or candidate. Both arms finish research before terminal evaluation.

The revised controller completes and passes terminal evaluation on 12/12 incoming demands, versus 0/12 for the baseline. Production remains 12/12 in both arms, and the shared prefix passes 36/36. Each successful revisit uses the consumer's 192 paid rows for selection, then buys one 24-row confirmation trajectory. It adds 11,165 computation units, including screening and portfolio selection, and no measurements. Mean incoming terminal MSE falls from 0.8538 to 5.60×10^{-19}. Whole-sequence observations increase from 9,600 to 9,888 (3%), computation from 2,040,654 to 2,174,634 (6.57%), and measurement spending remains 11,808. No case fails execution and no external model is called.

All selected offers derive from two retained prefix concepts. Protocol, frozen implementation and outcomes are in `artifacts/paid_revisit_20261006_main_12/`.

To isolate the newly produced concept's contribution, a post-hoc intervention reuses all 12 post-production checkpoints. Under the same frozen controller, one arm retains that concept and the other removes it and its associated library entries. Earlier concepts, research procedures, paid observations, production reports and all incurred costs remain identical. Both continuations finish before terminal scoring. Incoming demands and production tasks each pass 12/12 in both arms; final programs and fresh confirmations are identical in every pair. Observations and measurements are unchanged. Retention increases whole research computation from 2,151,594 to 2,174,634, exactly 1,920 additional screening units per case. The new product therefore provides no marginal predictive benefit for these demands and adds search cost. All cases complete without an execution failure or external model call. The intervention and receipts are in `artifacts/new_product_value_20261006_posthoc_12/`.

This attribution uses previously observed cases and preserves production costs; it does not estimate the value of skipping production. The two prefix sources' individual contributions and long-horizon net benefit remain open.

Appendix F. Language-model studies and development record

This appendix preserves the protocols, intermediate failures and numerical results behind Appendices H.3.11–H.3.15. The studies are separate development cohorts, not successive measurements of one fixed algorithm on one independent test set. Their detailed record makes it possible to distinguish a working numerical mechanism, a frozen learned procedure, an adopted library item and an autonomous research decision.

F.1 Common-model study and task saturation

The completed comparison uses gpt-6-luna at medium reasoning effort through Codex authenticated with ChatGPT. All 90 registered source-to-target chains finish their research loops and independent evaluation. Each task has the same 24-observation, 16-call and 300,000-reported-token ceilings, with three paid fresh observations required for confirmation. The larger token allowance follows a retained development trial in which repeated

input overhead prevented submission; cached input remains charged to the common allowance.

Knowledge condition	**Source successes / 30**	**Target successes / 30**
Code/ReAct	29	29
Textual memory	30	29
Executable research knowledge	26	26

All submitted models also pass the 256-point independent evaluation. One Code source times out and its target is unattempted. Textual memory has one target token-budget failure. Hegel has four source token-budget failures; its target failures comprise three exhausted confirmation reserves followed by token exhaustion and one model-capacity error. On the 29 targets attempted by all arms, Code solves 29, textual memory 28 and Hegel 25. Hegel performs 36 successful inherited experimental-program calls on targets, including 15 unchanged calls, but this execution does not translate into superior task completion.

The completed run contains 894 calls and at least 20,395,181 reported tokens; 70.3% of known input tokens are cached. Earlier GPT, Qwen and interrupted Gemini cohorts remain separate service/interface records, not additional samples in this comparison.

This task is saturated for ordinary code investigation: it publicly enumerates four response laws, and direct response probes can identify them without resolving the instrument. The completed comparison establishes no advantage for executable research knowledge. We retain it as a negative comparative result and a bounded execution example. A subsequent development environment expands the legal trajectory horizon: two physical explanations agree on every legal short experiment and diverge on longer experiments.

F.2 A measured child repairs a temporal representation

The temporal apparatus expands its legal horizon from three to 24 steps while preserving its physical parameters. A bounded offline controller first extends a quadratic control-history model. When paid trajectory error persists, it opens an instrument question before acquiring auxiliary readings. Existing finite-history and continuous-state fitters propose instrument explanations; their predicted disagreement selects the next measured experiment. The selected instrument program becomes an executable coordinate of a general quadratic parent response, evaluated from control history alone.

Across 12 new paired apparatuses, all 24 targets pass 48 fresh confirmation rows and a separate 256-row evaluation at MSE below 10^{-10}. All 12 finite cases use history revision without measuring the instrument. All 12 recurrent cases acquire 15 short measured rows followed by a four-step distinguishing probe; the returned state program changes the parent predictor. Their maximum terminal MSE is 7.36×10^{-17}. A three-lag quadratic refitted on exactly the same preconfirmation response rows fails all 12 recurrent cases, with MSE 0.00762–0.316. Target acquisition uses 144 rows for finite cases and 163 rows plus 57 measurement units for recurrent cases. The prospective records contain the acquired trajectories and selected programs. Each final predictor is fixed before confirmation and terminal evaluation. This experiment uses no LLM calls.

A subsequent matched-repertoire comparison uses 30 new paired apparatuses. A flat adaptive researcher has the same instrument candidates, response basis, experiment planner and observation limits. It fits instrument-conditioned response candidates on 63 paid rows and selects on two other complete trajectories totaling 48 rows. Both policies confirm and independently solve 60/60 targets, with zero false confirmations. In every recurrent case, flat selects a successful response before the instrument alone is uniquely supported. It uses 159 target rows and 45 measurement units, compared with 163 and 57 for recursive research, while making three response fits instead of one. Both policies use 144 rows and no meter readings on finite cases. Thus the common representation repertoire achieves the same predictive success without separate child confirmation. The computation cap covers instrument search; these results do not compare total computation.

F.3 LLM selection of shared research operators

We expose the individual history, instrument, response-fitting, disagreement and assessment operations to all three gpt-6-luna knowledge conditions. The LLM chooses acquisitions, operator calls, returned predictors and submission; the operators do not execute a complete research policy. One new apparatus draw supplies finite and recurrent continuations, each preceded by short-source learning. Each task allows 240 observations, 24 model calls and 600000 reported tokens checked between calls, with 48 fresh confirmation rows.

All six sources pass. Code and textual memory each confirm and independently solve 2/2 targets, while Hegel completes 0/2 targets before its token boundary. Hegel actually re-executes a retained short experiment. On the recurrent target it purchases a proposed distinguishing measurement but does not subsequently assess or refit the instrument. Each unsuccessful target has a rejected fresh confirmation; no submitted model is falsely confirmed. No recorded question has a parent ID. The complete pilot contains 98 calls and 3668858 reported tokens; 43.9% of known input is cached. The textual recurrent target finishes at 865992 tokens because its final call exceeds the between-call threshold. Interface repairs made afterward retain long training trajectories and compact large source files in the prompt; the reported model outcomes are unchanged.

A subsequent development retest adds explicitly agent-authored post-acquisition operator sequences. One new paired apparatus yields 2/2 correct sources and 2/2 correct targets in every arm. Hegel's recurrent target buys two six-step meter experiments and immediately assesses the inherited instrument candidates: the finite candidate is refuted (MSE 0.008481), while the recurrent candidate is supported (7.12e-19). The LLM then replaces the inaccurate parent response (paid MSE 0.001031) with a response using that coordinate; fresh-confirmation and terminal MSE are 1.18e-17 and 9.26e-18. On the finite target, the frozen source predictor succeeds unchanged. The recurrent target needs 72 observations versus 96 for either control; including source learning reverses that difference, to 192 versus 174. Across both sequences Code, text and Hegel use 342, 378 and 360 observations and 27, 19 and 33 model calls, respectively. All 79 calls report 1905425 tokens, with 50.34% of input tokens cached.

The retest establishes successful use of measured evidence to select a retained supporting hypothesis and revise its dependent response. Its limitations are specific: the new apparatus and multiple interface changes do not isolate an effect of continuations, and both ordinary

controls also succeed. The LLM writes continuations in query arguments but does not incorporate them into its retained research program. Its online questions remain roots; a later dependency manifest does not establish recursive problem decomposition. That pilot does not test learning and transferring the complete repair procedure.

F.4 Learning executable research procedures

We distinguish an acquisition plan from a research procedure that analyzes its observations, chooses whether to revise an earlier explanation, and submits a predictor. All procedures below are authored by `gpt-6-luna` using the shared numerical operators. The initial donor is the successful recurrent target's paid experience; terminal evaluation is excluded from learning.

Early development establishes the difficulty of preserving a useful procedure. Retrospective consolidation produces interface errors and then a short acquisition-only plan. Paid practice executes an inherited program, revises it and freshly confirms its own exported predictor. The first two attempts fail confirmation at MSE 0.646 and 0.261 without revising the changed instrument; a later attempt refutes that instrument but exhausts observations before rebuilding it.

An expanded practice refutes the inherited instrument at MSE 0.01497, then learns a procedure that fits a new instrument and its dependent response. Its predictor passes 48-row fresh confirmation at MSE 4.62e-19. Practice uses 288 observations and 11 model calls. The frozen procedure acquires three 24-step measured trajectories and refits on current data, without carrying practice-fitted coefficients into transfer. A retention/deletion intervention restores the original predictive snapshot and shares the new verbal lesson. Both conditions confirm and independently solve 2/2 targets. Retention/deletion use 264/216 observations, 30665/4348 charged computation and 15/16 model calls. The recurrent retained agent copies the exact learned acquisitions and fitting steps into an inline query. Thus this case establishes behavioral transfer with equal accuracy and higher data/computation costs when the procedure is available.

Two conditional-practice attempts then allow programs to read actual operator feedback and analyze existing paid records. Neither publishes a freshly confirmed method. Repairing inconsistent feedback formats permits different stages of one unchanged source to execute, but its decisions concern record counts and result presence. The last source edit is unexecuted. These attempts also begin by refitting the apparatus, removing the initial contrast between valid and invalid prior knowledge.

We therefore introduce paired practice. Both cases receive identical earlier calibration and the same public interface; the mechanism remains unchanged in one and changes in the other. A program starts with empty current observations, chooses experiments and analysis steps, and explicitly declares its final model. Ordinary fresh confirmation either accepts it or returns paid counterexamples. The next invocation uses this actual feedback. No LLM chooses intermediate actions once the program is frozen, and no initial refit is supplied by the host. The first paired learner confirms both cases in five model calls. Its fixed program also solves four frozen cases across two earlier calibrations, but always buys 72 measured observations and refits: predictive success alone leaves reuse untested.

A further learning task explicitly requires confirmed reuse of the unchanged predictor in one case and confirmed revision in another, without revealing which case supports either choice. Its proposed prior-first submissions expose a runtime error: the host requires paid training rows before allowing fresh confirmation. We interrupt this run and repair that precondition. An initial submission now skips only the absent training diagnostic; actual 48-row confirmation remains mandatory. We then freeze the last recorded LLM-authored candidate and replay it from empty workspaces, without restoring the erroneous feedback.

The unchanged program first submits the inherited predictor. On both unchanged mechanisms, fresh confirmation passes and execution ends with that exact predictor, using 48 observations, no meter readings and 48 charged computation units per case. On the changed mechanisms, the inherited predictors fail at MSE 0.5322 and 2.3136. The program then purchases 48 measured observations and fits instrument and response hypotheses. It selects a response using paid instrument consistency and whole-trajectory validation, then requests a new confirmation. Both revisions pass, with terminal MSE 7.25e-18 and 4.62e-19. All four cases pass confirmation and a separate 256-row terminal evaluation, performed only after all four executions stop. The second calibration was unavailable during authoring and reverses which mechanism is unchanged. Its supplied predictor role is used without editing the procedure.

Across these four matched cases, the prior-first procedure uses 384 observations, 288 measurement units and 15820 charged computation, compared with 480, 864 and 93828 for the earlier always-refitting program. Both execute without LLM calls and solve 4/4 cases. Changed cases cost 144 observations each, versus 120 for fixed refitting, because the rejected prior test is also paid; unchanged cases cost only 48. These are observed execution costs of two development candidates. Acquisition patterns also differ, so the comparison does not isolate knowledge retention's causal contribution.

A subsequent online attempt uses the repaired runtime but ends after 17 calls and 1,022,484 reported tokens without publishing. It achieves three confirmed revisions and one confirmed reuse across different program versions, repeatedly changing executable branches between trials. The final source is confirmed on only one case. We therefore expose each current program's completed and missing practice cases directly in write/practice feedback. A development continuation starts from the last actual candidate, with new empty practice records. It confirms a revision at MSE 1.90e-18 and prior reuse at 1.06e-17, but an intervening edit again leaves the current source confirmed on only one case. Its sixth provider call fails because the selected model is at capacity. It reports 194,933 tokens before that failure, whose usage is unavailable. Neither attempt supplies a published method or a completed retention/deletion comparison.

A third phase restores the saved candidate and paid trial history after the provider interruption. Confirmation on case_b remains valid for that exact source; the earlier case_a result belongs to another version and is excluded from its coverage. The LLM performs two new case_a trials without changing the source, then publishes the procedure. This phase uses three calls, 91,617 reported tokens, 240 observations and 44,514 charged computation units. The published program assesses the inherited instrument on current measured trajectories. Supported instrument hypotheses lead to reuse of the unchanged predictor; refuted hypotheses lead to instrument and response refitting, followed by fresh predictive confirmation. It passes all four frozen cases across the two calibrations, with maximum

terminal MSE 8.77e-16 and no execution-time LLM calls. Each case uses 120 observations and 216 measurement units; unchanged/changed cases use 192/22,257 charged computation units respectively. This completes publication through continued development, including the earlier failed attempts. The saved results keep new-phase costs separate from retained trial evidence.

The completed frozen replay establishes an executable prior-first confirmation-and-repair procedure in this bounded family. Online publication is now complete, while the added state feedback has no isolated efficacy estimate. Reuse and revision are explicit task objectives, roles are supplied, and numerical fitting families remain developer authored. The earlier frozen program reacts to predictive failure without independently localizing its cause to the instrument; the subsequently published procedure assesses measured instrument support before selecting reuse or refitting. Two previously studied calibrations do not establish population-level reliability, open-ended method invention, or stable long-horizon net benefit.

F.5 Availability of a learned research procedure

We retain or delete the published procedure while holding original predictive memory, instrument roles and its final verbal lesson fixed. Both conditions use the same Luna model and single-invocation procedure interface. The LLM can call, modify or reconstruct a procedure, or choose individual research actions. Four paired cases cover stable and changed mechanisms under the two calibrations. Each run has limits of 240 observations, 48-row fresh confirmation, 100,000 charged computation units, 24 model calls and 600,000 reported tokens, with the token limit checked between calls. All eight agents stop before terminal evaluation.

The initial pilot and a subsequent fresh-agent development rerun are reported separately below. The rerun keeps the published source, predictive memory, lessons, tasks, seeds, model and limits fixed while giving both conditions the repaired submission interface described below. Each task starts with no current observations.

Measure	Pilot retained	Pilot deleted	Rerun retained	Rerun deleted
Confirmed and correct	3/4	3/4	4/4	4/4
False confirmations	0	0	0	0
Model calls	24	34	18	16
Reported tokens	1,378,181	1,856,123	794,795	848,375
Observations	408	396	408	372
Measurement units	792	612	648	396
Computation units	42,363	12,373	30,224	25,028

The pilot failures occur on different changed mechanisms. The deleted failure does not refit a refuted instrument and incurs a costly output-decoding failure. The retained failure selects a strong candidate but repeatedly edits its manifest after a missing-path rejection, never retrying confirmation. We repair direct consumer submission to register an explicitly selected model path while preserving existing research entries and paid fresh confirmation. Separate post-hoc completion of the unchanged, previously selected candidate passes 48

new rows at MSE 4.23e-20 without an LLM call or refit. The original pilot remains 3/4 in each condition.

In the rerun, all eight agents autonomously confirm a prediction that also passes terminal evaluation. Retained agents invoke the unchanged learned procedure ten times across all four tasks; its final invocation submits every accepted model. Both stable cases reuse the original predictor. On one changed case, a response fit initially lacks a separate validation trajectory; the LLM acquires another trajectory and the same procedure completes the revision. Deleted agents submit all four refitted predictors through direct finish. Neither condition edits a manifest or encounters a submission rejection, provider failure or output-decoding failure. The formerly blocked retained case confirms at MSE 1.23e-19 and obtains terminal MSE 1.69e-19.

Retention uses 6.3% fewer reported tokens in this rerun, but more model calls, observations, measurements and charged computation. Python execution totals are 2.78 versus 6.40 seconds. Thus learned-method use and autonomous completion are observed without an overall comparative advantage. The original pilot record and rerun record retain failed learning costs in the common ledger; their exact token total remains unknown. The same-case rerun is a development result, and stochastic changes in research trajectories prevent attributing the improvement solely to the repair. Both calibrations were previously studied; unseen-task generalization remains open.

F.6 Revising a research method from a parent counterexample

The preceding tasks usually change instrument and response together. We separate them using independently seeded response parameters, preserving the old default laboratory behavior and public interface. The original two donor memories and published procedure remain fixed. In an initial frozen screen, the procedure solves both stable cases and all 24 new-instrument cases, but fails on both response-only changes: 26/28 overall, with no false confirmations. Each failure correctly supports the old instrument, then submits the same incorrect parent predictor three times despite fresh counterexamples, spending 216/240 observations. Instrument validity does not establish the validity of a response using it.

A preselected six-pair live retention/deletion pilot completes three cases per arm, with one retained confirmation-budget failure and five service-capacity interruptions. All six confirmed outputs pass terminal scoring. One retained agent recovers by bypassing the procedure and refitting the response from paid evidence. Another buys 96 extra measured rows and refits both components, leaving too little budget for confirmation despite low candidate validation error. Neither rewrites the method. These trajectories distinguish local recovery from a reusable procedural revision. Initial screen and live pilot.

We then explicitly ask Luna to revise the executable method from one recorded response-only failure. The author receives the first donor's original knowledge, the old source, and that trial's paid observations and feedback. It chooses how to edit and practice on three opaque cases: stable, joint change and the observed response change. Publication requires the same final source to pass fresh confirmation on all three, with actual unchanged-predictor reuse and revision. The second donor and transfer outcomes are unavailable during authoring.

Luna makes one source edit, practices all three cases and publishes in five calls, using 166,174 reported tokens, 408 observations, 648 measurement units and 25,233 charged computation units. The new source reads failed predictive-confirmation feedback. In response-only practice it first supports the original instrument on 72 measured rows, then refutes the old parent prediction on 48 fresh rows. It retains that instrument and fits the response using the 120 already paid rows, with whole-trajectory validation, then passes another 48-row confirmation at MSE 4.57e-18. No instrument refit or additional measurement occurs in this repair.

We freeze the old and newly published methods on the same 18 transfer tasks, using identical original numerical priors, confirmation seeds and limits. The response-only tasks use four new response seeds under each of two priors; four new instrument seeds each supply an instrument-only and a joint-change task. Only the method and its description transfer from authoring. All subsequent execution is programmatic, with no LLM selecting intermediate actions.

Calibration change	**Original method**	**Revised method**
Stable	2/2	2/2
Response only	0/8	8/8
Instrument only	4/4	4/4
Instrument and response	4/4	4/4
Total confirmed and correct	10/18	18/18

Both methods produce zero false confirmations. Every revised response-only run executes the same branch: support the instrument, refute the old prediction, refit only the response from existing evidence, and confirm on 48 new rows. Terminal MSE ranges from 5.79e-18 to 8.59e-16. All ten other tasks retain exactly the old method's fresh and terminal scores, including direct reuse on stable cases. Frozen observations fall from 2,928 to 2,544; measurement units remain 3,888 and computation rises from 183,048 to 200,712. Authoring costs are additional. This establishes a measured counterexample changing an executable research rule, followed by preserved earlier behavior and successful transfer of the revision. Method-learning and matched frozen results.

F.7 Evidence scope after a calibration checkpoint

We next test a limitation of the learned trigger: supporting measurements can precede the change that invalidates a parent prediction. The apparatus now discloses a possible recalibration after 72 observations and asks for predictions under its subsequently fixed physics. Stable, response-only, instrument-only and joint-change cases share their public interface and initial observations. The change occurs between whole trajectories. Instrument and response operators can fit selected complete paid records, retaining the remaining history and its costs.

The published method still passes all 18 preceding stationary tasks and both stable checkpoint controls, but fails all 12 changed-checkpoint cases: 20/32 confirmed and terminal-correct, with zero false confirmations. Its earlier instrument assessment concerns

the pre-change evidence. Repeated response fits do not establish whether that instrument remains applicable afterwards.

An initial author phase supplies a real instrument-change failure and four opaque practice cases. Luna makes 11 source edits, four inspections and five practices in 20 calls, reaching the 1,208,226-token ceiling without publication. An earlier source confirms the stable case; four response-change practices fail. The last two acquire new meter readings and fit the instrument on those records, but response training and refitting still mix earlier and current labels. Their fresh-confirmation MSEs are 0.00455 and 0.00507. No executed program reassesses the old instrument on the new measurements. This phase uses 1,032 observations, 1,512 measurement units and 58,290 charged computation units, without a provider error or timeout. A separate programmatic execution of the unedited final draft on the four practice cases confirms only the stable case. This diagnostic replay does not publish the draft or establish transfer. Failure-selectivity experiment.

A new author phase starts from the last successfully published method and one failed practice from that run. Its feedback now exposes a compact chronology of the already paid evidence: support on rows 0–71, a failed prediction on rows 72–119, new measurements on rows 120–191, and response refitting on rows 0–191. The recorded failed source is identified separately from the starting method. No replacement program or hidden change label is supplied. Luna authors one source revision, executes six practices and publishes in nine calls, using 554,008 reported tokens, 1,080 observations, 1,944 measurement units and 68,067 computation units. The same final source confirms all four cases, retaining the unchanged predictor in the stable case. The preceding unsuccessful phase and its costs remain separate.

The revised program buys new paired meter/response trajectories after the old prediction fails, fits an instrument on those records, and explicitly restricts response fitting to the same records. It then confirms on fresh observations. Thus the evidence used to fit a supporting computation also bounds the evidence used to fit its parent response. We freeze this program on the same 32 cases, with the same original priors and limits and no LLM calls during transfer.

Task group	**Previous published method**	**Evidence-scoped revision**
Previous stationary tasks	18/18	18/18
Stable checkpoint	2/2	2/2
Response changes after checkpoint	0/4	4/4
Instrument changes after checkpoint	0/4	4/4
Both change after checkpoint	0/4	4/4
Total confirmed and terminal-correct	20/32	32/32

Both methods produce zero false confirmations. All twelve repaired tasks finish within 240 observations. Observations rise from 5,376 to 6,816, measurement units from 6,912 to 12,960, and computation from 237,096 to 629,340. Stable tasks still reuse the original predictor; earlier changed tasks preserve accuracy with more acquisition and refitting. The method repairs evidence scope, but refits both instrument and response after every predic-

tive failure. It has not learned to retest the old instrument and preserve it when only the response changes. Evidence-scope learning and frozen comparison.

F.8 Preservation, adoption and method revision

F.8.1 Selective repair while preserving earlier capabilities

We explicitly ask the author to demonstrate two responses to a failed parent prediction: preserve the inherited instrument when new measurements support it, and reopen instrument research when they refute it. The selected response must use the corresponding instrument and pass fresh confirmation. These outcomes are identified from paid execution traces, without exposing hidden change labels. The same source must also retain unchanged-predictor reuse and solve every practice case. Operators, hypothesis families and per-task limits remain fixed.

The first learned revision satisfies this objective in eight calls (450,058 reported tokens) and succeeds on all twelve additional calibration cases, with four instrument-retention and eight reopening trajectories. However, it regresses on eight earlier tasks: an instrument already invalid at the start triggers repeated measurement because refitting is conditioned on an earlier failed parent confirmation. The strong baseline passes 44/44, whereas this revision passes 36/44. Its four practice cases all began with a valid instrument, leaving that entry condition untested. Initial selective-revision result.

We add a fifth opaque practice case reproducing this initial-refutation failure, with a stable checkpoint that preserves its physics and the common public interface. Its actual failed replay supplies the evidence for another author phase. All preceding 44 tasks become regression controls; twelve further calibration cases are fixed before authoring. The comparator remains the earlier fully successful refit-both method, rather than the regressed revision.

Luna makes two source edits and confirms all five cases in eleven calls, using 732,491 reported tokens, 1,656 observations, 3,240 measurement units and 71,427 charged computation units. The final program changes the research order. After its initial acquisition, it tests the inherited predictor directly. A failed prediction leads to fresh measured trajectories and an assessment of the old instrument. Support leads to response-only fitting; refutation leads to instrument and response fitting. This also handles initially invalid instruments by obtaining a parent counterexample before the supporting investigation. The same source and original numerical priors then execute without further LLM calls.

Both methods confirm and pass terminal evaluation on all 56 tasks, with zero false confirmations. This preserves all 44 regression controls and succeeds on all twelve new calibration cases. The final program retains the instrument in 20 tasks, reopens it after measured refutation in 32, and directly reuses the old predictor in four. Among the new twelve, these repair counts are four and eight.

Frozen execution cost, 56 tasks	**Refit-both baseline**	**Selective revision**
Observations	12,576	12,960
Measurement units	23,328	23,328
Charged computation	1,169,268	754,464

Computation falls by 35.48%. The 384 additional observations come from eight initially changed-instrument tasks, where testing the inherited predictor adds a 48-row failed confirmation before recovery. On the twelve new calibration cases, both observation and measurement costs remain equal, while computation falls from 269,964 to 186,792 (30.81%). Authoring costs are additional: the two selective-learning phases together use 19 calls and 1,182,549 reported tokens. The earlier 36/44 regression remains recorded. These results establish an executable method that conditionally preserves a supported component, repairs a refuted component, and recovers its earlier predictive capability after revision. Selective preservation and matched results.

F.8.2 Historical distinguishing experiments under live control

Historical experiment methods under live LLM control. We also expose the measured v0.76 method book to Luna on the original window/event task family. An initial two-source pilot solves both tasks through self-designed queries but produces no operator-managed method entries, so its planned transfer pairs do not start. We then directly retain or remove the historical book in two matched target pairs. Both conditions receive identical current paid calibration, supplied response candidates, tools and limits.

All four targets pass fresh confirmation and the original 128-row terminal activation/exit evaluation with zero MSE. In the retained event task, Luna requests a distinguishing experiment. The planner recalls the old temporal recipe, rebinds its control under the new calibration and threshold, and skips fresh synthesis. Four purchased observations return $[0, 0, 1, -1]$, uniquely supporting the event response among five supplied candidates. Measured feedback updates the historical method with cross-task support. The other targets use informative self-designed queries without explicit method execution.

Across these two pairs, retention reduces observations from 336 to 252 but increases measurement units from 432 to 468, charged computation from 23,540 to 25,946, and reported tokens from 173,825 to 283,230 (four versus nine calls). The retained event agent first fits uninformative all-zero data. The two-pair pilot establishes live consumption of an existing measured method, with mixed cost effects. Historical acquisition costs remain separate. Protocol, traces and results.

We subsequently expose the existing research-action policy as a paid-evidence choice operator, with executable plans from the same experiment library. In a second four-run pilot on those tasks, every Luna agent requests this comparison, purchases four distinguishing observations, and confirms the uniquely supported candidate. Both retained runs execute historical methods; both removed runs reconstruct a method. All four frozen predictions again pass terminal scoring with zero MSE. Both conditions use 248 observations, 456 measurement units and six calls. Retention reduces charged computation from 31,042 to 26,329 (15.2%) but increases reported tokens from 179,812 to 206,813 (15.0%). The redundant zero-response fit does not recur. Research-choice traces and matched results.

F.8.3 Source returns and unfinished consumers

An offline session study connects paid inquiry episodes to the existing revisit scheduler. In 12 fixed apparatus pairs, the complete learned library is retained or removed after identical consumer observations and source research. All training precedes final scoring. Retention

triggers 12 revisits and produces 12/12 consumers passing fresh confirmation and terminal evaluation, versus 0/12 after removal; source tasks pass 12/12 in both conditions. All retained consumer programs call the source concept. Whole-sequence observations are 5,907 versus 3,744, measurement units 2,592 in both, and charged computation 1,361,114 versus 920,268. Each successful callable retains its actual source template and role binding through revision. A fresh native comparison supplies provisional source support; the Agent's subsequent fresh confirmation compares the unchanged transferred explanation and its incumbent again. All 12 consumers obtain confirmed source returns that update the producing research procedure. Two feedback stages per apparatus yield 24 historical records. These results use the original 12 apparatuses after repairing source propagation, with unchanged final programs, observations and measurement costs; the additional paired comparisons cost 576 computation units. Session protocol and results.

A subsequent same-model pilot gives Luna autonomous control of this session interface on four of those apparatuses. Both knowledge conditions share the paid consumer prefix, tools and limits. Retention produces three completed targets and one call-limit exhaustion; task-local learning produces three completed targets and one service-capacity interruption. All six submitted predictors pass fresh confirmation and terminal evaluation. Among the three pairs without service interruption, retention completes 2/3 targets versus 3/3 for task-local learning. Both use 288 consumer observations; retention uses 72 additional measurement units and 597,151 versus 524,046 reported tokens. Shared source production remains charged once per condition in the complete ledger.

No Agent selects native research, a revisit or explicit source confirmation. The retained condition invokes the ordinary instrument, response and history operators once each. Its unsuccessful run finds a near-exact trajectory-held-out quadratic fit, then explores weaker features without submitting before its call limit. This pilot exposes a research-control failure and shows that the earlier offline scheduler ablation does not establish an advantage over an ordinary adaptive Agent. The source knowledge was produced by the native offline researcher, and the apparatuses were previously studied. Live-session protocol and results.

F.8.4 Retaining predictive programs through change and return

We next connect the existing evaluated-model snapshot and research-action policy to ordinary Python candidates. New observations require a paid reassessment of the retained predictor using its original fitting data. A fixed offline controller tests this mechanism on 12 new finite-history apparatus instances: learn an initial response, change it, then return to the original response. Candidate coefficients are frozen before assessment. Retaining all confirmed explanations and retaining only the latest each solve 24/24 targets, with all 12 shared sources also passing. On return, retention selects the original learned code in every case without refitting; latest-only refits in every case. Including initial acquisition once, charged computation is 59,040 versus 84,672 (30.27% lower), with 3,456 observations and no measurement units in either condition. Actual training times are nearly equal, 18.37 versus 18.47 seconds.

Separately, a disclosed development replay of the unsuccessful Luna apparatus completes in 12 calls. Its first fresh confirmation fails; after incorporating the 24 counterexample rows, it compares the old snapshot and revised predictor on the same 96 paid rows, with MSEs

1.82×10^{-7} and 9.73×10^{-30}, respectively. The final predictor passes a new confirmation and 128-row terminal evaluation. This single replay does not isolate the repair's causal effect, and the LLM still does not select the research-choice or revisit operators. The offline experiment establishes candidate reuse under a fixed controller; this motivates a live changed-and-returned sequence. Candidate-retention experiments.

A subsequent live pilot uses two new apparatus instances. Luna learns each source once, then each knowledge condition continues through a changed response and its return. Archived Python predictors carry their original fitting data and dependencies; each branch inherits its own confirmed state. All ten research episodes end before terminal evaluation. Both sources and all four targets per condition pass fresh confirmation and the separate 128-row terminal test.

On both returns, retention submits the unchanged original predictor in two calls; latest-only researches anew in 11 and 8 calls. In one retained return, the Agent asks the research-choice operator to compare the original and changed predictors, obtaining paid MSEs of 2.46×10^{-29} and 0.38385, then confirms the original. The other evaluates and confirms the original directly. Including common source acquisition once per condition, retention uses 30 versus 42 calls, 697,061 versus 1,188,554 reported tokens, and 576 versus 600 observations. Latest-only's failed submission on one return purchases counterexamples and leads to a successful revision. Physical usage is 61 calls with complete usage reporting; cached input accounts for 54.28% of reported input tokens. Live retention protocol and traces.

F.8.5 Selecting and using supporting investigations

A follow-up diagnostic pilot reuses these two learned sources, each on a parameter change and a change to recurrent dynamics. An explicit same-source fitting tool holds out whole paid trajectories. Under a fixed controller, refitting repairs both parameter changes; its validation errors of 0.116 and 0.029 on the recurrent tasks instead lead to the existing instrument and response investigations. All four targets pass fresh confirmation and terminal evaluation. Live Luna solves the two parameter changes but exhausts its 12-call limit on both recurrent tasks. This first live run omitted the numerical confirmation criterion from the public interface. After publishing the unchanged 10^{-10} MSE threshold, a development replay of all four tasks again yields 2/4, with no false confirmations. The replay uses 41 calls and 1,260,301 reported tokens. One failed investigation explicitly recognizes underdetermination and buys meter readings, but its next action still fits control history without using those readings. Both parameter tasks succeed through instrument/response research; neither live run selects the new refit tool. The remaining operational gap is turning evidence of failure into an effective research branch. That version of the Python research-choice interface offered retention and confirmation, whereas revision planning already existed for the internal program language. Diagnostic pilot and preserved failures.

We subsequently expose the existing refit, instrument and response investigations through the Python chooser. A deterministic consumer executes only its proposed next action, without selecting branches from private task labels or numerical errors. On the same four development targets, both parameter changes require refit and confirmation; both recurrent changes additionally require measured instrument validation and response revision. All four pass fresh confirmation and terminal evaluation. The 12 proposed

actions are all executed, using 480 observations, 288 measurement units and 10,598 charged computation units, with no LLM calls. Replayed control trajectories remain together when splitting response training and validation. This result establishes an executable bridge to prior research mechanisms.

The subsequent live replay passes all four targets, with no false confirmations. Both recurrent cases acquire instrument readings and invoke the existing instrument and response operators; the second does so at its tenth model call, after unsuccessful control-history fitting. The two parameter changes are solved by refitting a three-lag model. Neither the research-choice nor the refit operator is invoked by the live Agent. Total usage is 33 calls, 954,230 reported tokens, 504 observations and 144 measurement units; cached input is 54.59% of reported input. These traces establish live use of the older research branches. Routing implementation and live traces.

We next place concrete recommendations directly in action feedback and compare showing versus withholding them on the same four development targets. Both conditions compute the recommendations and retain the same tools, including the explicit chooser; they share model, prompt, initial observations and a 12-call limit. The visible condition completes 3/4 targets, compared with 2/4 when advice is withheld, with no false confirmations. Both solve the parameter changes; only the visible condition solves one recurrent change. In that pair, both spend 144 measurement units, but the successful Agent tests an instrument distinction, assesses the acquired readings and uses the supported instrument to revise its response. Terminal MSE is 1.09×10^{-18}. Thus a supporting investigation actually changes the parent revision in this live trace.

Across the four targets, visible/hidden advice uses 32/36 calls, 968,505/1,128,135 reported tokens, 456/696 observations, 216/144 measurement units and 17,086/22,488 charged computation units. The 240-row observation difference comes entirely from a pair in which both Agents fail. Six of 15 visible recommendations with a subsequent event exactly match that next action and its required arguments. In the remaining visible-condition failure, instrument ambiguity is identified, but the Agent returns to high-order history fitting; near-zero paid-data error becomes 4.18×10^{-4} on fresh observations. The completed experiment uses 68 model calls, with all usage reported and 49.76% cached input. Advice intervention and traces.

We then keep unresolved advice available after ordinary Python actions and in the research state. On four new apparatuses, live source learning succeeds on three; the failed source blocks four registered target episodes. Persistent and event-only advice both complete 4/6 attempted targets (4/8 registered), with no false confirmations. Removing a pair affected by service capacity leaves both at 4/5; persistent advice uses 4.91% more tokens, 16.67% more observations and 75% more measurement units. In one failure the same distinguishing purchase remains visible across four ordinary actions but is never executed. In another, the event-only Agent finds a low-error candidate but uses its last call for evaluation, leaving fresh confirmation unfinished. Continued presentation is operational, while effective adoption and completion remain unresolved. Frozen persistence study.

F.8.6 Availability and non-adoption of complete procedures

We subsequently expose the existing procedure executor as one bounded live action. Both conditions receive the same editable, developer-authored adapter to the research policy; the intervention allows repeated execution in one model call. Explicit offline execution completes all six targets. In the live availability comparison, the bundled/atomic conditions complete 3/6 and 5/6; excluding two service-capacity-interrupted pairs leaves 2/4 and 4/4. No Agent invokes the supplied program in either condition. The two research failures stop after paid candidate evaluation and after instrument discrimination, respectively, without fresh confirmation. Thus making a complete procedure available does not establish its adoption or benefit. Execution intervention and traces.

A follow-up places the procedure option directly in current feedback. Both conditions complete 5/6, or 4/4 after removing two service-interrupted pairs; the latter comparison uses 36 versus 20 calls. None of 24 displayed options is selected. This presentation is therefore retained only as an experimental condition. Feedback-option intervention.

F.8.7 Learning procedures over archived Python predictors

We then reuse the method-authoring and paid-practice loop with actual archived Python predictors, retaining their training files and dependencies. Development exposes mismatched practice and runtime trace formats and an incomplete description of paid evaluation versus fresh confirmation. After correcting these interfaces, continued authoring produces a conditional procedure: confirm the inherited predictor, refit after rejection, and investigate the instrument when refitting fails separate-trajectory validation. Instrument evidence then supports response revision and fresh confirmation. The authoring process ends before testing its last edit, so we freeze that exact candidate for an externally scheduled replay.

The unchanged source passes all three practice conditions. On two other archived predictors, each tested under unchanged, response-parameter and recurrent-instrument conditions, both the learned candidate and fixed procedure complete fresh confirmation and independent terminal evaluation on **6/6 targets**. The candidate directly reuses the old predictor twice, refits it twice, and completes measured instrument research followed by response revision twice. Its maximum terminal MSE is 6.34×10^{-19}. It uses 768 versus 672 observation rows and 11,606 versus 10,406 computation units; both programs use 288 measurement units. The candidate's initial fresh rejection tests make it more expensive in this comparison. Replay uses no model calls. All preceding authoring phases, including failures, total 49 calls and 2,015,212 known tokens, with one call's usage unreported. Frozen-candidate replay and retained authoring outcomes.

A subsequent workflow makes an author's explicit submission complete missing paid practices before applying the same publication gate. Restoring the actual author checkpoint, Luna edits the program once and submits within six calls, consuming 165,997 tokens (36.10% of reported input cached). The submitted program passes all three practices and is published. Frozen execution on six targets with advanced trajectory streams again yields **6/6 for both learned and fixed programs**, with two unchanged-prior reuses and no false confirmations. The new source replaces selective refitting with measured instrument reconstruction after every prior failure. Consequently, observations rise to 960 versus 672,

measurement units to 864 versus 288, and computation to 37,392 versus 10,406. This completes the author-requested practice, publication and transfer chain, while exposing a loss of selective repair. Cumulative authoring, including failed phases, uses 55 calls and 2,181,209 known tokens plus one unreported call; source acquisition and the earlier offline replay remain separately charged. Publication continuation and costs.

F.8.8 Fresh rejection after a validated refit

To preserve selective repair while testing recovery, we add the existing laboratory's public 96-row response-change checkpoint. The earlier selective program validates a refit, encounters a fresh counterexample and repeatedly resubmits the rejected candidate. Paid feedback supports a model-authored revision. Following a service interruption and exhausted author phase, an explicitly added phase with completion guidance submits the last authored source unchanged. All four practices pass: parameter changes retain zero-measurement refitting, while fresh rejection after validation triggers further research using newly measured response records. On eight frozen cross-source targets, the revised program solves **8/8**, versus 6/8 for the earlier selective source, 8/8 for the previous published program and 6/8 for the fixed controller. The latter confirms two models before the checkpoint that fail settled-state evaluation. Against the previous 8/8 program, observations decrease from 1,344 to 1,248, measurement units from 1,296 to 864, and computation from 61,172 to 23,160; measured Python execution time increases from 6.00 to 8.35 seconds. New authoring, including the interrupted call and failed practice phase, uses 13 calls and 369,866 known tokens plus one unreported call. The 24 completed reference episodes are reused by the evaluator and never supplied to the author. Selective repair and counterexample recovery.

F.8.9 Evidence-dependent repair and its engineering integration

We next retain these cases and add an instrument-family change at the same announced checkpoint. The previous published program solves all ten cross-source targets. Across five further author phases, the final candidate confirms all five practice cases but retains an absolute fresh-loss cutoff for choosing scoped refitting. It demonstrates no zero-measurement recovery after refit rejection, so publication fails and its ten transfer slots remain unexecuted. A researcher-arranged diagnostic then tests the same current-record refit in both change conditions. A response change with fresh MSE 0.447 yields scoped-validation MSE 1.87×10^{-29} and passes subsequent confirmation; an instrument change with smaller fresh MSE 0.0284 retains validation MSE 0.0237. This demonstrates why loss magnitude cannot identify which repair is adequate in these cases. The diagnostic costs 312 observations and no meter readings. Supplying it to the author does not produce another revision: the last phase spends its calls reading evidence. All five phases together consume 40 calls and 1,521,964 tokens, with 27.01% of reported input cached. The sixteen unchanged reference outcomes and four newly acquired reference outcomes are each charged once. Scoped-evidence comparison and failed method adoption.

A subsequent engineering integration reuses the evidence-dependent repair rule already present in earlier implementations. It replaces the candidate's fresh-loss cutoff with current-record refitting and separate-trajectory validation; the other branches and numerical operators remain unchanged. All five source practices and ten cross-source targets pass. A

later offline replay of the exact unmodified candidate also solves all ten targets, allowing a direct development comparison: observations fall from 1,722 to 1,628, measurement units from 1,278 to 852, and computation from 50,388 to 29,020. Both response-change targets recover without meter readings; failed scoped validation in both instrument-change targets leads to measured repair. Those structural cases incur 48 additional computation units in total. The five source and twenty new target executions use no model calls; the twenty earlier reference outcomes are reused. Historical repair integration.

F.8.10 Adoption of a supplied repair program

We then test adoption of this supplied program. Three independent Luna authors start with the unpublished candidate as their current file and receive both programs, their provenance and compact source-practice evidence. Candidate positions alternate and practice confirmation streams advance. Each author copies the historical repair program using ordinary Python and submits it on the next call; all fifteen fresh practices pass the unchanged publication gate. The six calls consume 145,243 tokens, with no reported cached input. Each final source is identical to the supplied program, so its existing ten target outcomes are reused as historical evidence; no new target execution is counted. Adoption of supplied research programs.

F.8.11 Learning to reopen a validated response

A further task changes the response after an instrument-supported response has passed validation. The supplied program then stops after fresh refutation and solves ten of twelve cross-source targets. Three Luna authors given this source counterexample publish no revision: two spend their calls inspecting and editing; one confirms three original cases before a provider-capacity interruption. We then connect each source edit to the existing paid practice engine, immediately returning execution feedback on the added case. Two of three authors publish programs satisfying all six source conditions; each frozen program solves all twelve targets without false confirmation. In one successful trajectory, the author repairs repeated acquisition and stale-record mixing, then fits a current measured batch and reduces fresh MSE from 2.03 to 3.04×10^{-19}. Both programs preserve earlier reuse and scoped-repair behavior. The third author passes the added case once but breaks it in a later edit and does not publish. All 25 practices, including ten failures, are retained. This condition uses 29 calls and 1,750,532 tokens, with 27.16% of reported input cached. Its 24 new target executions evaluate two programs on the same twelve tasks; the unchanged controls are reused. The first published program uses 2,296 target observations versus the unchanged program's 2,010. Method revision with immediate experimental feedback.

F.9 Recovering a supported earlier method

The third author in the immediate-practice study passes the added response-change case with trial_003 at fresh MSE 1.50×10^{-16}. A later edit removes the second acquisition-and-repair path; trial_004 then encounters fresh MSE 0.630481 and stops. Its final program is the failed edit. This history motivates retaining practiced source candidates with their own confirmed and failed cases.

A first recall continuation gives three authors four calls each with the actual failed state and a source-candidate ledger. All three spend their allowance inspecting evidence; none restores a source or publishes. The resume context is approximately 20,700 characters because it contains a long escaped prior output. Ordinary execution returns only the last 12,000 stdout characters, so printing that context can omit the state being sought. A compact-context continuation preserves the complete prior action in a separate file. It again yields no publication or explicit restoration in three four-call attempts. Two authors practice drafts that pass five cases but fail case_4; the third confirms only case_5. The fourteen new practices concern new drafts. These are two development continuations of one failure state. Initial recall; compact-context recall.

The next intervention uses a supported older source as the submission comparison. Before testing the draft, the host identifies an earlier different source with positive practice evidence and no unresolved failure on a required case. A failed draft triggers testing of that source, first on the failed case if it lacks its own confirmation, then on its remaining missing cases. All original publication requirements remain in force. If comparison fails, the draft is restored and publication remains unsuccessful. Confirmations are not combined across sources; a later same-source failure revokes that case's support.

In three Luna attempts under the earlier four-call allowance, one author reaches submission. Its draft fails case_4; the host tests trial_003 on that case and the four other missing cases. The source's existing case_5 confirmation is reused, and it publishes at 6/6. The other authors end in a provider-capacity interruption and call exhaustion while inspecting files. The published source passes 12/12 frozen targets with zero false confirmations, in twelve new executions.

New authoring uses 11 calls and 346,259 known tokens, with one call's usage unreported and 19.02% of known input cached. Author practices cost 2,040 observations, 1,728 measurement units and 127,720 computation units. Target execution adds 2,208, 1,728 and 128,608 in the same resource order, without model calls. A separate preceding offline submission diagnostic costs 1,176 observations, 1,080 measurement units and 79,438 computation units. The comparison is 1/3 versus the preceding compact-context cohort's 0/3 author publications; these are repeated attempts on one development state, not independent tasks or a randomized reliability estimate. The restored source is LLM-authored, while its selection in this intervention is performed by the host. Retained-method comparison.

F.10 Cohort-specific qualifications

The live hypothesis-retention cycle contains two parameter instances within one family. It tests historical retention's incremental value and establishes neither a population-level accuracy gain nor stable long-horizon benefit. Archived fit callbacks can rebuild parameters on their original rows; the two retained returns do not fit on current target labels. Predictive success also leaves structural explanations unresolved: one latest-only Agent attributed an old-coefficient failure to insufficient history and adopted nine lags, although a three-lag model also solved that task. Research on parameter failure versus structural inadequacy remains necessary. The subsequent diagnostic pilot uses the same two sources; its replay is neither an independent sample nor an isolated causal estimate of the interface repair. Failure of one fitting procedure does not establish that every model in its family is inadequate.

The later 4/4 routing replay reuses the same known tasks and does not isolate the new chooser's effect: the live Agent never invokes it. Its instrument components are not separately assessed on prospective readings, although the composed response passes fresh confirmation and terminal prediction. An earlier startup was operator-interrupted after concurrent tests exposed interface errors; its seven saved calls and 178,244 reported tokens remain separately recorded, with unreported in-flight usage unknown. It supplies no task success/failure outcomes.

The subsequent advice intervention has one discordant pair out of four known tasks derived from two apparatuses. It isolates visibility within the shared tool environment, rather than the complete framework versus a vanilla Agent. Each condition is sampled once per target. The observed additional success and lower aggregate token use do not establish a stable comparative advantage, and measurement cost increases. The later persistence study uses three successfully learned sources out of four new apparatuses and retains the blocked targets. One provider call has unknown usage, so its complete token total is unavailable. That study also exposes a routing dead end after fitting an instrument without paired readings. A subsequent repair restores acquisition and passes offline confirmation; it is outside the frozen live comparison.

The archived-Python method studies use two previously studied apparatus families. The initial replay tests an unpublished last candidate; the later author-requested publication uses a revised source and host-scheduled completion of missing practices. Those stationary comparisons match fixed-controller accuracy at higher observation and computation costs. The subsequent selective-repair study uses an announced change checkpoint, explicit behavioral publication requirements, and an additional prompted completion phase. Its eight targets do not establish autonomous change-point detection. Fitting families remain supplied and experimental sequences are embedded in the source. The learned revision still purchases meter data after a validated refit fails fresh confirmation. The subsequent five-case study and manually arranged scope comparison establish that current-record refitting can separate two repair needs, but its author fails to adopt the demonstrated choice. The diagnostic is researcher-directed evidence, not an autonomously selected experiment. The subsequent successful integration is a manual edit, evaluated on the same five conditions and two target archives. Its exact-parent comparison is added after the integration run; it measures a local development intervention, not autonomous method learning or independent confirmation on unseen task families. Interface clarifications and additional author allowances are part of these development interventions. The adoption study supplies a human-integrated program and its source-practice evidence; three successful authors establish a narrow adoption result, not autonomous invention or general retrieval from a large library. It also repairs stale author context after an ordinary Python source edit. The separate effects of this repair and the supplied library are not isolated. The subsequent response-reopening study remains within these known families and an announced checkpoint. The host initiates practice after an edit; it does not supply the repair or initiate publication. The three development attempts do not establish a reliable population-level advantage, and one control author is interrupted by provider capacity. The successful programs rebuild the instrument as well as the response, increasing acquisition cost. Token stopping is checked before a call and may be exceeded by that call's reported usage. At this stage, retention of previously successful method revisions remained unresolved. Appendix F.9 reports the subsequent host-selected

recovery and distinguishes it from autonomous recall. Instrument-assessment failure and open-ended acquisition revision remain untested. Development changes do not isolate the contributions of interface repairs, prompting and requested reasoning effort.

The historical-method LLM pilot contains two development pairs with supplied response candidates. The book is also visible in the prompt, so its language-level influence is not isolated from executable reuse. The observed successful reuse does not establish stable net benefit or new hypothesis invention.

The temporal controller in Appendix F.2 uses a developer-authored revision procedure and existing candidate families. Its same-data history control tests the value of a learned instrument coordinate. The subsequent flat comparison finds equal accuracy with fewer measurements and more response fits; it does not establish an independent benefit of separate child confirmation. Earlier raw-Python temporal LLM pilots complete zero of two targets in both Code and Hegel arms. The shared-operator pilot in Appendix F.3 is a separate, single-apparatus development comparison. It measures selection of prescribed operations, not invention of their fitting algorithms, and still establishes no advantage of executable memory. The later repaired-interface comparison in Appendix F.5 completes all four cases in both conditions. It tests two previously studied calibrations with supplied fitting operators and does not isolate a causal effect of the interface repair or establish overall method-retention gains. Appendix F.6 independently varies instrument and response parameters within that same recurrent family. Paired worlds share parameters and are not independent population samples. Five capacity interruptions prevent a complete initial live comparison; its three completed cases per arm cannot be interpreted as equivalent algorithmic success rates. The subsequent successful method-learning run is explicitly prompted to repair a supplied counterexample, with prescribed fitting operators and publication cases. Its learned trigger responds to any preceding failed confirmation. Appendix F.7 repairs stale evidence scoping through a second, explicitly prompted author phase. Its refit-both strategy does not establish selective diagnosis of which supporting component failed. The two author phases differ in their supplied failure, feedback presentation and stochastic trajectory, so their outcomes do not isolate the causal effect of the compact chronology. Appendix F.8 subsequently demonstrates instrument retention versus reopening under an explicit selective-repair objective. Its initial regression and added practice case show that preserving earlier capabilities is not automatic. The program still uses prescribed instrument-assessment operators and fixed acquisition patterns. It does not determine whether response coefficients also changed when an instrument is refuted. The disclosed checkpoint does not test detection of arbitrary unannounced drift or noisy failure sources. The frozen contrast compares old and revised methods, not ordinary Code agents or an additional live-LLM ablation. Parameter transfer does not establish new structural families, and additional authoring/compute costs prevent an overall cost-dominance claim.

The general recursive-research formulation is broader than the demonstrated algorithms. The calibrated-response study establishes live two-level supporting investigations, but its question types, dependencies and 96-hypothesis language are prescribed. Its structural transfer holds out source-to-target compositions, not primitives from the researcher, and all three researchers reach equal accuracy under the common ceiling. The earlier nested-relation case uses prescribed revision operators and a bounded experimental loop; its dependency graph is compiled from completed fitting steps. It does not establish autonomous generation

of arbitrary-depth research subproblems, formal theorem proving, or self-invention of the rules of justification. The RL, robust-optimization and causal-identification connections are conceptual instantiations, with no reported end-to-end experiments in those domains. Numerical convergence, empirical confirmation and deductive proof supply different kinds of support. An additional assumption supports a conditional claim, and some assumptions cannot be tested from the available observations. The Hegel-inspired preservation principle concerns earlier valid content within its scope; a theory containing every historical claim need not exist. Priors can improve search efficiency but cannot guarantee freedom from overfitting.

The reported results are controlled studies of finite scalar and object-process families. New apparatus instances vary realizations within those families; they do not establish transfer to unseen scientific domains or arbitrary new structural compositions. The object interface supplies a schema, variable roles, and measurement options. The joint-growth interface anonymizes roles and changes units, but public ranges constrain candidate coordinate transformations and the response families remain algebraic analogs. Primitive operators, instrument catalogues and search bounds are designed in advance. The library changes the effective search language; learned procedures compose existing laboratory commands and fitting subquestions. Neither mechanism establishes unrestricted invention of representations or experimental tools.

The studies do not isolate every action-selection mechanism or establish reliable autonomous choice among all revision levels. A preserved composition can retain an obsolete component with a cancelling correction, so successful repair need not produce a minimal expression. The response-retention intervention uses a preselected shared challenge; it does not test autonomous choice of that challenge. Its additional conditional-feedback record provides no benefit. Joint growth learns executable scope conditions in a separate family, but this does not repair the earlier null feedback result or establish general validity predicates. The separate target-binding withdrawal mechanism also cannot distinguish old and changed regimes when their public task identity is the same.

The component-change study covers a response-gain change, a relation-radius change and an unchanged condition. The gain change alters a contribution to the readout rather than the physical exchange law. Both revisions use existing program and research families; the result does not establish general discovery of new mechanisms or optimal selection among arbitrary research actions. This study contains no unresolved outcome, so it does not explain or repair the earlier deletion arm's stopping failures.

A successful fresh trajectory is finite evidence. It does not establish logical conservative extension, a uniform guarantee over the old scope, or statistical immunity to adaptive overfitting. The 11/12 procedural result and the 72/72 task-accuracy result measure different properties. The mechanism studies contain 12 instances each. The three subsequent object-process studies and the joint-growth study each contain 30 apparatus instances, not 30 scientific domains. In the retention intervention, deletion produces four internally confirmed first-revision candidates that fail terminal evaluation. Joint growth also has two such persistent-arm failures and six externally successful but unconfirmed tasks. Its stochastic pass rule is deliberately broad and does not certify calibrated uncertainty; measured interval coverage remains below nominal.

The task-local controls retain the corresponding researcher's within-task capabilities, including conditional-expression search in the joint study. The other controls cover particular algorithms: the dense fixed-basis learner lacks adaptive model selection, and passive research removes adaptive revisits. A stronger fixed-family learner and an independent code agent remain useful comparisons for the earlier task families. The new calibrated-response study includes a strong flat active control. Its completed common-model comparison also exhibits a ceiling for ordinary Code/ReAct and no Hegel advantage (Appendix H.3.11). The public finite grammar and direct response access make it a weak test of autonomous representation or research-method discovery. The replacement trajectory environment remains a development task. Source removal identifies knowledge availability and its downstream consequences, not a complete attribution of all sources and interactions. Method deletion isolates a procedure-book effect, but not the contribution of each compiled step. Static branch and dense-fit diagnostics are not complete-agent baselines. The new-expression resource interval includes zero.

The inquiry-session comparison is offline and uses the same native scheduler without available transfer knowledge as its removal control. Its 12 successful consumers require more research expenditure. Source returns compare a fitted transferred explanation against its frozen incumbent; they do not separately identify the causal contributions of binding, fitting and the learned call. The same-apparatus repair rerun adds no independent sample. These outcomes do not establish the complete combined loop under autonomous LLM control.

Charged computation differs from measured process CPU time, which also depends on the machine and workload. Main-arm order is randomized in the joint study, while retention and additional deletion branches run in fixed orders. Their CPU differences can include cache and order effects. Joint-growth computation savings coexist with higher actual CPU, and acquisition differences have intervals that include zero. The results establish neither a universally favorable scalar cost trade-off nor stable long-horizon net benefit. The comparative performance gains come from offline numerical and symbolic research; the completed language-model comparison shows no overall gain.

The studies use separate frozen runtime snapshots from the development sequence. In particular, the source-deletion composition snapshot precedes subsequent observation-view repairs. The four-arm study uses one later snapshot for all arms and includes every registered instance. Comparisons between the studies cannot identify the effects of intervening software changes because their apparatus cohorts also differ.

F.11 Autonomous selection from method history

This experiment separates model-selected recall from the host comparison in Appendix F.9. Twenty fixed author attempts resume the same historical regression state: ten retain all practiced source candidates, and ten remove the supported sources and their trial files while retaining the actual failed candidates and outcomes. Outcome labels are unchanged. The removed supported program had passed only case_5 at the starting checkpoint; cases 0–4 remained untested for that source and had to be completed after selection. Every author sees a direct summary of all available sources, their latest case-specific evidence and untested cases. Candidate display order alternates across replicas; no source is recommended or

preselected. Both conditions disable the host's fallback to an older source. Restoration or copying must therefore follow an action selected by the model. Editing and practice remain available, and submission applies the original six-case publication requirements.

All authors use gpt-6-luna with high reasoning and the same ceilings: eight model calls, 350,000 reported tokens, 2,400 observation rows, 1,500,000 charged computation units and 180 seconds of cumulative Python execution per author. The provider separately uses a 180-second timeout per model call. Further confirmation seeds receive a replica-specific offset, with the existing practice-index increment retained. The available histories contain five and four prior practices, respectively, so the conditions have equal budgets but do not use identical future confirmation streams. The twenty attempts are repeated decisions on one failure state, rather than twenty independent research tasks. Provider failures and unfinished attempts remain in their registered condition.

The outcomes distinguish selection of supported history, unchanged recall that passes publication, new source edits, submission of an unchanged refuted source, inspection-only attempts and service failures. Restoring a refuted program for subsequent revision is a legitimate research action. Only a published source identical to the program already evaluated in Appendix F.9 inherits that program's twelve recorded target outcomes; those outcomes are referenced once, with no new target execution. A newly authored source receives only its observed source publication result in this study; the subsequent frozen transfer is reported separately in Appendix F.12. The larger call allowance, direct evidence summary and clearer continuation instructions all differ from the earlier four-call recall attempts, so a cross-study improvement would not isolate one of these changes. Registered protocol.

All twenty attempts complete their registered allowance or publication. With supported history, all ten authors autonomously select the old source; seven publish it unchanged and one edits it before successful publication. Two exhaust their call allowance without submitting: one has confirmed cases 0, 1, 2, 3 and 5, while the other has only the inherited case_5 support. Neither is counted as a publication. Without the supported source, all ten authors revise; two publish, five exhaust tokens and three exhaust calls. No author submits an unchanged initially refuted source. Every published program passes the six-case gate, including the same reuse, selective-repair and scoped-repair requirements.

Table F1. Newly incurred recall-study costs. Totals include unsuccessful authors and all their paid practices. Historical source acquisition and the previously completed twelve-target evaluation remain separately recorded.

Resource	Supported history retained	Supported history removed
New practice executions	45	62
Model calls	53	70
Reported tokens	1,687,907	3,717,879
Observation rows	7,392	14,704
Measurement units	4,104	19,920
Charged computation	310,562	1,235,959
Python execution seconds	52.03	133.67

The complete study uses 123 model calls and 5,405,786 reported tokens, with no unknown usage. Cached input accounts for 1,252,096 of 5,229,980 input tokens (23.94%). One recoverable model-output JSON decoding error occurs in each group: a value-decoding error in a retained-history attempt and an invalid escape in a removal attempt. Both remain charged events, and the attempts continue. No attempt's final status is provider failure; the remaining unsuccessful outcomes are the call and token limits listed above.

Seven final programs are identical to the source evaluated in Appendix F.9. Its twelve successful target outcomes are referenced once, with zero new target executions. The retained-history author that revises after recall and the two successful removal authors publish different sources; only their six-case source evidence was available in the original recall study. Appendix F.12 now evaluates all distinct published sources on fresh target trajectories. The 8/10 versus 2/10 contrast concerns successful completion on one shared development state, with different future confirmation streams. It supports local usefulness of the available method history, while leaving new task families and repeated replacement sequences untested. Final results; costs and recorded format errors.

F.12 Fresh transfer of every distinct recalled or revised publication

This follow-up freezes the final sources actually published in Appendix F.11. Literal source equality gives four programs from ten publications: `published_0` is the unchanged supported source published seven times; `published_1` is the edited source from retained-history attempt 01; `published_2` and `published_3` come from removal attempts 05 and 08. All twenty author attempts, including ten nonpublications, retain their original status and costs. Seven equal publications do not create seven independent transfer methods.

Before executing targets, the protocol records these four programs, the existing fixed `historical_bridge` repair, all task specifications and all trajectory seeds. Each receives the same donor predictor and paid prefix on each target. Three available source archives each supply eight conditions: unchanged, response-only, instrument-only, both changes, a post-validation response change, a post-validation instrument change, a later response change, and return to the old condition after a changed prefix. The delayed changes use the existing laboratory's row checkpoints, including the late change after 167 rows and old return after 96. One donor was used in authoring; two were not. The task families remain finite-history and recurrent instruments with quadratic responses. This is new trajectory and intervention-combination evidence, not cross-domain method discovery.

Every execution permits 384 observations, 360,000 charged units, 120 seconds of Python execution and twelve method steps. It buys the common three-by-24-row prefix and 24-row fresh confirmation from that allowance. There are no model calls or tokens. The executor is the existing `PreparedPythonMethodAgent` with `arm=code_react`; no additional support graph or Hegel manifest gate is introduced. All 120 research executions stop before terminal scoring. Each frozen predictor is then evaluated on a separate 128-row trajectory with MSE below 10^{-10}. Source programs are not changed after target results.

Table F2. Fresh transfer after source deduplication. Outcomes are paired on 24 targets, blocked by three donor memories. CPU is sequentially measured host process plus waited child CPU; it differs from the separately recorded elapsed Python execution time.

Frozen program	Confirmed and correct	Observations	Measurements	Charged computation	CPU seconds
Unchanged supported source	24/24	4,464	3,888	279,984	34.13
Edited after recall	24/24	4,464	3,888	279,984	33.65
Removal-author revision 05	24/24	4,464	3,888	280,032	35.24
Removal-author revision 08	24/24	4,608	3,888	277,728	33.70
Existing fixed repair	21/24	4,161	3,195	115,893	18.23

There are zero false confirmations. Each published program passes all eight conditions for all three donors. The fixed repair stops without completing the three late-response-change targets, while its other 21 targets succeed. All four publications therefore have frozen downstream support; the three changed sources do not improve accuracy over the retained successful source. Their greater acquisition and execution costs relative to the fixed repair accompany broader completion and are not an equal-accuracy efficiency gain. The unchanged source and the first edited source have equal charged resource vectors; small wall/CPU differences are descriptive timing observations.

Prior authoring costs remain the full 123 calls and 5,405,786 reported tokens from Appendix F.11. Historical source acquisition retains all four attempts, including one failure; it is not newly charged 120 times. This study adds no model calls. The 24 paired target specifications and three shared donor histories are the units of the extension, not 120 independent environments or four independent discoveries. The authoring state remains the single regression state of Appendix F.11.

Protocol, complete results, compact results, and runner preserve the exact sources, unaltered author outcomes, failed source acquisition and resource vectors. The distinct execution semantics of earlier controls are summarized in the archive-control evidence.

F.13 Executable hypothesis delegation and reuse of paid refutations

Task and inherited mechanisms. A simplified local crystal-control fixture uses the DiscoveryWorld agent interface. Instruments expose density, gap and a separately acquired signal; a controller installs a crystal, adjusts frequency and observes Boolean activation. Reference crystals supply measured features and successful settings. The task is to activate two further crystals under a changed response mechanism. Successful settings are operational labels inside an activation band, rather than exact latent resonance values. A declared basis proposes a response family; the existing researcher fits coefficients, compares it with paid positive and negative outcomes, investigates simpler supports and jointly revises coefficients and a shared connected activation band. World actions include measurement, installation and tuning as well as outcome acquisition.

The current comparison follows an earlier model-authored cubic proposal and a frozen content intervention: retaining that proposal solves 8/8 cubic conditions versus 0/8 after deletion, with square controls 2/2 in both conditions and aliased-sensor controls 0/2 in both. A separate same-program agent comparison supplies the larger sixteen-declaration archive to both Hegel and ordinary Code/ReAct: both adopt it in 9/9 runs, and complete 5/9 versus 6/9. These earlier results and failures are retained in the structure-transfer record and matched-adoption record. The present experiment returns to the historical nine-declaration stage to investigate adoption of structural revision after those candidates fail.

Fixed assignment comparison. All six continuations restore the same complete 184-observation failure, including the nine declarations, files, notes, questions and prior refutations. Both conditions use Hegel, the same numerical operators and the requested model alias `gpt-6-luna` with medium reasoning. Each receives an additional allowance of 24 model calls, 512 world actions, 600 cumulative Python seconds and 5,400 wall seconds; an inquiry can consume at most twenty calls, leaving the ordinary parent loop the remaining allowance. Individual Python and provider limits are 30 and 300 seconds. The prior 24 calls and 184 world actions remain recorded acquisition costs, outside these continuation totals.

The open assignment asks for an explanation, an informative experiment or a specific unresolved prerequisite. The executable assignment restores a historical instruction to submit a measured-variable basis through `reference_hypothesis`, with the existing fitter and physical controller responsible for execution. The old inline measurement table is removed; both assignments refer to the same current paid records. Neither supplies a new basis or target setting. Three attempts per assignment are fixed before execution and alternate within the scheduled order. No scientific hints, runtime edits or selective extensions are introduced during this batch.

Table F3. New outcomes and costs of the six continuations. Each column contains three repeated attempts on the same development state.

Outcome or resource	Open assignment	Executable assignment
Physical task completions	0/3	2/3
Attempts adding a structural declaration	0/3	2/3
Formally confirmed new requested points	0	0
Model calls	30	11
World actions	120	225
Research computation units	106,084	933,930
Input tokens	6,598,243	2,303,392
Output tokens	31,047	12,323
Cached input tokens	961,792	421,376
Python execution seconds	193.36	225.14

All 41 calls report usage; thirty have positive cache usage, totalling 1,383,168 cached input tokens. Open-assignment repetitions use 4, 17 and 9 calls; executable-assignment repetitions use 1, 1 and 9. All four failures voluntarily finish before exhausting their allowance.

Executed hypothesis–counterexample–revision chain. Both successes submit a basis containing signal, signal squared and signal cubed. The first least-squares prediction is approximately 7,715.405 Hz and receives an actual negative response. Within that same operator invocation, the existing research program considers simpler supports, selects the single cubic term and jointly revises its coefficients and activation-band parameters using the accumulated Boolean responses. It next selects 7,905 Hz. The target activates at 7,906 Hz during adjustment, and the environment records task completion. This new target completion is separate from the already solved target in the inherited state. Each successful continuation uses one model call and 99 new world actions; no second model decision mediates the post-counterexample numerical revision. The pending 7,905-Hz prediction remains inconclusive at termination.

Reusing earlier exclusions. The completed traces expose redundant trials of old families already excluded by the paid evidence. Their full-family exclusion records had not reached the existing point-candidate filter. After the batch, that filter is connected to the retained exclusions when basis, measured representation and supporting history match; unresolved calculations are not treated as exclusions. Replaying the first successful model-authored proposal from the same prefix reuses eight excluded families covering nine declarations. Task completion is preserved with 35 rather than 99 new world actions and 420,819 rather than 466,965 research computation units. All 45 earlier refuted answers remain. The replay adds no model calls; it tests execution of an already recorded proposal. Its requested point is again inconclusive.

Limitations. This is a local simulated-control fixture, not an official DiscoveryWorld benchmark. The six operator-started continuations concern one development failure, and the cubic structure was discovered earlier in this environment. They therefore establish local rediscovery and assignment-sensitive adoption, not autonomous initiation or discovery across unseen tasks. The third executable-assignment attempt still proposes no new structure. Physical completion during adjustment does not independently confirm the requested point or identify a unique response law. Executable delegation uses more physical actions and research computation; fewer model calls do not establish overall cost reduction. Both arms are Hegel, and the separate ordinary-archive comparison favors Code in total completion. Neither the comparison nor the zero-call replay establishes an advantage of recursive organization. The earlier archive comparison also includes execution-timeout, provider and missing-helper failures, retained in its denominators.

Assignment protocol, aggregate results, per-attempt traces and costs, and paid-exclusion replay retain the distinct evidence records. The original six-run runtime remains frozen separately from the subsequent repair.

F.14 Autonomous initiation and return to the parent program

Protocol. A fixed twelve-run cohort begins with zero paid observations and the nine-declaration archive preceding the earlier cubic discovery. Changed-mechanism and aliased-measurement conditions each receive three Hegel and three Code/ReAct runs. Both agents share the supplied program, numerical researchers, inquiry tools and the model and overall allowances of Appendix F.13; a native inquiry may use the remaining model calls. The model makes the first adoption decision; the runner supplies no intermediate research assignment,

new basis or target setting. All twelve agents adopt the archive. The Code condition is a strong executable-archive comparator; its shared tools also support inquiries.

Table F4. Native research from zero observations. Costs sum all three repetitions per row. C5 confirmation requires an actual positive outcome at the requested point associated with the newly declared basis or its generated support, excluding earlier C4 calibration.

Condition	Agent	Task completion	New C5 confirmation	Model calls	World actions	Research compute
Changed mechanism	Hegel	3/3	2/3	72	774	2,737,254
Changed mechanism	Code	2/3	2/3	28	614	3,578,238
Aliased measurements	Hegel	0/3	0/3	59	570	2,489,110
Aliased measurements	Code	0/3	0/3	51	483	2,542,336

The 210 calls report 37,691,007 input and 174,179 output tokens, including 8,274,176 cached input tokens; 189 calls have positive cache usage.

Hegel independently submits a cubic-only signal basis on its second call in all three changed-mechanism runs. Its first fitted target, 7,879.943 Hz, fails physically. The model then pursues supporting questions, including measurement repeatability and alternative target ranges. After the child returns, the existing parent acquires negative reference-boundary responses and revises its target experiment. One run activates at 7,906 Hz while approaching the unconfirmed 7,905-Hz request; two refute 7,905 Hz and subsequently confirm the requested 7,906-Hz point. Each uses 24 model calls. The two successful Code runs use two calls each: their composite signal basis first fails at 7,715.405 Hz, after which the shared researcher simplifies and revises it to a confirmed 7,906-Hz experiment.

Returning an executable result to the parent. The first Hegel trace exposes a control-allocation error: an available parent experiment executes only after the model allowance is exhausted. A local return already exists, but completing a hypothesis trial does not invoke it, and the parent does not accept that return as a signal to continue. Connecting these existing mechanisms preserves the unresolved scientific question while returning the current executable experiment. A fixed replay makes all 24 original model decisions available under the original 24-call allowance. The old runtime consumes 24 decisions, 241 actions and 855,235 research units; the repaired runtime completes after two decisions, 180 actions and 749,738 units, leaving 22 calls available. Both activate at 7,906 Hz while approaching the unconfirmed 7,905-Hz request. This comparison uses no new model calls. An earlier candidate that expanded direct-basis eligibility is retained as a negative result: under a separate two-recorded-decision protocol, it changes a 180-action success into a 229-action failure and is not adopted.

Limitations. These are three repetitions of each previously used development condition. The results establish autonomous initiation, structural proposal and counterexample-driven revision within the available expression language; they do not isolate a benefit of recursion. The return intervention fixes execution of one recorded trajectory; Appendix F.15 reports its subsequent autonomous runs and regressions. The aliased condition supplies no distinguishing additional instrument, and more deterministic functions of identical readings cannot resolve its shared-response conflict. One Code run terminates on an invalid

action-name assertion in the fixture; it remains in the denominator and its recorded costs end at that failure. Source snapshots retain the modules loaded at launch; dynamically imported dependencies were subsequently preserved separately for replay.

Protocol, complete totals, per-run declarations and trials, and run analysis retain the twelve outcomes. The fixed return-intervention replay and rejected eligibility intervention retain the separate execution diagnostics.

F.15 Returning refutations to the author

Follow-up cohorts. The broad return intervention does not produce stable autonomous gain. A fresh twelve-run cohort completes changed-mechanism tasks in 1/3 Hegel and 2/3 Code runs; both agents remain 0/3 under aliased measurements, with no new C5 requested-point confirmation. After a shared current-observation interface repair, a six-run changed-mechanism cohort completes 2/3 in both arms, with requested-point confirmation 0/3 versus 1/3 and 60 versus 34 model calls. All outcomes are retained in the twelve-run record and six-run record.

A controlled continuation. Both latter failures submit a signal-by-gap basis that the researcher subsequently excludes. The automatic return condition nevertheless ends the author session because the feedback also offers an experiment under an older family. We restrict this return: a verified exclusion of the submitted family leaves its author active. Paired original-decision replays return the Hegel and Code authors to the same C5 investigation after 171 rather than 238 observations, saving 1,791,854 research units each before the next decision. A successful control retains all 22 recorded decisions, 206 observations and 902,387 research units.

A fixed six-run continuation then compares the two policies in Hegel, with three new model repetitions each. Every run replays the same three original decisions and shares the first 171 observations and nine initial declarations. The model is subscription `gpt-6-luna` at medium reasoning; the original 24-call allowance leaves 21 new calls. World-action, cumulative Python and wall-time ceilings remain 512, 600 seconds and 5,400 seconds. Both policies use the same corrected historical-object view. No new scientific assignment, formula or setting is supplied during continuation.

Table F5. Continuation after one shared refuted proposal. Actions and research computation exclude the common 171-observation/934,942-unit prefix. Both rows are Hegel. New function-basis submission excludes the replayed interaction proposal and does not count every supporting question or premise.

Return policy	Task completion	New basis submission	C5 point confirmation	New model calls	New actions	New research compute
Original	0/3	0/3	0/3	63	374	5,478,007
Retain excluded-family author	1/3	1/3	1/3	63	342	5,983,148

Successful mechanism. On global call 11, the repaired author submits a cubic signal basis. Its fitted 7,879.94-Hz prediction is refuted at 7,880 Hz. The response researcher

retains cubic parameters and activation bands compatible with the observed positive and negative outcomes. A supporting child returns the point refutation on call 12. After the model allowance ends, the parent controller selects 7,906 Hz; observation 300 confirms activation at that requested point. Of 32 finite response hypotheses, eight predict activation and 24 predict inactivity; the observation retains eight and rejects 24. The new structure comes from the model, while the successful experiment comes from the existing researcher and controller.

Limitations. These are repeated continuations of one opened development state, not independent task discoveries or a Hegel-versus-Code comparison. The two repaired failures open further premise questions or test old-family settings but submit no new function basis. All six use 21 new calls; repaired runs use more research computation. That cohort also exposes a handoff defect: the successful run already offers the 7,906-Hz experiment after call 11, but a pending child blocks return; completion of that child does not reconsider the handoff. The parent only resumes when the model allowance is exhausted. This single confirmed continuation does not establish reliable research allocation or a unique benefit of recursion. No run ends through a provider, runner or timeout failure. Cache usage is reported for all 126 new calls, with 109 positive cache-hit calls and 4,250,112 cached input tokens; replayed decisions are excluded from that denominator.

The protocol, six-run summaries, analysis, analysis script, and paired replay and diagnosis retain the complete comparison.

Integrated connections and fresh decisions. A subsequent fixed six-run comparison adds retained full-family exclusions, reconsideration of a parent experiment after a blocking child returns, and physical trials within an author proposal without adopting its draft. Both arms are Hegel; the control already retains excluded-family authors. Three recorded decisions precede fresh Luna choices under the same allowance. The repair changes experiments inside the third recorded action, so the first fresh decision follows 171 versus 180 observations. Costs below cover the complete run.

Table F6. Fresh decisions after integrating the three connections.

Implementation	Task completion	C5 point confirmation	New model calls	World actions	Research compute
Before integration	3/3	0/3	27	685	3,682,174
Integrated	1/3	1/3	43	892	7,734,499

All three control successes introduce cubic signal terms and activate at 7,906 Hz while approaching other requested settings. The integrated success instead confirms its requested 7,906 Hz. One fresh model call proposes the composite cubic basis; the existing researcher refutes its initial 7,715.405-Hz prediction, tests 7,881 and 7,961 Hz, and selects a cubic support before confirmation. Both integrated failures exhaust their 21 fresh calls. One pursues older questions after a mixed quadratic-interaction proposal; the other returns to equivalent quadratic representations after a density-by-gap proposal is excluded. In the latter, author-side trials at 2,672 and 2,676 Hz execute and return their negative outcomes correctly, but fail to produce a useful revised explanation.

Limitations. These three repetitions per implementation concern one opened development task, with changed observation histories and no shared random-seed pairing. The comparison does not isolate component effects. Better confirmation in one continuation accompanies worse task completion and higher aggregate cost. No provider, runner or timeout termination occurs. The 70 fresh calls report 55 positive cache hits and 5,501,440 cached input tokens. Cache counts in both this cohort and Table F5 are recomputed from raw provider completions: nested research-event order made prefix slicing undercount positive hits; original run summaries and token totals are preserved. The integrated comparison, analysis and analysis script retain all six outcomes.

Reopening after new parent evidence. The first integrated failure returns a mixed quadratic-interaction proposal to the parent, but subsequent negative responses do not resume its author. We connect the existing shared-premise research obligation to the same author after a completed parent experiment adds evidence. A fixed six-run comparison uses the integrated runtime in both arms and the same seven recorded decisions, followed by at most 17 fresh subscription Luna/medium calls under the original 24-call allowance. The first fresh decision follows 297 observations in the control and 223 after reopening; actions and research units therefore cover the full run, whereas model calls and tokens count only fresh provider requests.

Table F7. Fresh decisions with author reopening after new evidence.

Implementation	Task completion	C5 point confirmation	New model calls	World actions	Research compute
Integrated control	0/3	0/3	51	1,046	8,040,720
Resume the same author	1/3	1/3	47	1,004	7,017,205

In repaired repetition 3, global call 13 (the sixth fresh call) proposes the single coordinate signal[3]. Predictions at 7,880, 7,905, 7,876 and 7,879 Hz fail, but the retained response family remains useful. The model subsequently proposes other coordinates. After call 20 proposes a joint signal-density and signal-gap basis, the existing researcher selects the earlier cubic family, and observation 274 confirms its requested 7,906 Hz. The final interaction proposal is not the explanation confirmed by this trial. The 7,906-Hz candidate was already present in an earlier experiment panel. Adding the new basis changes the global family scope and resets stagnation from three cycles to zero, allowing the retained candidate to run without a new response or nonlocal progress at that comparison. Thus the successful timing depends on the scope-reset policy, not solely on a newly informative explanation. The model supplies candidate structures; the existing response researcher revises their compatible parameters and selects the experiment. Both repaired failures resume the intended author but test further individual settings, propose no new basis, and explicitly end that inquiry before later exhausting their overall model allowance.

Limitations. This is a three-repetition comparison on one opened development failure, with no paired random seeds, Code arm or independent task generalization. All five failures end at the model-call limit. The successful trace demonstrates within-task retention and reuse of a model-proposed family, not a new cross-task transfer result or a unique benefit

of recursion. Aggregate research computation falls by 12.7%, but input tokens rise from 12,764,920 to 12,988,374; this is not a uniform cost improvement. All 98 fresh provider completions report usage, with 80 positive cache-hit calls and 5,275,136 cached input tokens. The reopening report, fixed protocol, all six outcomes and aggregation retain the full comparison.

Preserving the assignment across reopening. A resumed author's current scientific question had been promoted to its fixed assignment, replacing the original request to revise an executable explanation. We reconnect the existing assignment/question separation: the caller restores the research objective while retaining the author's narrowed question, notes, identity and acquired evidence. A fixed six-run comparison changes only this connection in the caller and inquiry frame. Both Hegel arms have the prior reopening repair and replay seven decisions to the same 223-observation state; they then receive at most 17 fresh Luna/medium calls under the original limits. No cubic basis or known successful setting is supplied in the restored assignment.

Table F8. Fresh decisions with the original research assignment restored. World actions and research units cover the full run; model calls are fresh provider requests.

Assignment on reopening	Task completion	New basis submission	C5 point confirmation	New model calls	World actions	Research compute
Current narrow question	1/3	1/3	0/3	49	1,010	7,399,603
Original research objective	2/3	3/3	1/3	25	895	5,888,311

The first repaired author proposes a single signal^3 coordinate on its first fresh call. The existing researcher tests the fitted prediction, retains compatible response parameters after refutation, and confirms the requested 7,906 Hz at observation 264. The second repaired success submits signal, squared-signal and cubed-signal coordinates on its seventh fresh call; activation at 7,906 Hz while approaching the requested 7,905 Hz completes the task at observation 287, without confirming that requested point. The repaired failure introduces density-signal and density-gap interaction coordinates, then tests eleven individual settings and exhausts its calls. The single control success proposes a cubic coordinate on its fifteenth fresh call and also completes through intermediate activation. Thus submission of a new structure and physical completion are distinct from confirmation of its prediction.

Limitations. These are three unpaired repetitions per arm on the same opened development failure, with no ordinary Code arm or independent task generalization. The two control failures and one repaired failure reach the model-call limit; no provider or timeout failure occurs. Research units decrease by 20.4% and input tokens from 11,737,173 to 6,783,190 in this cohort. All 74 fresh calls report usage: 62 have positive cache hits, totaling 3,038,976 cached input tokens. This result supports preserving the caller's research objective in this continuation; it does not establish stable autonomous discovery or a unique benefit of recursive organization. The assignment record, fixed protocol, six outcomes, aggregation and analysis script retain the comparison.

F.16 Frozen numerical hypotheses on new objects

We separate transfer of a functional basis from reuse of its learned coefficients and response interval. Nine completed source conditions provide archived exact hypotheses of the form $h = (f, \ell, u)$, predicting activation when $\ell \leq s - f(x) \leq u$. We retain every archived joint witness compatible with all final source responses. Both continuation arms receive the same source declarations and controller; one additionally receives these fixed numerical hypotheses. The two target signal measurements are changed to 5.25 and 12.5. Each source generates a successor with the same mechanism and another with its intercept increased by 60, giving 36 paired executions.

Before adaptation, public instrument measurements generate frozen forecasts, recorded before executing target trials. In the same-mechanism conditions, all 11 unanimous positive and 18 unanimous negative predictions match exact requested-point observations. Six further trials have disagreeing predictions, and 7 of 18 objects have no unanimous positive proposal. Under the intercept shift, all 11 old positive predictions fail, while all 18 negative predictions remain correct. These trials use separate laboratories from the continuation study.

Table F9. Numerical knowledge supplied to the same adaptive researcher.

Condition and knowledge	Completed	C4 / C5 confirmed requests	World actions	Research compute
Same mechanism: basis only	9/9	9/9 / 4/9	767	1,866,438
Same mechanism: fixed numerical hypotheses	9/9	7/9 / 9/9	493	328,606
Changed intercept: basis only	9/9	9/9 / 5/9	812	2,658,061
Changed intercept: fixed numerical hypotheses	9/9	9/9 / 6/9	824	3,735,847
Changed intercept: recall then resume fitting	9/9	9/9 / 5/9	812	2,678,059

Numerical reuse reduces same-mechanism actions by 35.7% and research computation by 82.4%. The two lost C4 confirmations comprise one refuted requested point, followed by recovery of an observed positive setting, and one endpoint left unadjudicated when the task terminates. In all nine changed-mechanism continuations, the three initial reference responses reject every transferred model before a frozen target experiment executes. Subsequent ordinary research still completes all tasks, but total computation increases by 40.5%. Initial archive screening costs only 600 charged units; the immediately following general research costs 1,459,016. We then separate initial archive screening from general research and return a rejected archive to the existing local fitting stage. Eighteen further executions preserve all nine same-mechanism action traces, confirmation outcomes and research costs exactly. In changed-mechanism cases, all nine return to local fitting, and seven subsequently enter full research when needed. Actions and substantive outcomes match the basis-only control, with 19,998 additional screening units (0.75%); C5 confirmation returns from 6/9

to the control value of 5/9. This removes the premature research step while preserving numerical reuse. All 54 executions use zero model calls and record no unsupported positive verdicts.

We additionally test later counterexamples with the same nine archives, crossing each with static gain multipliers 0.999 and 1.001. All 18 conditions are retained, including those without later withdrawal. Across 36 paired executions, completion remains 18/18 and C4/C5 requested-point confirmation remains 14/18 and 8/18. Thirteen conditions withdraw previously selected numerical knowledge, six after an earlier requested point was physically confirmed. Returning these cases to local fitting reduces computation from 7,725,123 to 5,650,326 (26.9%), while actions increase from 1,307 to 1,315; nine cases still require subsequent full research. Nine additional same-mechanism executions preserve the earlier action traces, confirmation outcomes and charged computation exactly. All 45 additional executions use zero model calls and record no unsupported positive verdicts. The later-withdrawal protocol, paired results, preservation results, and stage audit record this extension.

Limitations. Source archives are explicitly supplied, and these fixed new objects belong to already developed mechanism families. Archived joint witnesses cover only part of the transient source hypothesis pool. Continuation may refit after frozen hypotheses fail. The stage repair covers initial rejection and later target counterexamples in static task laws; applicability learning across subsequent tasks remains untested in this comparison. The result establishes conditional numerical reuse, without testing autonomous archive retrieval, unseen structure discovery, a unique recursive advantage, or amortized long-term benefit. The fixed protocol, forecast records, paired outcomes, outcome audit, and execution script retain the initial comparison. The stage continuation protocol, 18 new outcomes, and stage audit retain the repair and its reused controls.

F.17 Publishing revised hypotheses while retaining earlier knowledge

We test whether the revised numerical products in F.16 help subsequent tasks, and whether earlier knowledge remains useful when its law returns. We retain all 13 branches in which selected source knowledge was subsequently refuted. Replaying the frozen source inputs preserves their complete action traces, endpoints and charged research computation. Exporting before temporary state deletion yields 183 compatible exact candidates from nine branches; four yield none and remain in the study. Final-source verification costs 92,729 additional units. Earlier archives remain separate from current applicability: publication retains them, whereas new observations determine which members can be used.

Each branch supplies the same final controller and measurement declarations to three knowledge conditions: old only, revised only, and both retained. Successor targets have signal measurements 6.5 and 13.5. One successor preserves the adapted law; another restores the original law, giving 78 executions. The existing response researcher screens fixed models against successor evidence and resumes fitting when needed. No source observations are inserted into successor evidence, and combination does not refit coefficients or intervals.

Table F10. Revised knowledge and recurrence of an earlier law.

Law	Knowledge	Completed	C4 / C5 confirmed requests	Actions	Research compute
Adapted law	Old only	13/13	13/13 / 5/13	944	5,604,998
Adapted law	Revised only	13/13	12/13 / 9/13	703	467,396
Adapted law	Both retained	13/13	12/13 / 9/13	790	983,357
Original law returns	Old only	13/13	11/13 / 10/13	630	742,128
Original law returns	Revised only	13/13	12/13 / 2/13	944	5,715,473
Original law returns	Both retained	13/13	10/13 / 10/13	652	848,196

Adding revised knowledge to the old archive reduces adapted-law computation by 82.5% and raises C5 request confirmation from 5/13 to 9/13. Retaining old knowledge alongside the revision reduces original-law-return computation by 85.2% relative to revised knowledge alone, raising C5 confirmation from 2/13 to 10/13. For all eight empty-revision successors, retaining both reproduces the old-only action traces, endpoints and computation exactly. All tasks complete; the measured gains concern research effort and some confirmations. All 91 executions, including source replays, use zero model calls and produce no unsupported positive verdicts.

Research after compatible knowledge exhausts its experiments. In one Table F10 mixed-archive case, compatible old members offer no executable experiment, yet trigger 609,097 units of full research before local fitting. We connect returned frozen trials to the existing archive screen and local fitter, preserving surviving hypotheses. If fitting exhausts, full research remains available. Repeating the same 78 conditions preserves all 26 both-retained action traces and endpoints. Adapted-law computation remains 983,357; original-law-return computation falls from 848,196 to 20,196 (97.6%), with the same 652 actions. All 13 compatible-but-exhausted recalls enter local fitting. In one revised-only case, a fitted prediction at 3204.855 is physically refuted; four subsequent full-research comparisons lead to a confirmed request at 3203. Its computation falls from 911,705 to 594,336, while actions rise from 80 to 82. The stage protocol, 78 outcomes, and stage audit retain the follow-up separately from Table F10.

Closing a completed point trial. Immediate analysis after acquisition already existed for response-family experiments. Ordinary fitted points waited for the next controller call, which never occurred when tuning completed the task. We attach the existing answer to the adjustment expected to reach the closest accessible setting; support still requires the actual installed-crystal response within the existing 0.51 Hz tolerance. Replaying 22 unresolved terminal trials preserves every recorded physical action: eight observed positives become recorded support, while 14 activations away from the requested setting remain unconfirmed. This public-observation replay uses only previously acquired evidence. Six further complete executions cover three affected patterns, two incidental-activation controls and the preceding 3203 confirmation. All six still complete, preserving every physical action, charged research computation and exported hypothesis: 324 actions and 599,483 units in

total. The three affected executions record their final point confirmation; the other outcomes remain unchanged. The six-run comparison overlaps the replay cases and adds no model calls.

Counting distinguishable forecasts in experiment choice. The earlier response panel already groups models that predict identical outcomes across its offered experiments. We reuse this rule for finite response-member selection, retaining every hypothesis and source for subsequent feedback. Reconstructing all 26 initial retained-archive choices from their paid prefixes identifies six changed choices. A fixed 52-execution comparison uses the same archives, limits and final point analysis in both arms. Both complete 26/26 tasks without unsupported positive verdicts. Actions decrease from 1,442 to 1,410 and charged research computation from 1,003,553 to 645,281. For the 18 successors with genuinely mixed archives, actions decrease from 889 to 879 and computation from 132,625 to 120,552 (9.1%); C4 confirmation remains 16/18 and C5 decreases from 15/18 to 14/18. The eight empty-revision fallback cases retain all confirmation outcomes. They account for most computation saved: in the largest case, a later finite-member comparison chooses reference-boundary experiments instead of distant target requests, reducing comparisons from 11 to nine and actions from 136 to 114 before confirming the same final target. The paired protocol and stratified results record all cases with zero model calls.

A negative test of trajectory-based selection. We next adapt the historical multi-step probe interface to each reactor's observed control menu, forecasting outbound tuning and declared reference restoration. Distinct full-path Boolean signatures score each experiment by $(1 - \sum_g (n_g/N)^2)/(2L)$, where n_g counts retained forecast classes sharing a path outcome, N is their total and L is the number of tuning adjustments. Repeated observations of the same split add cost without adding information. In 52 matched executions, both policies complete 26/26 tasks with 24 C4 and 19 C5 requested-point confirmations. One C5 confirmation is gained and another lost. All 207 predicted adjustments across 38 selected segments match actual controls and readouts; two subsequent adjustments are counted separately. Nevertheless, actions increase from 1,410 to 1,427 and research computation from 645,281 to 841,495 (30.4%). For the 18 mixed-archive cases, actions rise from 879 to 909. A short reference experiment can distinguish models yet delay target confirmation: one square-law case increases from 44 to 54 actions. We withdraw this candidate and restore the preceding selector. The frozen comparison, outcomes and withdrawal record preserve the negative result. All 26 control traces reproduce the preceding comparison, and no model calls are added.

A negative test of target-consequence selection. The existing family researcher compares both possible outcomes through their consequences for other predictions. We adapt this rule to frozen numerical members: first test an unfinished target on which all retained forecast classes predict activation; otherwise prefer guaranteed reductions of distinct forecasts on other target settings, excluding the tested point itself. If no reduction is guaranteed, a conditional opportunity requires both outcomes to retain members. In 52 fixed paired executions, both policies complete 26/26 tasks. Actions decrease from 1,410 to 1,398, while charged computation changes from 645,281 to 645,344. C4 requested-point confirmations remain 24 and C5 decreases from 19 to 18. Across the 18 mixed-archive cases, computation increases from 120,552 to 123,904. In the lost-confirmation case, confirming C4 first supplies an additional counterexample that makes C5 settings 2167 and 2169 equally

discriminating. The controller requests 2167, but activation at 2169 completes the task during adjustment. Both policies observe that activation; only the earlier policy requests and confirms 2169. We withdraw the candidate and retain the preceding selector. The complete outcomes distinguish task completion from prospective confirmation; no model calls are used.

Returning an unfinished experiment after intermediate refutation. Intermediate tuning observations already update the surviving fixed hypotheses, but the controller previously continued to the requested setting even after every member supporting that setting had been refuted. We connect this feedback to the existing unfinished-trial and parent-research flow. The unvisited request remains inconclusive. Seven such returns occur in 52 fixed paired executions: five resume compatible-archive selection and two resume research after complete archive refutation. Both policies complete 26/26 tasks and preserve every case-level C4/C5 confirmation outcome, totaling 24 and 19. Actions decrease from 1,410 to 1,396 and research computation from 645,281 to 643,343. In the motivating case, activation at 2169 refutes all members supporting the requested 2167. The controller now leaves C5 active and returns to confirm C4, avoiding six actions; the original C5 request remains unconfirmed. Two square-law cases instead require two additional actions each; mixed-archive computation increases from 120,552 to 121,535. We retain the integration. All 26 control traces reproduce the preceding baseline, and complete results include zero model calls.

Repairing a response condition before abandoning a trial. We next connect the existing fixed-program band reconstruction to intermediate refutation. Keeping coefficients fixed, paid positive residuals require a connected interval and negative residuals constrain its boundaries. When the pending setting admits a compatible wider interval, the minimal and wider intervals give testable predictions; the controller attempts this local condition repair before returning to parent research. An already active target retains the preceding return policy. In 78 new executions, with 26 archived preservation controls, every paired task and requested-point confirmation outcome is preserved. The 26 preservation cases retain 26/26 completions and 24/19 C4/C5 confirmations, reducing actions from 1,396 to 1,392 and online research computation from 643,343 to 642,626. The 26 new-object pairs use target signals 7.25/14.75 with the same source archives and laws: both complete 26/26 with 13/21 confirmations, while actions decrease from 1,431 to 1,426 and online computation from 524,540 to 524,509. Three of nine condition revisions support continued trials and receive actual requested-point confirmation. In the new-object example, the revised narrow and wide intervals predict opposite responses at 569; its observed activation rejects the narrow interval and avoids five actions. The other six revisions find no compatible positive prediction and return to research. We retain the integration; all results use zero model calls.

Consolidating response conditions for later tasks. An expired interval need not invalidate its coefficient program. We connect the existing residual-band constructor to source knowledge export, preserving every compatible old model and rebuilding intervals only when all archived intervals for a fixed program fail. A negative residual inside the required positive hull still rejects that program. Three unchanged source replays recover all 17/16/24 final paid responses: 15 of 21 saved programs admit revised shared intervals, yielding five programs and 20 finite interval representatives per source. In 24 matched successor executions, using two new object pairs and unchanged or 1.01-times source gain,

both arms complete 12/12 tasks. C4 requested-point confirmations remain 10, with one case-level gain and one loss, while C5 increases from 8 to 9. No unsupported positive verdict occurs. Actions change from 775 to 774 and research computation from 798,735 to 702,817. On unchanged-parameter successors, computation falls from 116,190 to 9,344; the new knowledge is selected in five of six cases and yields a confirmed request in three. In the gained-confirmation case, old sources are rejected after C4 feedback; a retained revised program supports C5=3237, followed by another interval revision and actual confirmation. Including the three source exports and terminal diagnostic exports, the paired computation difference is -96,044 units. Complete outcomes and contribution traces use zero model calls.

Limitations. In the original comparison, combined knowledge costs more than the applicable single archive: 515,961 additional units under the adapted law and 106,068 when the old law returns. Five case-level confirmation losses occur relative to a single-archive control: three follow changed experiments and activation during adjustment; two empty-revision cases exhibited the finalization issue repaired above, where a matching final physical response left its research question open. The point-closure repair covers settled adjustments; earlier activation along an unfinished adjustment can still leave a point question open. The stage follow-up completes 76/78 tasks: two single-archive executions stop at the cumulative 240-second Python limit, after unchanged prefixes of 104/133 and 106/137 actions. These truncated costs are not counted as savings. Four separate reruns, pairing old and new controllers under the same limits, all complete and preserve both full action traces and endpoints. New-controller Python times are 97.58 and 109.74 seconds, versus 127.65 and 141.16 seconds for the controls. Computation falls from 3,256,192 to 2,711,234 and from 3,223,718 to 2,706,143. The confirmation results establish completion within the original limit; they do not identify the specific host-load cause or replace the two raw truncations. The 13 branches come from nine consumed development sources, with explicitly supplied archives and static successor laws. Source learning, publication and successor decision costs are separate; terminal diagnostic exports are recorded separately. The forecast-grouping follow-up adds six actions across the nine mixed-archive adapted-law cases and loses one C5 requested-point confirmation: 2167 is refuted, after which the controller restores the observed intermediate success at 2169. Grouping is relative to the current offered experiments, not a learned probability of source applicability. The withdrawn trajectory heuristic models tuning cost only; navigation, installation and early task termination are recorded as actual execution outcomes. Accurate control forecasts and finite-member disagreement do not establish useful downstream research decisions. In the two costly intermediate-return cases, a negative response at 471 refutes the finite archive, but the control policy activates the reactor at its next requested setting, 472. Returning to research preserves eventual confirmation but adds two actions and 514/489 computation units. Refuting an experiment's current model support does not establish that the experiment has lost its physical value. For the condition-repair follow-up, terminal knowledge export adds 516 computation units in the preservation cohort and 53 in the new-object cohort; including export, the respective differences are -201 and +22. All three locally useful revised intervals are rejected by subsequent C5 observations, leaving no new interval under the then-current exact-window exporter. One rejection is within 7.65e-7 of a boundary, so not every exclusion identifies a substantive mechanism mismatch. The policy prioritizes local condition repair rather than comparing its global value against alternative parent experiments. In the lost C4 confir-

mation, the added archive requests 715 after 717 fails; both are refuted. Restoring the already observed success at 716 completes C4 without a new prospective request at that point. The control instead fits and confirms a 716.147 request at the actual setting 716. The consolidation follow-up still adds five actions and 10,928 research units in its six changed-parameter cases. The newly added source is selected in two such cases, confirms no request, and is eventually rejected in live recall in all six. The comparison covers three consumed development branches and static parameter changes, not independent mechanisms. General applicability learning, autonomous LLM retrieval and long-sequence net benefit remain untested. The original protocol, complete results, source preservation, outcome audit and runner retain the earlier comparison.

Appendix G. Learned hypotheses in SC-KG

G.1 Predictive compression and the same-data control

The initial bridge uses five-dimensional, noise-free RZDT1/RZDT2 tasks in three four-task sequences. Shared and task-local research complete all twelve tasks each. Sharing reduces learning simulator calls from 312 to 240 and charged computation from 15,042 to 4,491, with retained knowledge reused on all nine successors. After learning, each frozen model receives eight new GP training observations and 64 evaluation observations. Mean GP MSE is 5.924×10^{-21} with the learned program basis versus 0.009305 with the native quadratic basis; their feature dimensions are two and eleven. Learning and GP evaluation.

Prediction does not settle decision value. A three-seed optimizer comparison uses a total budget of 64, including the 32 paid research observations when provided. Mean final objectives are 1.286606 for the learned basis plus paid data, 1.281809 for the default basis with the same data, and 1.443123 for a cold default optimizer; all recommendations are feasible. The learned basis has zero wins, one loss and two ties against the same-data default. These controls distinguish the advantage of reusing observations from an independent advantage of the learned prior. Matched data-reuse comparison.

G.2 Decision confirmation, computation and incomplete revision

Table H9 uses the `independent_confirmation` cohort of the frozen goal-completion study (v0.53, compared with v0.52), rather than its 34 development cases. The three task profiles and new seeds lie within the same scripted quadratic family. Separate final evaluation measurements never return to the controller. Relative to the preceding Hegel controller, mean constraint MSE increases from 0.000329532 to 0.000469769 despite better objective decisions. Objective-research computation rises from 96,408 to 156,351, and recorded optimizer time from 21.07 to 24.70 seconds. The arms match simulator access, while their additional research work differs. Protocol; confirmation and development results.

The later confirmation compares the frozen constraint-knowledge extension (v0.57) with its predecessor (v0.56), retaining their nine exact decision ties. Its three separate constraint-change cases reduce mean constraint MSE from 0.000217372 to 0.000135406, while both retain final objective 0.5 and feasibility in all three cases. The prediction gain therefore has no demonstrated incremental terminal decision benefit in that contrast. Later confirmation.

G.3 Returning deployment discoveries to subsequent research

A development sequence compares batched learning before deployment, interleaved learning and deployment, and interleaving with deployment feedback removed. Three seeds each contain four three-dimensional tasks, with a budget of 64 per task and arm. Every condition returns feasible recommendations on 12/12 tasks and the same mean final objective, 0.468455. Interleaving reduces research computation on the nine successors from 3,913 to 3,364 (14.03%), with 180 research observations in every condition. Six successors call a concept learned during earlier deployment. Removing deployment feedback restores the batched research outcomes. This isolates useful return of deployment knowledge to later search, with unchanged terminal optimization in this sequence. Sequence protocol; recorded outcomes; paired interpretation.

G.4 Direct comparison with the historical four-property profile

We extract the SC-KG runtime from commit `aa1f61e0` and rerun native SC-KG, its full structural-prior configuration, and frozen Hegel v0.53 on the nine cases of Table H9. All use the same historical backend and optimizer settings. The full profile fits its original spectral, frequency and additive models from the common 32 paid observations through a record-input adapter. Its pilot gate and coefficient prior remain unchanged. Hegel uses its original research and revision controller. Each arm then buys 32 further simulator observations; the original evaluation records remain outside the controller.

This is a retrospective comparison on previously evaluated tasks. The initial observations were selected by the historical Hegel researcher and are now shared exactly across all arms. No new source archive is purchased, and the full-prior model is fitted within the current task. Its source fitting, initial gate and subsequent optimizer execution are timed separately. The comparison measures complete-system decision performance conditional on that shared prefix, while the original source-transfer and high-dimensional questions remain separate. Protocol; comparison runner.

Table G1. Historical structural-prior comparison on the same nine cases. Lower regret is better. Time totals cover the newly executed optimizer phase across nine cases, with one BLAS thread and execution order rotated across cases.

Method	Mean regret	Feasible	Optimizer seconds
Native SC-KG	0.100000	9/9	24.40
Four-property full profile	0.129478	9/9	624.82
Hegel v0.53	0.017944	9/9	26.56

Hegel improves all nine paired recommendations against both controls. Relative to native SC-KG, the full profile has five improvements, one tie and three regressions. Its profile means are 0.013053 for stable interactions, 0.106763 for center changes, and 0.268617 for joint center/interaction changes. The latter regressions outweigh the gains in its overall mean. Native and Hegel reproduce all nine historical recommendation points, true objectives and feasibility outcomes exactly.

The four-property models are fitted, but both heads select `coordinate` in all nine initial and final gate records. This is the learned coordinate model of the structural-prior

implementation, not the native coordinatewise quadratic basis. The result measures the full configuration's selection and subsequent optimization, rather than a forced spectral/additive model. Prior fitting and initial gate preparation add 8.45 seconds to its recorded optimizer time. Historical prefix acquisition and Hegel's initial program learning are reused and excluded from these newly incurred timings. The experiment adds 864 simulator calls, with no new initialization or evaluation measurements. Complete results; compact results.

Appendix H. Expanded framework, methods and complete controlled-study results

These sections retain the detailed formulations and complete studies underlying the three experiment packages in the main paper. Their H-prefixed numbers distinguish them from the shortened main text. Appendix F preserves the separate authoring cohorts and all failed attempts; Appendix G retains the optimization application.

H.1. A research state that can be revised

H.1.1 Hypotheses, evidence, and effective languages

We distinguish a **hypothesis**, which is a claim being investigated, from an **assumption**, which is a premise provisionally used in an inference. The same proposition can occupy either role: a fitted relation is initially a hypothesis, becomes a working premise in a composition, and returns to being a research question after failure. Its role can change without its status becoming certain.

A representation ϕ groups detailed situations by the information it retains. Writing $x \sim_\phi x'$ when $\phi(x) = \phi(x')$ makes the abstraction explicit. A predictor defined on these groups can succeed only to the extent that the discarded distinctions are irrelevant to its query and scope, or are represented through uncertainty. For a causal query, relevance also depends on the interventions considered. Thus abstraction is task-relative: a coarser description can be adequate for one prediction and inadequate for another. A constant forecast is already a predictive hypothesis; a useful learned law must additionally survive tests beyond the cases used to construct it.

An agent encounters tasks indexed by t. Within a task, it chooses experiments e and observes outcomes y. An experiment specifies controllable inputs, a trajectory length, and requested measurement channels. Acquiring and processing the resulting data incurs a resource vector

$$c(e) = (c_{\text{obs}}, c_{\text{meas}}, c_{\text{comp}}).$$

We describe its research state as

$$\mathcal{R}_t = (\Phi_t, \mathcal{L}_t, \mathcal{E}_t, \Pi_t, K_t, D_t, b_t),$$

where Φ_t contains available observation representations, $\mathcal{L}_t$ contains executable primitives and learned definitions, $\mathcal{E}_t$ contains available experiments, Π_t contains research procedures, K_t stores knowledge and its dependencies, D_t is acquired evidence, and b_t is the remaining budget. This notation describes the objects that can participate in research; different implementations may permit different subsets to change.

A predictive hypothesis contains a representation ϕ, a program p, an applicability description s, and any explicit supporting premises Γ. We write $h = (\phi, p, s, \Gamma)$ to expose these components. Given an observed history $x_{\leq n}$, its deterministic form predicts

$$\widehat{y}_n = p(\phi(x_{\leq n})).$$

The task objective and resource accounting are fixed by the experimental protocol. An applicability description can include an executable condition, an instrument binding, or recorded empirical coverage. Coverage describes where evidence has been obtained; extending that coverage requires additional investigation. A knowledge entry also records supporting observations, counterexamples, source dependencies, and the procedure used to construct its program.

The framework also admits distributional predictions and structural claims such as conditional independence or invariance. These claims have different criteria for support. A research-method hypothesis concerns whether a procedure resolves a question at an acceptable cost; a meta-prior expresses a revisable preference over candidate structures or research directions. Keeping these types distinct prevents a useful search preference from being mistaken for an established law of the environment.

Learning a new callable definition changes the agent's **effective research language**: expressions can be proposed, evaluated, and combined using an acquired abstraction. In a fixed underlying programming language, the definition may already be expressible by expanding its body. Its value can nevertheless be measured through improved access to solutions under finite search and acquisition budgets. This distinction lets us evaluate knowledge growth through actual use rather than through the number of newly introduced names.

H.1.2 Structural relations as testable assumptions

A structural relation specifies a pattern together with the roles and conditions under which it can be tested. Monotonicity involves an ordering and an inequality; symmetry involves a transformation and a prediction-preservation condition; conservation involves a collection of quantities and an allowed transition. These constraints differ in arity and in the observations needed to assess them.

For example, a conservation candidate for an exchange process can predict that the total quantity after a transition equals the previous total plus externally supplied input. A local event candidate may depend on which objects lie within a relational boundary. A response composed from these candidates can fail because the boundary changes while conservation continues to hold. Representing the two components explicitly makes this a question about local repair and retained dependencies.

The corresponding search bias is also revisable. A preference for separable models can guide an inexpensive first fit; persistent interaction residuals can motivate coupled candidates. In the implementation, these meta-priors are structured search and applicability records. They influence candidate generation and research reuse, and are evaluated by the consequences of using them.

H.1.3 Experiment-induced distinguishability

The same pair of hypotheses can agree under one set of observations and disagree under another. Let P_h^e denote the outcome distribution predicted by hypothesis h under experiment e. For a finite experiment set E, define

$$h \sim_E g \quad \Longleftrightarrow \quad P_h^e = P_g^e \quad \text{for every } e \in E.$$

With strictly positive weights w_e, a compatible pseudometric is

$$d_E(h, g) = \sum_{e \in E} w_e \, \mathrm{TV}(P_h^e, P_g^e).$$

It becomes a metric on the corresponding observational equivalence classes. Adding an experiment refines these classes whenever it distinguishes hypotheses that were previously equivalent. Appendix A gives the elementary argument. This perspective captures the useful part of the functional viewpoint: the tests available to an agent help determine which distinctions matter to its research.

The implementation below uses finite prediction-disagreement scores and task losses. The distributional formulation explains the role of experiments; it does not require a calibrated posterior over programs. Likewise, candidates that happen to agree on a small acquired sample remain empirically unresolved. Population-level observational equivalence is a stronger statement.

H.1.4 Revision and preservation

Let D_{res} contain observations that motivate revision and let $D_{\mathrm{old},s}$ contain earlier successes within an applicable scope. An empirical preservation objective asks for a revised hypothesis h' satisfying

$$\widehat{R}_{D_{\mathrm{res}}}(h') < \widehat{R}_{D_{\mathrm{res}}}(h), \qquad \widehat{R}_{D_{\mathrm{old},s}}(h') \le \widehat{R}_{D_{\mathrm{old},s}}(h) + \epsilon,$$

together with accurate predictions on additional observations. Preservation can also be realized by retaining h for its old conditions and learning h' for the changed conditions. The task sequences below explicitly return to both conditions to evaluate this possibility.

The equations define an evaluation objective. In the implemented loop, newly acquired trajectories confirm current candidates, while returned tasks test whether useful earlier knowledge remains available. Resource comparisons then determine whether this form of preservation reduces the total cost of subsequent research.

This formulation separates three decisions about an acquired program p: whether to accept p as the current predictor, whether to retain p as a callable computation, and whether to use p when constructing a replacement. A counterexample bears directly on the first decision. The other two depend on subsequent predictive and research value. In particular, a replacement can contain a call to p together with a learned correction even when p alone is inaccurate. Preserving the computation then preserves an accessible research resource, while confirmation determines whether the new explanatory commitment is supported.

H.1.5 Recursive research and conditional justification

The object of recursion is an unresolved obligation supporting a conclusion. Suppose a parent question asks whether a response law explains an observed discrepancy. A subordinate question may ask whether a sensor is calibrated, whether an interaction boundary changed, or whether the proposed experiment distinguishes two explanations. If answering that question requires an unknown coordinate binding, the binding becomes a further question. A result flows back to the parent as a usable computation, a supported condition, a counterexample, or an unresolved dependency. The parent's conclusion must respect those dependencies.

This gives a reusable research pattern: identify the blocked inference, formulate a subordinate claim with observable or deductive consequences, investigate it, and update the parent. The same pattern can be applied at another level when needed. Depth is justified by the problem's dependencies and limited by available evidence and resources. A parent can remain conditional when a child is unresolved. Reopening a parent after a counterexample creates a new research episode over accumulated evidence.

Least squares and Newton's method clarify the smallest ingredients of this pattern. Least squares chooses parameters to minimize $F(\theta) = \frac{1}{2}\sum_i (y_i - f_\theta(x_i))^2$. A Newton step solves a local quadratic model through $\nabla^2 F(\theta_k) d_k = -\nabla F(\theta_k)$ and updates $\theta_{k+1} = \theta_k + \alpha_k d_k$. In full-rank linear least squares, the quadratic model is exact and an undamped step reaches the minimizer. For more general objectives, curvature conditions and step selection matter. These are numerical operations within specified premises. A research-level question arises when the agent investigates whether the chosen response family, error description, or local approximation is adequate. Optimization success settles the fitted objective; empirical adequacy requires consequences that can fail on new evidence. Boyd and Vandenberghe, 2004.

Adding a condition is another legitimate research operation. A difficult question q may become tractable under an additional assumption a:

$$\Gamma \cup \{a\} \vdash q \quad \Longrightarrow \quad \Gamma \vdash (a \Rightarrow q).$$

The result is a conditional theorem in a deductive setting. For an observable applicability condition, its empirical counterpart restricts evaluation to $S_a = \{x : a(x)\}$. A structural or distributional assumption instead restricts the admissible model class to $\mathcal{M}_a = \{M : M \text{ satisfies } a\}$. In either case, support concerns the resulting conditional claim. The added condition should identify a meaningful, nonempty set of cases or models and yield consequences on newly acquired evidence. Whether a applies to the current situation is itself a possible subordinate question. If its validity cannot be established, the answer remains explicitly conditional. If later evidence violates a, the agent can investigate a broader model or a different condition.

For example, a global linear response may fail because two operating regimes have different mechanisms. A conditional model can ask whether linearity holds within a measurable regime, and another question can investigate how to identify that regime before predicting. Conversely, if a proposed measurement cannot distinguish the regimes, the agent must address the missing information or retain uncertainty. Narrowing the scope, enriching the

representation, and widening the model family are different responses to the same failed commitment.

H.1.6 Reinforcement learning, robustness and causal identification

These fields provide concrete instances of assumptions that a research agent can expose and reconsider. Their objectives and evidential requirements remain distinct; the shared framework concerns which commitments support an answer and what action can resolve a failed commitment.

Reinforcement learning: assumptions behind a decision state. In a stationary Markov decision model, $s_t = \phi(x_{\leq t})$ is taken to contain the history relevant to the next transition and reward. Value learning estimates the consequences of actions under a policy and environment; a model-free algorithm can do this without explicitly fitting a transition model. A research question arises when histories mapped to the same state systematically have different consequences. The agent can investigate memory, a new observation, or a different state abstraction, then test the resulting prediction or control performance. This separates learning within a decision model from studying whether its representation is adequate. The reward or task objective supplies the criterion for that study. Sutton and Barto, 2018.

Robustness: revising which environments are possible. A nominal decision uses one model M_0. A robust decision can instead optimize a lower envelope,

$$\max_{\pi} \inf_{M \in \mathcal{U}} J(\pi; M), \qquad \mathcal{U} = \{M_0\} \implies \max_{\pi} J(\pi; M_0).$$

Here $J(\pi; M)$ denotes expected return under policy π and environment model M. Enlarging $\mathcal{U}$ admits more possible environments and can make the decision more conservative. An uncertainty-set hypothesis concerns what belongs in $\mathcal{U}$, how its coverage is justified, and which nominal performance is sacrificed. Robust dynamic programming additionally depends on the structure of that set; rectangularity is a substantive condition for the decomposition studied by Iyengar. Within our framework, a failed nominal prediction can motivate investigation of variability or model uncertainty, while later evidence may justify a simpler description. Iyengar, 2005.

Causal inference: assumptions behind an intervention claim. Observational agreement can coexist with different predictions for $P(Y \mid \mathrm{do}(X = x))$. Identification asks whether the causal query is fixed by the available distribution and the assumed causal structure. A research agent can seek an intervention, investigate a supporting assumption, or report the remaining range of answers. Pearl's instrumental-variable example illustrates conditional tractability: a causal effect that is generally not identified nonparametrically becomes identified under appropriate linear structural and instrumental-variable assumptions, with a nonzero instrument–treatment covariance. The extra assumptions create a more restricted identification problem. They also become dependencies of the answer. Pearl, 2009.

The common operations are therefore specific: expose a premise, obtain discriminating evidence, revise its scope or representation, propagate the change to dependent conclusions, and assess the consequences. Research can also target the experiment-selection procedure itself when it repeatedly fails to expose relevant distinctions. This organization makes it

possible to study assumptions about the world and assumptions about how to study the world within one persistent state.

H.1.7 Generalization, structural correspondence and the history of hypotheses

A broader account can explain an earlier law as a special case, a scoped approximation, or a component inside a larger mechanism. If ρ binds the old variables and conditions into a new model h', preservation can be expressed as

$$\sup_{x \in S} |h'(\rho(x)) - h(x)| \leq \epsilon,$$

for a specified old scope S, with a corresponding limiting statement when appropriate. The bound states what a proposed generalization would preserve; demonstrating it requires a proof or evidence suited to the claim. The broader model also needs consequences outside that scope. These requirements give generalization content beyond storing several unrelated answers. When unification is unsupported, separately scoped models can remain available.

For an illustrative extension of the composition example, a family $G(x; \lambda) = A(x) + 0.7B_\lambda(x)$ contains C at an old boundary $\lambda = r$ and a revised response at $\lambda = r'$. Learning the family, identifying the applicable λ, and showing that it predicts new cases are separate research achievements. The implemented additive repair $D = C + 0.7(B_{r'} - B_r)$ preserves useful computation even before a compact, scope-selecting family has been learned.

Reuse across tasks similarly requires a proposed structural correspondence. A relation can involve an ordering, a transformation, several conserved quantities, or a conditional dependency. The agent must bind the current quantities to the relation's roles and test the resulting consequences. For predictive programs, a useful correspondence can be written schematically as $p_{\text{new}}(x) \approx T[p_{\text{old}}(\rho(x))]$, where ρ maps input roles and T accounts for a justified output transformation. The mapping and its scope are hypotheses too. Successful reuse executes the acquired relation with the current binding; failed reuse can prompt revision of the binding, the relation, or the experiment used to assess it.

This gives a precise engineering interpretation of the Hegelian motivation. Hegel's account of the history of philosophy treats earlier systems as moments in the development of later thought. We take from it a research principle: the history of successful abstractions and their limitations can inform the construction of a more adequate account. A retained hypothesis carries executable content, empirical scope, failure evidence and dependencies. Earlier conclusions can be recovered where warranted, and their failures can explain why further structure was needed. Hegel, 1892, Introduction A.3.

Thus the unit of accumulated knowledge is a scoped abstraction together with the research dependencies supporting its use. Its history changes both the answers available to the agent and the investigations that become accessible next.

H.2. The Hegel Machine research loop

We implement the framework in controlled numerical laboratories. The object-composition and joint-growth controllers share an executable language and predictive knowledge library. The former tests preservation and local repair of learned definitions; the latter combines

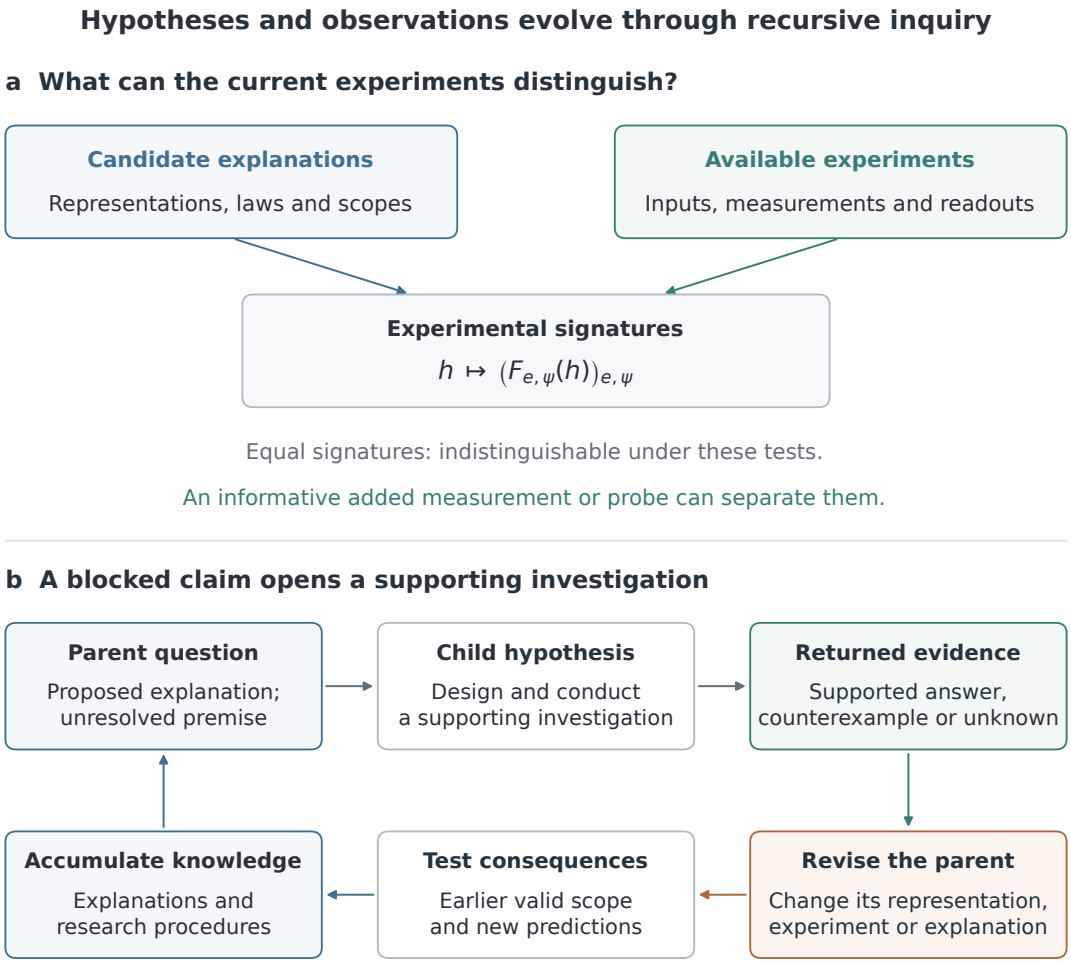


Figure H1. Hypothesis spaces and recursive supporting research. Candidate explanations and observations jointly determine distinguishability. An unresolved supporting premise can open a child investigation; returned evidence changes the parent representation, experiment or explanation. New predictions and earlier valid scope are then tested. Accumulated explanations and research procedures can both be revised. The diagram states the general framework; the experiments realize bounded subsets.

expression, representation, scope, procedure and uncertainty revision. Appendices H.2.1–H.2.6 describe these operations. The online controller in Appendix H.2.7 and language-model agents in Appendix H.2.8 provide separate implementations of supporting inquiry and code investigation through a common public laboratory. Appendix H.2.9 makes executable research programs the objects of learning and preservation. Appendix H.2.10 connects learned hypotheses to the prior representation of a downstream optimizer.

Each study fixes its controller and protocol. A retained definition can be called unchanged, while a retained procedure can construct a newly fitted response. The experiments record these forms of reuse separately.

H.2.1 Executable knowledge and paid evidence

The current implementation uses an abstract syntax tree language containing numerical operations, predicates, object and relation aggregation, temporal operators, and calls to learned definitions. Temporal operators include lagged values, windows, event differences, and accumulating state. Programs can contain nested calls to earlier definitions, making their source dependencies executable.

The environment exposes control variables and a finite measurement interface. Acquired observations are stored with their experimental context. A supported projection can reuse a measurement in another view without purchasing the same observation again. Candidate fitting, validation, and further research operate on this accumulated evidence. Observation acquisition, measurement, and charged program computation are recorded separately.

Confirmed knowledge stores a response program and, where available, its parameterized abstraction and research procedure. A retained response can be called directly. A retained procedure can instead reconstruct a response from the current data, refitting its parameters. These operations serve different purposes and are recorded separately in the evaluation.

H.2.2 Competing explanations and dependent research

Research begins with inexpensive response fits in the current coordinates and relevant retained knowledge. Available directions include calibration, alternative measured views, conditional responses, object relations, residual composition, and temporal structure. The candidate generators use finite families, including affine and sparse responses, separable and coupled terms, low-dimensional projections, and bounded history.

A candidate can depend on a subproblem whose answer supplies a coordinate, a component, or a residual explanation. A research graph records these dependencies, and executable answers can be composed into the parent response. In the nested relation-revision path, the generator first fits candidates and then compiles its completed operations into an answered dependency graph. The controller subsequently obtains distinguishing observations and refits within a bounded loop. These operations implement finite dependent revision; Appendix H.3.9 traces their execution. General composition search uses a small beam and a limited number of expansion rounds and fitted terms.

Fitting a more complex model is only one possible response to failure. The agent can purchase a different view, reconsider a condition, or revisit a previously unsuccessful direction after new evidence becomes available. These choices compete within the resource

budget. The session controller can also revisit an earlier task when newly learned knowledge creates a potentially useful research opportunity.

H.2.3 Choosing and revising experiments

For candidates that predict directly from controllable coordinates, the agent constructs a finite pool of probes and compares their predicted responses. One implemented selection rule is

$$e^* = \arg\max_{e \in \mathcal{E}_t^{\text{pool}}} \frac{1}{|e|} \sum_{x \in e} \left[\max_{h \in \mathcal{H}_t} h(x) - \min_{h \in \mathcal{H}_t} h(x) \right].$$

The pool includes random sequences and structured variations such as alternating endpoints and piecewise-constant inputs. Other research directions use probes for instrument calibration, relational boundaries, or temporal conditions. An experiment's construction therefore depends on the current candidates and the problem being investigated. Required measurements are acquired through the environment interface.

The interpretation of a failed experiment also matters. If an earlier structure fit was based on too little evidence, a new counterexample can justify fitting that direction again using the expanded data. Treating a direction as permanently exhausted after its first attempt would confuse the outcome of one fit with the limits of the hypothesis family. Conversely, revisiting a direction without new evidence or a changed candidate can consume resources without adding a distinction.

H.2.4 Fresh confirmation and scoped revision

A selected program is frozen before its confirmation trajectory is acquired. Successful confirmation permits it to enter the reusable library. An unsuccessful trajectory becomes evidence for subsequent research, after which a revised candidate requires new confirmation. The distinction between candidate construction and confirmation is maintained even when the research process resumes an earlier task. These empirical thresholds do not provide calibrated sequential error control across repeated revisions; false confirmations are retained in the reported outcomes.

Retained programs are evaluated on current paid observations. Their failures are recorded even when another candidate wins the comparison. In the ordinary transfer path, a losing retained call adds a counterexample to its empirical prior and changes that record to conditional. Its executable knowledge remains available for later reassessment. Current observations can therefore reject an old response on a changed task and support it when the earlier conditions return. Appendix H.3.4 separately tests the consequences of retaining the response and of adding this conditional feedback.

Composition makes preservation more precise. If a combined response fails while one of its source definitions remains useful, the revision can retain that definition and change the residual contribution. The experiments inspect the resulting program, test its predictions, and intervene on source availability. These observations address different aspects of the same revision: structural retention, predictive adequacy, and subsequent value.

H.2.5 Learning research procedures

A research procedure records selected transformations, dependencies, and fitting operations. Its reuse transfers the structure of an investigation to another task, where coefficients or conditions can be fitted again. A successful direct call and a successful procedure replay therefore need not produce the same execution trace, even when they ultimately produce the same predictive program.

Research procedures and their applicability are stored with supporting evidence and counterexamples. The session controller uses available knowledge and the remaining budgets to price further visits. Its portfolio search compares affordable single visits and selected pairs, executes an available action, and updates the plan from the measured result. This provides a concrete mechanism for allocating research across dependent tasks.

When an unfinished task resumes, the controller also considers retained concepts that were unavailable to its earlier visit. Screening uses that task's purchased observations and charges the adaptation and comparison work. A squared-error candidate that meets the fit tolerance on these observations and improves the incumbent can be frozen for one new confirmation trajectory. This permits a revisit without purchasing a second selection trajectory; failure ends the quoted package and retains the new evidence. Appendix E.3 tests this continuation under an unchanged resource ceiling.

The overall loop is summarized below. Each operation refers to the bounded candidate generators, evidence store, and research actions described above.

```
Input: task interface, reusable knowledge K, resource budget b
Acquire initial observations and compare current fits with applicable knowledge.
While an affordable research action remains:
    Construct or adapt candidates using paid evidence and dependent subproblems.
    Compare candidates, required measurements, and remaining research cost.
    Select a candidate and an experiment appropriate to the unresolved
     ↪ distinction.
    Freeze the candidate; acquire a fresh confirmation trajectory.
    If confirmation succeeds:
        Store the executable result, its dependencies, and its research procedure.
        Make the new knowledge available to later tasks and affordable revisits.
        Return the supported response for this visit.
    Otherwise:
        Retain the counterexample and revise the record of the failed application.
        Reconsider candidates and research directions using the enlarged evidence.
Return the best supported response and the unresolved research state.
```

H.2.6 Joint growth of the research state

A second finite instantiation makes the joint revision loop explicit through a single persistent researcher. It shares the executable language and knowledge library with the preceding implementation, while adding an observation-driven expression search, evidence-dependent procedure compilation, coordinate transfer, and a repeated-measurement uncertainty model. This instantiation is evaluated separately from the object-composition experiments below.

Effective vocabulary and applicability. Candidate search first tests previously acquired definitions under empirically compared role bindings. Failed predictions generate

residuals from which products, absolute-value relations and thresholded linear terms are proposed. Both persistent and task-local search can construct these expression families. A retained definition provides an additional starting point; its absence does not remove conditional models from the task-local hypothesis language. An old response p can be embedded in a new response $q(x) = \mathbf{1}_{c(x)}p(x) + \mathbf{1}_{\neg c(x)}r(x)$, where the condition and replacement are learned from acquired observations. The old model is then exactly preserved on the learned branch. Nearby observations bracket a proposed threshold, and additional probes target that unresolved interval.

Roles and representations. A callable definition separates its body from its observed arguments. Candidate argument permutations are assessed on current responses. Public coordinate ranges additionally propose affine changes of units and origin; the measurements determine whether a proposed correspondence predicts the new task. A learned procedure can directly compare paid instrument views on current pilot data. If no applicable procedure resolves the mismatch, the fallback investigation repeats fixed controls before requesting that comparison. Selecting a view changes the inputs available to subsequent explanation search. The public ranges and instrument catalogue constrain what it can seek.

Generated research procedures. Acquisitions produce evidence handles, fits consume the observations actually used, and confirmations consume a frozen candidate. The ancestor graph of a successful confirmation becomes a reusable procedure. Failed confirmations remain usable as counterexample acquisitions; unsuccessful or unused search branches are omitted from the stored procedure. A successful instrument comparison becomes a subproblem generator: on a later task it constructs a fitting question for each currently available instrument, then selects a view by measured validation loss. The recorded winning instrument index is not assumed to retain its meaning. Subsequent failure can restrict the procedure's applicability and produce a child with different executed steps. Primitive laboratory commands remain fixed, while their dependencies, bindings and composition are learned from completed investigations.

Uncertainty and reversible simplification. Repeated residuals at fixed controls estimate within-setting variation. A misspecified deterministic mean that produces the same residual on every repeat does not by itself generate a positive noise estimate. Constant and coordinate-dependent variance proposals are compared by leaving complete repeated settings out, rather than treating replicates at one setting as independent validation locations. A fresh response batch then assesses the mean prediction relative to the estimated noise scale. Simpler predictive expressions compete under the same task objective; their adoption preserves earlier complex definitions for later reassessment. Evaluation separately measures mean error and variance calibration, so a larger declared variance cannot make a poor mean predictor pass the external task criterion.

These mechanisms connect changes to effective vocabulary, explanatory scope, representation, procedure and uncertainty in one loop. Their operational scope and separate evidence contrasts are reported in Appendix H.3.7 and Appendix D.

H.2.7 Online supporting investigations

A third, finite instantiation makes supporting questions operational before their evidence is collected. A calibrated-response laboratory exposes two anonymous controls and three

experiment types: observe the response, measure an unknown physical coordinate, and calibrate the instrument against a known reference. Its response is $y = f(ou_j)$ and its instrument reads $r = a(ou_j) + b$. The public language contains linear, quadratic, absolute-value and threshold responses, two control bindings, two orientations, two calibration signs and three offsets: 96 joint hypotheses. These primitives and their allowed parameters are shared by all researchers.

The recursive researcher opens a response question. Unresolved coordinate bindings produce a supporting question; unresolved instrument parameters can produce its calibration subquestion. Observations filter the shared hypothesis set. A returned answer changes the parent's remaining alternatives and therefore its choice of experiment. When a retained research step distinguishes fewer current alternatives than another available step, the researcher rewrites that step before acquiring its observation. The revised executable procedure is retained after confirmation. Questions already answered by current evidence need not be expanded.

Two controls isolate this organization. A flat active researcher selects from the complete experiment menu using information gain about the response law, without a support tree. A frozen-method researcher uses the same recursive organization and source knowledge, attempts the retained steps unchanged, and then permits fresh adaptive fallback. Freezing a procedure therefore removes step rewriting without withholding the decisive measurements or the candidate language. Responses are represented in the existing Program language; a retained Definition can be called with a newly established coordinate binding. This is executable reuse after reassessment, not an exemption from research.

H.2.8 Common-model code investigation

The completed comparison uses **gpt-6-luna with medium reasoning through the Codex ChatGPT interface** in all three knowledge conditions. Each agent can buy any public laboratory experiment, write and run Python, evaluate on purchased observations, and submit a predictor. The host supplies observations and executes code; this experiment supplies no hypothesis fitter or hidden task parameters. A submission is frozen before three new confirmation observations are collected. Their targets remain outside model-readable research files during scoring. Failed confirmation then makes those observations available as counterexamples. Earlier GPT, Qwen and Gemini service cohorts are separate records; the completed comparison and its failures are reported in Appendix H.3.11 and Appendix F.1.

The ordinary Code/ReAct condition starts each task without persistent knowledge. The textual-memory condition carries the source agent's own lesson. The Hegel condition carries the source agent's executable predictors, experimental programs and editable hypothesis record. A stored experimental program must actually have selected source experiments before it can be retained; a question is recorded before those observations are acquired. Existing evidence may resolve a task without additional supporting questions. New tasks can execute or revise the retained files through the same Python interface available to all three arms. Executed entry points are distinguished from agent-authored dependency references.

This comparison tests representation and persistence of research knowledge under a common proposal model. The LLM authors its programs and decisions; it is not a language front end to the finite researcher in Appendix H.2.7. Both controllers use the same public laboratory and independent prediction criterion, allowing their evidence to address complementary questions.

H.2.9 Research programs as hypotheses

The temporal method studies make the investigation procedure itself executable. A method reads the observations and feedback currently available, chooses a laboratory or analysis action, and eventually proposes a predictor. A language model can edit this program and practice it on public source problems. Each practice runs the actual source, charges its acquisitions and computation, and ends in fresh predictive confirmation or a recorded failure. Transfer freezes the final program and executes it on other archived sources without further language-model calls.

Evidence belongs to the program that generated it. Publication requires one source to satisfy the specified practice cases and preservation requirements; successful cases from incompatible edits do not form a successful method. Previously practiced sources remain available with their own results. A later failure withdraws that source's support for the affected case until new evidence repairs it. The current implementation can also compare a submitted revision with a supported older source, completing that source's missing practices before replacement. This submission-time comparison is a host retention policy. The experiments distinguish it from an agent selecting an old program itself.

This creates the same research problem at a second level: a predictive hypothesis is tested against observations, and a procedural hypothesis is tested by the investigations it produces. Both can be revised, conditionally retained and reused. The numerical solvers and laboratory primitives remain shared components; the learned object is the program deciding how to use their results.

H.2.10 Turning learned hypotheses into an optimizer's prior

The SC-KG integration asks whether research knowledge can change a downstream decision model. Its motivation includes four properties of the KG_op structural prior: low-frequency concentration, orthogonality, sparsity and additivity. These describe one structured prior, rather than four independent predictors. The implemented bridge lets a learned executable hypothesis specify a GP feature map. The native comparison uses a coordinate-wise quadratic basis,

$$\phi_0(z) = (1, z_1, \dots, z_d, z_1^2, \dots, z_d^2), \qquad \phi_p(z) = (1, p(z)).$$

A finite researcher proposes affine, sparse, additive, coupled-quadratic, projection and history-dependent programs. Measurements select their parameters and applicability. Ordinary refitting is tried first; unresolved counterevidence can create a residual subproblem $r = y - p_{\text{retained}}(z)$, whose answer forms $p_{\text{new}} = p_{\text{retained}} + r_\theta$. Candidate disagreement and probes in a projection's null space can direct additional observations. Thus an inadequate explanation supplies both a new question and an executable component of its replacement.

`compile_sc_kg_program` installs the learned constraint program as ϕ_p. Every already purchased simulator vector enters the original `SingleOLHKGAlgorithm`; its posterior fitting, candidate scoring and subsequent simulation remain the optimizer's operations. The total simulator allowance includes these initial observations. Adoption compares predictions made before subsequent measurements arrive. The objective-research extension can similarly replace the objective GP basis with $(1, q(z))$ after future paired validation, and reopen research when deployed predictions fail. The reported studies use `proposer:search` and no LLM calls. Their learned objects are numerical programs within the supplied grammar and their evidence-dependent deployment, complementing the language-model-authored procedures studied above.

For a direct structural-prior comparison, we also reuse the historical KG_op `full` profile: low-frequency regularization and frequency adaptation, symmetric orthogonalization, coefficient shrinkage and additive adaptation. Its original pilot gate selects among the permitted representations; enabling all four properties does not force every property into the final model. The constraint head uses this gate, while the objective retains the historical coordinate representation. We fit the prior on the same paid task prefix supplied to Hegel, then retain its native posterior updates and representation adaptation. This compares the complete Hegel research controller with the four-property model under shared observations, without changing the optimizer's acquisition settings.

H.3. Controlled studies

H.3.1 Evaluation questions and protocol

The experiments address five questions:

1. **What should survive a counterexample?** Appendices H.3.3–H.3.6 intervene on the availability of learned computations and vary the component that changes.

2. **Can the object of revision extend beyond the current answer?** Appendices H.3.7, H.3.9, H.3.10 and H.3.12 test representations, scope, supporting questions and experiments, including a recorded hypothesis life cycle.

3. **Can a research procedure itself improve and remain useful?** Appendices H.3.8 and H.3.13–H.3.15 distinguish knowledge availability, method revision, subsequent adoption, preservation and autonomous recall.

4. **Do these mechanisms improve an autonomous Agent?** Appendix H.3.11 provides the complete common-model comparison; subsequent temporal studies locate the remaining adoption and research-control failures.

5. **Does learned research knowledge improve decisions downstream?** Appendices H.3.16 and G distinguish prediction, data reuse and optimization gains when acquired programs become GP features.

Three 30-instance object-process studies, a 30-instance joint-growth study and matched online-inquiry studies provide the controlled mechanism evidence. Appendices C–E retain the smaller mechanism and integration cohorts. Appendix F contains the complete

language-model development record, including interrupted attempts, interface changes, negative comparisons and resource ledgers. Appendix G adds the SC-KG decision and deployment-feedback studies. Each study has its own frozen implementation and protocol. We compare interventions within a study, retain registered failures, and distinguish parameter instances, repeated authors and repeated execution of the same target contract.

In the composition studies, each complete learning sequence is followed by independent evaluation on four trajectories of 64 observations per task. A task passes when its mean squared error is below 10^{-6}. Research-time confirmation uses a threshold of 10^{-7}. The reported resource totals include source learning, candidate search, unsuccessful research, and confirmation; terminal evaluation is performed separately. Charged computation is the implementation's operation accounting, whereas observations and measurements represent separate acquisition costs. The joint-growth study has its own deterministic and stochastic criteria, specified in Appendix D.

Source-deletion arms begin from the same actually learned prefix as the retention arm. The selected source and its dependent knowledge are removed, and both arms continue with the same researcher settings and budget limits. Deleted knowledge may be relearned. The contrast measures the total downstream effect of making that source available at the intervention point. All arms complete learning before terminal evaluation is requested.

A procedural criterion supplements prediction accuracy. It asks whether sources were learned from an empty library, subsequently used, contradicted by acquired evidence, revised, and available when the original conditions return. The composition study additionally requires actual calls to both components, preservation of the unchanged source definition, and later calls to the original and revised compositions. These criteria make particular knowledge-lifecycle events observable.

H.3.2 Task family and supporting mechanisms

The main experiments use a controlled object-exchange family with selectable coarse and fine measurements. The total-quantity response A is conserved through internal exchange, whereas the event response B_r depends on a relation radius r. Their combined response is $C = A + 0.7B_r$. The six-task sequence learns A, learns B_r, learns C, changes the radius to 0.45, returns to C, and returns to the changed response. Each initial task allows 384 observations, 4,608 measurement units, 360,000 charged computation units and six research rounds. The agent learns the components from observations and can later call their acquired definitions.

Three earlier mechanism studies provide supporting evidence (Appendix C). The single-source study shows a small complete-sequence saving from retention, but its inexpensive fixed-library learner solves all 48 tasks with much less computation than full research. The two-source study identifies positive, separately measurable contributions from two acquired sources. The object-composition study preserves the exact unchanged component inside a repair and reduces complete-sequence computation by 41.11% relative to deletion of that component. Its procedural criterion passes in 11/12 instances, while both arms solve all 72 terminal tasks: the remaining instance recovers the right program by replaying a procedure rather than directly calling the acquired revision. These findings motivate the prospective

comparisons below and establish that prediction accuracy, actual reuse and total research cost need separate assessment.

H.3.3 Complete research algorithms on the composition sequence

We compare four algorithms on the six-task sequence in Appendix H.3.2. The full researcher retains knowledge across tasks. A task-local researcher runs the same hypothesis search, instrument learning, confirmation, and within-task memory, but starts an independent session with an empty library for every task. The fixed-library learner fits a predeclared quadratic and two-step-history basis. The passive researcher learns representations and shared knowledge from four precommitted random trajectories of 24 observations per task, without adaptive acquisition or revisits. Both fixed controls use the finest legal measurement view. All arms access the same public laboratory interface.

Initial task budgets match Appendix H.3.2, giving sequence ceilings of 2,304 observations, 27,648 measurement units, and 2,160,000 charged computation units. The full researcher can allocate unused sequence resources to revisits. Each task-local session keeps its own budget, so this contrast evaluates the combined benefit of cross-task knowledge and resource allocation. Actual expenditure is reported separately from these common ceilings.

After repairing the passive researcher's selection of mutually exclusive measurement views on a development instance, we froze the implementation and registered 30 new apparatus seeds, with 15 assigned to each existing generator profile. Both profile groups were run prospectively under that freeze; their labels do not denote training and test partitions in this experiment. Every arm finishes learning before any arm receives terminal scores. Evaluation uses four common new control trajectories per task and each program's required public measurements. Uncertainty is computed from 10,000 paired bootstrap resamples of apparatus instances, stratified by generator profile. The unit of resampling is a complete six-task sequence.

Table H1. Four-arm composition comparison. All 30 registered instances completed. Each arm is evaluated on 180 tasks; resource totals include complete research sequences. CPU time is measured process time in seconds.

Arm	Tasks passing	Observations	Measurements	Charged computation	CPU seconds
Full	180/180	17,489	75,192	9,078,989	1,939.14
Dense fixed basis	0/180	17,520	210,240	11,074,896	406.06
Passive experiments	87/180	17,280	207,360	7,025,428	1,457.42
Task-local memory	179/180	42,677	298,572	38,234,202	2,557.32

Relative to task-local research, the full system reduces cumulative observations by 59.02%, measurements by 74.82%, and charged computation by 76.25%. In all 29 instances where both solve every task, full uses fewer resources in all three dimensions. The mean paired differences per six-task instance are -839.6 observations (95% interval $[-856.7, -813.1]$), $-7{,}446$ measurement units ($[-7{,}589.6, -7{,}226.4]$), and $-971{,}840$ computation units ($[-1{,}013{,}700, -918{,}906]$). Accuracy differs by one task: the mean solved-count

difference is 0.033 per instance, with interval $[0, 0.100]$. The main observed advantage is resource use at nearly equal task accuracy.

The task-local failure occurs on the first changed composition in apparatus 538840. Its confirmation error is 6.11×10^{-5}, and terminal MSE is 8.60×10^{-5}. It spends 359,841 of its 360,000 computation units while leaving observation and measurement resources unused. On the same task, full succeeds using previously learned components and 101,698 computation units. Thus the independent researcher correctly rejects its unsuccessful candidate but does not complete the task within its research allowance.

The full system's measured training CPU time is 24.17% lower in aggregate, but its paired mean difference of -20.61 seconds has interval $[-33.84, 2.22]$. Apparatus 548184 accounts for the only full-system revisit and requires 361.77 CPU seconds, compared with 73.22 for task-local research. The strong saving in charged computation therefore coexists with an unresolved runtime advantage and a substantial tail cost.

Passive research solves 87 tasks using less computation than full, while purchasing more instrument measurements. It confirms 155 candidates internally, of which 70 fail terminal evaluation on independent trajectories. This directly exposes limited confirmation coverage under its fixed four-trajectory design. The dense fixed-basis learner fails all tasks, including the conserved-total task, whose response lies in its linear subspace. In this object interface it fits 185 coefficients to 48 training observations and stops after failed confirmation. Its failure therefore reflects the implemented fitting and selection procedure as well as the limited basis available for event responses.

H.3.4 Retaining a response after an observed failure

The previous comparisons establish effects of knowledge availability. We next ask whether a response that has just failed remains useful for subsequent research, and whether recording that failure as conditional-prior feedback independently changes the outcome. These are two distinct interventions.

Each apparatus first shares the same naturally learned A, B, C prefix. We identify C by equality between the newly learned concept's expanded program and the confirmed response, using the measured program if instrument compilation subsequently changed its coordinates. We then freeze this response and purchase one predetermined random trajectory of 24 observations on the changed task. The measurement view is the least expensive public view that supplies the program's variables. This challenge uses acquired responses and the research-time error threshold; it does not use terminal evaluation.

After a measured failure, three continuations begin before any replacement D is learned. **Conditional retention** retains C and records the failure using the ordinary conditional meta-prior update. **Unmarked retention** retains the same knowledge but omits this challenge's feedback record. **Global discard** removes C and its dependent knowledge, preserving its ancestors A and B; the removed response can be learned again from new evidence. Normal research and feedback continue in every arm. A retained response is eligible for later paid reassessment; the conditional record does not supply an executable general validity predicate.

The suffix contains the first changed task, a return to the original condition, and a return to the changed condition. Each suffix task starts a fresh one-task session carrying only its

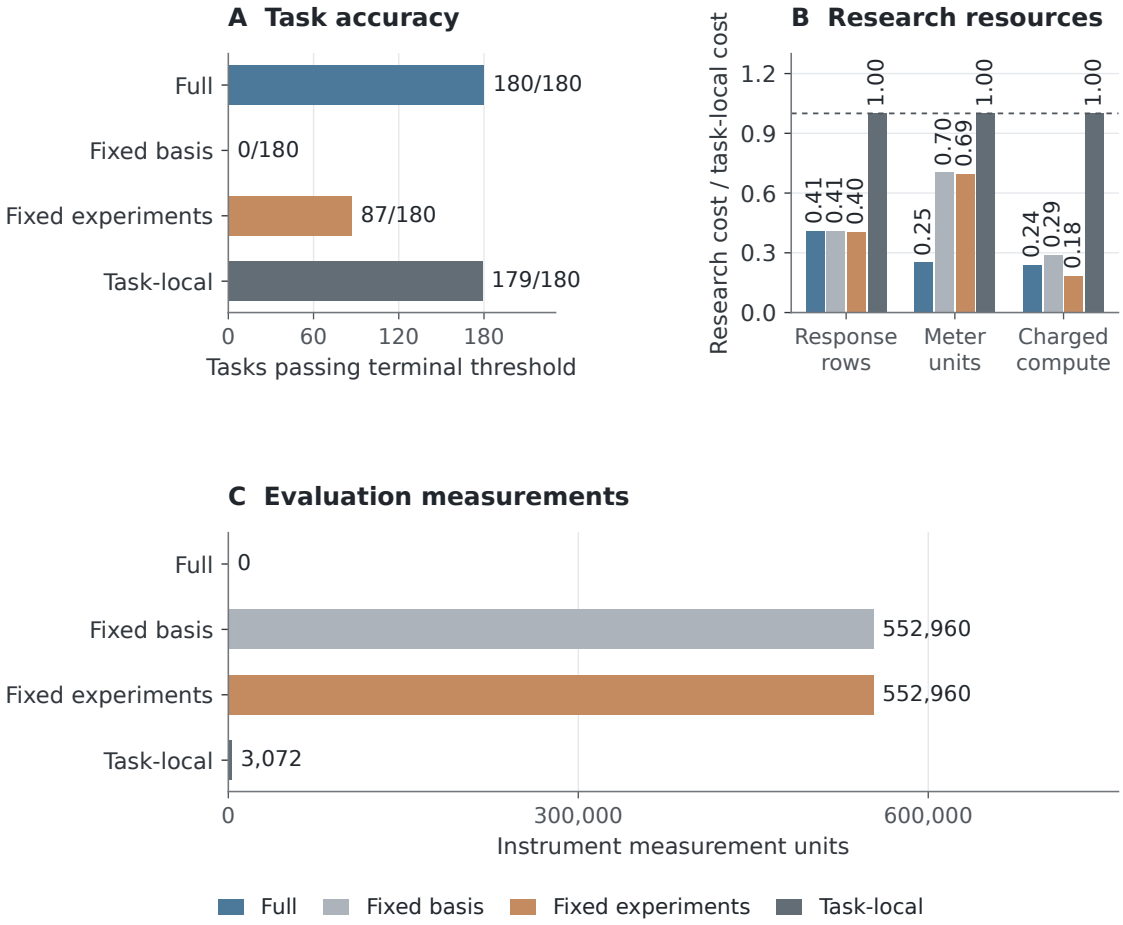


Figure H2. Accuracy and distinct resource dimensions across 30 apparatus instances. Panel A counts tasks with terminal MSE below 10^{-6}. Panel B normalizes each cumulative research resource separately by the task-local total. Panel C reports instrument measurements used during terminal prediction: full uses 0, task-local uses 3,072, and both fixed controls use 552,960. Every arm additionally incurs 46,080 evaluation observations and 46,080 charged evaluation computation units. Measured evaluation CPU totals are 66.37, 54.15, 11.50, and 18.17 seconds for full, dense fixed basis, passive experiments, and task-local memory, respectively. Research and evaluation costs are recorded separately. Vector PDF.

arm's persistent library. This removes cross-task session caches and resource borrowing from all three arms. The common challenge is charged to every arm and deducted from the first changed task's allowance. Its outcomes support the feedback intervention; they are not supplied as an additional free fitting sample. Prefix, challenge, research, confirmation and terminal evaluation costs are kept distinct.

We specified 30 new apparatus instances before execution, split evenly between the two generator profiles. All cases remain in the analysis, including any that do not learn an exact confirmed C or whose challenge does not refute it. The primary contrast is performance and research cost on the original-condition return. First-revision, changed-return and complete-sequence outcomes show where the effects arise. Paired uncertainty again resamples complete apparatus instances within profile, using 10,000 bootstrap samples.

Every registered case completed, and all 30 challenges refuted the learned response, activating the intervention. Both retention arms solve all 180 tasks and satisfy fresh confirmation on every task. Global discard solves 159 tasks and confirms 160 internally. Table H2 includes the common acquisition prefix once in each counterfactual arm, as well as the challenge and subsequent research.

Table H2. Retention after measured failure. Each arm comprises 30 six-task sequences. The first three tasks share a learned prefix; the final three use separate continuations.

Arm	Tasks passing	Observations	Measurements	Charged computation	CPU seconds
Conditional retention	180/180	17,987	73,680	8,553,189	2,678.81
Unmarked retention	180/180	17,987	73,680	8,553,189	2,581.20
Global discard	159/180	22,484	120,864	11,651,123	2,697.45

The primary original-return contrast has equal accuracy: all three arms solve all 30 return tasks. Both retention arms use 24 observations per return, compared with 40 after deletion. Aggregate charged computation on these returns is 30,930 versus 138,476. The paired mean difference is -16 observations, with interval $[-16, -16]$, and $-3{,}584.9$ computation units, with interval $[-3{,}868.0, -3{,}273.9]$. All 30 retained responses are selected through direct calibrated calls; deletion instead requires a research program to recover the response.

The quality difference arises principally during the first revision. Retention solves all 30 changed tasks, while deletion solves 10. Of the 20 deletion failures, 16 remain internally unresolved and four pass internal fresh confirmation but fail terminal evaluation. Three other deletion cases pass terminal evaluation while remaining unresolved, demonstrating that accuracy and research completion measure different properties. Each failed first-revision case retains at least 96 observation units, 2,064 measurement units, and 146,261 computation units. Thus these failures occur before exhaustion of the three resource accounts. The original compact records did not identify the stop cause. A subsequent frozen-runtime replay, reported below, identifies it without altering these outcomes. On the final changed-condition return, retention solves 30 tasks and deletion solves 29; the remaining deletion failure is unresolved and also retains substantial resources.

Across complete sequences, retention uses 20.00% fewer observations, 39.04% fewer measurement units, and 26.59% less charged computation than deletion. The paired mean computation difference per sequence is −103,264.5, with interval [−132,874.2, −78,700.7]. The first-revision computation difference alone has interval [−58,778.5, 5,844.6], so its computation saving is less certain than the whole-sequence effect. Measured training CPU is nearly equal: the paired conditional-retention-minus-deletion difference is −0.62 seconds, with interval [−12.93, 10.79]. Each arm additionally uses 46,080 observations and 46,080 computation units for terminal evaluation; retention uses no terminal instrument measurements, compared with 70,656 after deletion.

Conditional and unmarked retention produce exactly equal terminal errors and charged costs in every instance and successor phase. The recorded intervention is active: the former starts with a conditional prior and one counterexample for C, while the latter retains its supported prior without that counterexample. Both arms actually call C inside the selected response at all three successor stages in every instance. The measured benefit therefore comes from preserving executable knowledge and its downstream uses. This additional feedback record has no independent measured benefit under the tested protocol; normal later feedback remains active in both arms.

Frozen-runtime stopping diagnosis. We subsequently replay all twenty failed first revisions using the original study runtime, shared-prefix acquisition and discard operation. Every replay exactly reproduces the recorded status, selected candidate type, resource expenditure, confirmation loss and terminal MSE. The deletion actually removes the acquired C, its 17 dependent abstract/definition entries and its meta-prior. The ancestor programs remain. Generic methods, experimental-method records, action experience and other research experience are unchanged in these cases. This intervention is therefore broader than deleting a single call label, but does not empty the generic researcher.

Fourteen of the sixteen unresolved runs stop after their first confirmation and two after their second, under a six-round ceiling. Between one and three competing relation candidates survive the 24-row confirmation, setting `relation_ambiguous` and ending the visit. Fifteen final probe plans report `no_predicted_distinction`. Apparatus 701344 instead reaches the local 6,000-unit generation cap; its global computation account still has 312,572 units. None of the twenty sessions schedules a revisit. These are actual stopping branches, not inferred budget exhaustion.

The four false confirmations pass the short confirmation and finish normally; their independently evaluated responses are wrong. Across all sixteen unresolved cases, another random 24-row acquisition plus comparison of the current candidates is affordable under the remaining accounts. This is only a cost quotation: no additional recovery action is executed, so the audit does not show that more random data would solve the cases. The separate failure on the last changed-condition return is outside this first-revision replay. Original cohort results and uncertainty estimates are unchanged.

Replay summary and diagnostic runner retain the twenty per-case matches and stopping events. The algorithmic limitation is that failure to construct a decisive local probe can end a visit with remaining global resources; this study establishes the consequence, not a repaired controller.

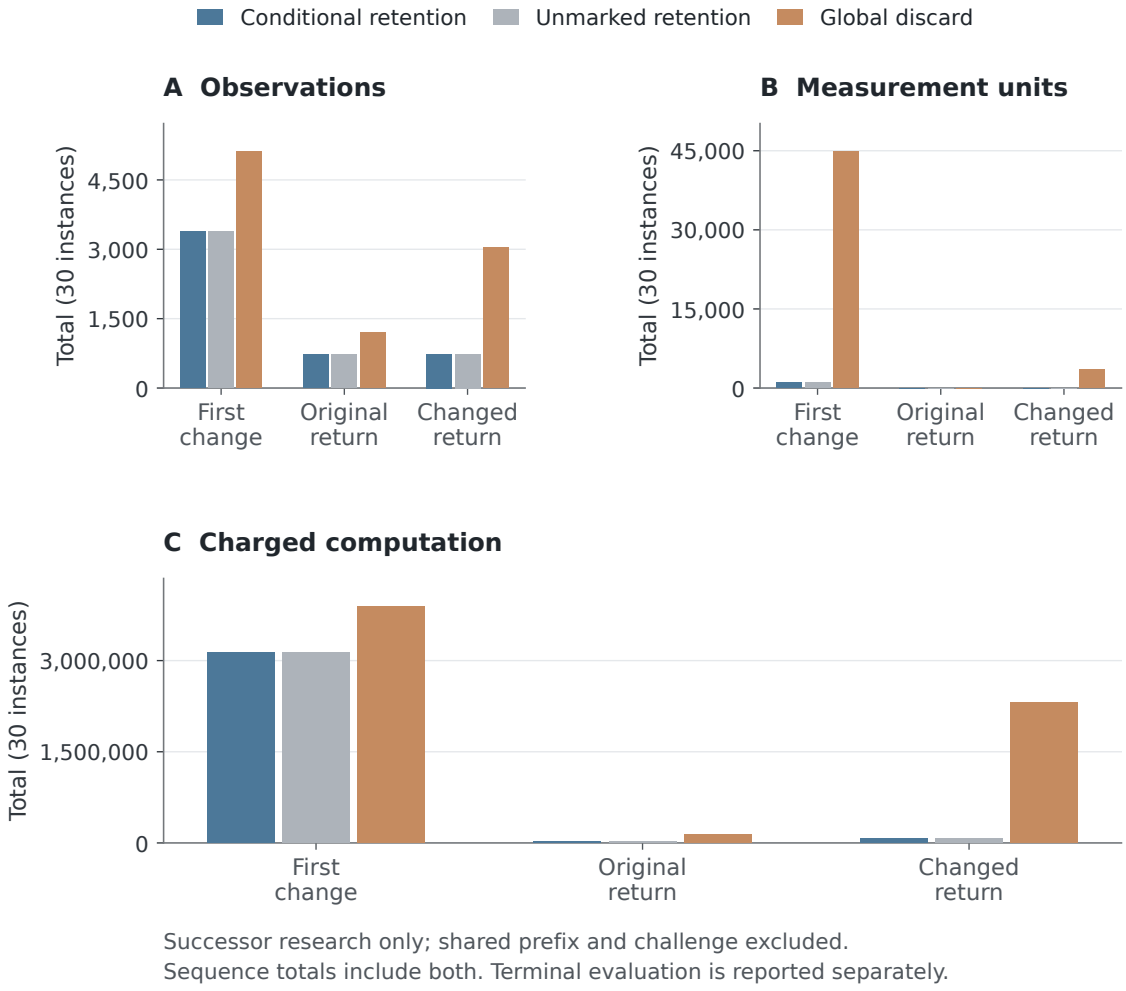


Figure H3. Where retention changes later research costs. Each bar sums one successor phase over 30 apparatus instances. The two retention arms have identical charged costs. These phase totals exclude the shared three-task prefix and common challenge, which are included in Table H2; terminal evaluation is reported separately. Vector PDF.

H.3.5 A boundary test: changing different components or neither

This study asks whether the retained composition supports different revisions while remaining reusable when nothing changes. After a common naturally learned A, B, C prefix, each apparatus forks into three conditions: unchanged $C = A + 0.7B$; a total-response gain change to $1.35A + 0.7B$; and a relation-radius change to $A + 0.7B_{0.45}$. Each condition is followed by a return to the original response. The gain intervention changes the total component's contribution without changing the event component, physical transitions or instrument readings. The radius intervention changes the event component while preserving the total. Both use the same public task description and measurement interface.

The researcher receives acquired observations through that interface. Branch labels and hidden intervention parameters configure the laboratory; they are not supplied to hypothesis search. All three conditions use matched probe seeds, the same resource allowances as Appendix H.3.4, and fresh one-task sessions carrying their own copies of the prefix library. The agent algorithm is unchanged, and no predetermined challenge is inserted. The retained response can be accepted, refitted, or used in further research according to the observations encountered by the ordinary loop.

We specified 30 new apparatus instances, with 15 in each generator profile. All learning finishes before evaluation. The prefix is evaluated once per apparatus, followed by six successor evaluations, for 270 research outcomes in total. We also evaluate the frozen prefix response on each successor's terminal controls, with its own measurement and evaluation costs. This reference tests whether revision was necessary for prediction; it is a fixed predictor rather than another research algorithm. The recorded actions, transfer attempts and exact called definitions describe how the agent investigates each condition.

All 30 instances completed, with all 90 prefix tasks and all 180 successor tasks passing terminal evaluation and fresh confirmation. The fixed original response passes every unchanged task and every old-condition return, but fails every first successor in both changed conditions. Its mean MSE is 0.2002 after the gain change and 0.02525 after the radius change. Accurate successor predictions therefore require adaptation in the changed conditions.

Table H3. Research behavior under three matched conditions. Each condition contains 30 current tasks and 30 original-condition returns. Observations and charged computation are mean research costs for the current task; no successor purchases auxiliary instrument measurements.

Condition	Current tasks passing	Returns passing	Frozen C on current task	Current observations	Current computation
Unchanged	30/30	30/30	30/30	24.00	427.00
Total-response gain 1.35	30/30	30/30	0/30	40.00	529.33
Event radius 0.45	30/30	30/30	0/30	113.07	107,719.20

All unchanged tasks select the original response through a calibrated call and add no new response concept. All gain-change tasks replay a research procedure and construct a new response using the exact original A and B definitions, without calling the old complete response C. They refit the composition using 16 more observations than in the unchanged

condition and conduct no relation-boundary comparisons. All radius-change tasks instead perform comparisons between relation candidates, retain the exact original component definitions, and call C inside the repaired response. Preserving a computation thus makes it available; the resulting explanation need not use every retained program.

On all 90 original-condition returns, the selected response again calls the original C, with 24 observations per task. The gain change and radius change incur different research costs because they require different operations. Relative to unchanged tasks, their paired mean first-successor computation increases are 102.3 units (95% interval $[98.3, 109.0]$) and 107,292.2 units ($[101{,}952.6, 113{,}324.9]$), respectively. These are differences between environmental conditions, not comparisons of competing research algorithms.

The common acquisition prefix costs 12,600 observations, 73,800 measurement units and 5,573,655 computation units across all instances. Including this prefix in each five-task branch yields totals of 14,040/73,800/5,599,275 for unchanged, 14,520/73,800/5,603,785 for the gain change, and 16,712/73,800/8,836,713 for the radius change, in the same resource order. Physical execution shares the prefix once, for 20,072 observations, 73,800 measurements and 8,892,463 charged computation units. Independent evaluation uses 69,120 observations and computation units, while the fixed-response reference adds 46,080 of each; both use zero instrument measurements.

H.3.6 What the composition studies establish

Across the three mechanism protocols, learned programs are used in later tasks, counterexamples lead to revised responses, and earlier successful knowledge can remain usable after revision. The independent-source experiment identifies two positive marginal contributions. The composition experiment adds preservation of an unchanged definition within a repaired response and a positive total effect of retaining source A. The subsequent four-arm study establishes substantial charged-resource savings relative to the same researcher restarted on every task, with nearly equal terminal accuracy. The retention intervention shows that even an observed-failing response can remain useful for repair and return. Its unmarked-retention control locates the gain in retained executable knowledge; it does not establish an additional benefit from the tested conditional-feedback update.

The component-change study extends this result to a different coefficient revision and to the decision to reuse an unchanged response. It connects the form of the observed change to distinct research behavior within the available search families. Together, the studies establish a concrete retention-and-revision mechanism: learned computation survives a failed application, participates in successful subsequent research, and remains available when earlier conditions return.

The resource results answer different questions. Source deletion measures knowledge availability and its downstream learning consequences. The task-local comparison evaluates complete research with and without cross-task knowledge and resource allocation. The fixed-library results show that the usefulness of a particular solver changes across task families. Direct calls and method replay distinguish two ways that accumulated knowledge can influence a later task. CPU measurements expose costs that the internal computation accounting does not fully capture.

H.3.7 Joint revision of expressions, representations and research procedures

The second instantiation tests whether several objects of revision can participate in one persistent research process. Thirty new apparatus instances, balanced over quadratic, cubic and absolute-value response families, each undergo eight tasks: source acquisition, changed coordinates, a local scope change, a hidden input, changed instrument roles, stochastic responses, simplification and return. The persistent and task-local arms have identical expression primitives, candidate search and per-task budget ceilings. In particular, the task-local arm can learn conditional responses from scratch. Five additional continuations per instance remove specific knowledge or measurement access from a common acquired prefix. The frozen campaign contains 630 research runs; all 30 instances completed. Appendix D specifies the protocol and maps the eight conceptual capabilities to their operational tests.

Complete research sequences. Persistent research passes independent evaluation on 238 of 240 tasks, versus 218 for task-local research. Requiring both this external criterion and successful research-time confirmation gives 232 versus 215. The latter paired difference is 0.567 tasks per apparatus, with a 95% family-stratified bootstrap interval of $[0.333, 0.800]$. Source acquisition, coordinate transfer, simplification and nonlinear return pass in both arms on all 30 instances. After instrument roles change, terminal passes are 30 versus 20; on the stochastic task they are 29 versus 18. The scope-change task favors task-local search slightly, with 29 versus 30 passes.

Table H4. Whole-sequence joint-growth results. Costs include all eight tasks, including source acquisition, unsuccessful search and confirmation. Confirmation and terminal evaluation are separate outcomes; neither count alone is their intersection.

Outcome	Persistent	Task-local
Independent evaluation passes	238/240	218/240
Research-time confirmations	234/240	218/240
Both confirmation and independent evaluation	**232/240**	**215/240**
Observations	23,780	23,840
Measurement units	21,840	22,800
Charged computation	17,068,659	22,000,724
Process CPU seconds	177.18	150.26

Charged computation decreases by 22.42%, with a saving in every apparatus. Its paired mean difference is $-164{,}402$ units per sequence, with interval $[-189{,}932, -139{,}616]$. Observation and measurement totals decrease by 0.25% and 4.21%, but their paired intervals include zero. Actual process CPU increases by 17.92%; its paired difference is $+0.897$ seconds, interval $[0.597, 1.191]$. The empirical advantage is therefore predictive success and charged computation, with an actual runtime penalty.

Which retained knowledge helps? In the new-expression branch, the target adds a cubic term to the acquired source. Retaining and deleting program knowledge both solve and confirm all 30 tasks. Retention changes the mean computation by $-2{,}303$ units, interval $[-6{,}019, 188]$; only seven instances save computation. The new abstraction is usable in a new composition, but its independent resource benefit is heterogeneous. For changed coordi-

nates, both arms again solve and confirm all tasks; program retention reduces computation in all 30 pairs, by a mean 5,568 units with interval [2,373, 8,399].

The procedure intervention retains the exact same program library and clears only learned methods before instrument names, units and ordering change. Both continuations solve and confirm 30/30 tasks. Keeping the methods reduces observations from 5,088 to 4,028 and computation from 5,219,559 to 3,944,154; both purchase 7,200 measurement units. The paired mean observation difference is -35.33, interval $[-45.47, -25.20]$, and computation difference is $-42{,}514$, interval $[-49{,}246, -35{,}687]$. These contrasts share an actually learned prefix, so including its acquisition cost leaves the absolute savings unchanged. The transferred procedures generate new fitting subquestions for the current instruments instead of copying the previously successful index. This supplies separate evidence that learned research steps can reduce later work.

Scope and observable distinctions. Learned scope revisions pass 29/30 tasks, whereas the frozen source response passes none. On the 27 cases containing an identifiable conditional branch, applying the new branch everywhere also fails every case; three equivalent hinge expressions do not support that diagnostic. Both old and new behavior are needed in this constructed setting. The remaining scope failure passes internal confirmation but has terminal MSE 5.00×10^{-5}, above the fixed 10^{-5} criterion.

When a hidden input affects the response, allowing paid instruments yields 30/30 confirmed terminal passes; withholding them yields 0/30 terminal passes. Access costs an additional 47.2 observations, 240 measurement units and 50,656 computation units per successor on average. All controls-only runs internally confirm stochastic descriptions. Such descriptions can represent unresolved variation, but cannot supply the full-response point predictions required by this deterministic task. This contrast demonstrates the value of the available distinguishing measurement, including its acquisition cost.

Uncertainty, simplification and return. The researcher introduces variance models in all 30 stochastic tasks and selects coordinate-dependent variance in 19. Against a constant variance estimate formed from the same paid repeats, the mean variance-estimation error decreases from 0.01461 to 0.01103. Normal intervals labelled 95% cover 91.09% of fresh responses, compared with 92.81% for the constant estimate. Better variance fitting therefore does not imply better coverage. Six persistent runs meet the external stochastic criterion but do not complete internal confirmation before exhausting the budget; one confirmed run misses the external mean-error criterion. On the linear task, learned simple programs and dense cubic fits to the same paid data both pass all 30 instances. Executing the learned programs on terminal data takes 69,120 primitive steps, versus 194,560 for the dense fits. All 30 subsequent returns recover the nonlinear response. This is a prediction-execution contrast, not a second complete research algorithm comparison.

H.3.8 Knowledge availability and marginal value

The integration studies connect learned procedures to the legacy controller's acquisition, confirmation and revisit operations (Appendix E). A continuation comparison starts both arms from identical post-production states with only 32 observation rows left. Selecting a candidate on the consumer's already paid data and purchasing one fresh 24-row confirmation

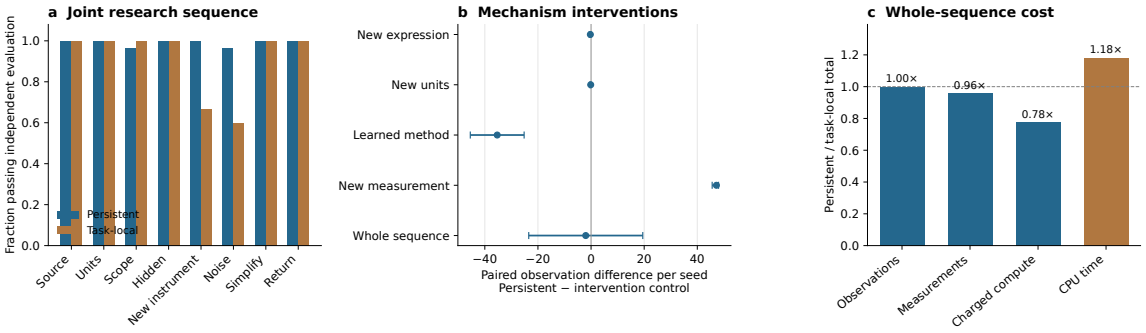


Figure H4. Joint growth and its costs. a, Independent-evaluation pass rates over 30 instances per stage; Table H4 separately gives confirmation and their intersection. **b**, Mean paired observation differences and 95% seed-bootstrap intervals for knowledge/access interventions and the whole sequence. Measurement access improves accuracy while increasing acquisition, so its positive cost is not an equal-accuracy inefficiency. **c**, Whole-sequence resource ratios; charged-computation savings coexist with higher CPU time. Vector PDF.

completes 12/12 previously unfinished consumers, versus 0/12 for the earlier controller. Whole-sequence charged computation increases by 6.57%, within the original ceilings.

All successful offers derive from earlier prefix concepts. A post-hoc intervention retains or removes the newly produced concept while preserving those earlier sources, procedures and incurred costs. Both arms produce the same final programs and confirmations in all 12 cases; retaining the new product adds 1,920 screening computation units per case. The controller therefore makes useful older knowledge available to an unfinished task, while this additional product provides no marginal predictive benefit. This contrast distinguishes the value of resuming research from the value of the new knowledge produced before that resumption.

H.3.9 An observed hypothesis life cycle

Apparatus 812676 from the confirmation profile of Appendix H.3.5 supplies a retrospective illustration of the recorded mechanism. The acquired library contains a total-response program A, an event-response program B_r, and their learned composition $C = A + 0.7B_r$. On the changed-boundary task, the first eight-row validation of the retained C gives MSE 0.0463443. The frozen C also fails independent evaluation, with MSE 0.0280351. The discrepancy motivates further research with the old computation still available.

The nested-relation generator constructs a correction inside the acquired composition. Its schematic form is

$$D = C + 0.7(B_{r'} - B_r).$$

The actual fitted program retains the original definitions and adds an executable correction. Its compiled research record links preserving the composed call, revising the internal relation, and refitting the answer. Those nodes describe completed operations: the initial retained fit has loss 0.0201398, while the final corrected fit has loss 1.69×10^{-19}. This is a concrete decomposition of the repair within a prescribed operation family.

The radius remains uncertain after a successful fit. The controller therefore proposes experiments on which competing relation boundaries disagree, acquires their outcomes, and fits again. Recorded refinements narrow the remaining interval as follows:

$$[0.4499800, 0.4575664] \longrightarrow [0.449999655, 0.450000288].$$

These bounds describe candidates consistent with acquired evidence; they are not statistical confidence intervals. After 100 research observations, the selected program is frozen. A further 24 observations confirm it at loss 2.01×10^{-19}, and an independent 256-row evaluation gives MSE 1.32×10^{-19}. The result enters the library as `concept_0004`.

When the original mechanism returns, the selected response directly calls the acquired C (`concept_0003`). No response concept is added. This task uses 24 observations in total, including eight for fresh confirmation, and passes the independent test at MSE 1.52×10^{-19}. Matching is operational: a stored program is bound to the current inputs and compared on current evidence. The experiment thus shows new hypothesis construction, an experimentally resolved internal uncertainty, fresh confirmation, and later use of an earlier hypothesis. The complete case record contains the candidate failures, acquired probes, parameter refinements and selected calls.

Direct reuse of the newly acquired revision is visible in the separate six-task trace for apparatus 26003 in Appendix C.3. Task 4 constructs D; task 5 calls C when the original condition returns; task 6 calls D when the changed condition recurs. Both return tasks acquire 24 observations and introduce no new response concept. All six tasks pass fresh confirmation and independent evaluation. The full cohort's 11/12 direct-call criterion and 72/72 terminal result, reported in Appendix C.3, distinguish this successful illustration from the aggregate outcomes.

H.3.10 Online recursion and frozen structural transfer

We prespecify six development and 30 separate evaluation source chains. Each chain first learns one source. Three researchers then receive identical copies of that acquired state for four independent continuations: unchanged structure and surface, unchanged structure with a changed surface, changed structure on the old surface, and changes to both. Surface changes rename and permute controls and change instrument calibration. Structural changes replace the response primitive. Each source-to-target composition is new to that source history; all four primitives remain available in the public research language. The four consumers never share newly learned knowledge.

Each task allows 24 observations and one million candidate-prediction computations, equally across arms. Correct executable hypotheses and independent predictions are the primary outcomes. Three new response observations confirm a frozen candidate; 256 independent observations evaluate it after research. Both use an MSE threshold of 10^{-10}. The 30 source tasks all pass. Each continuation arm passes confirmation and independent evaluation on 120/120 tasks, with no false confirmation and no runtime failure (Table H5).

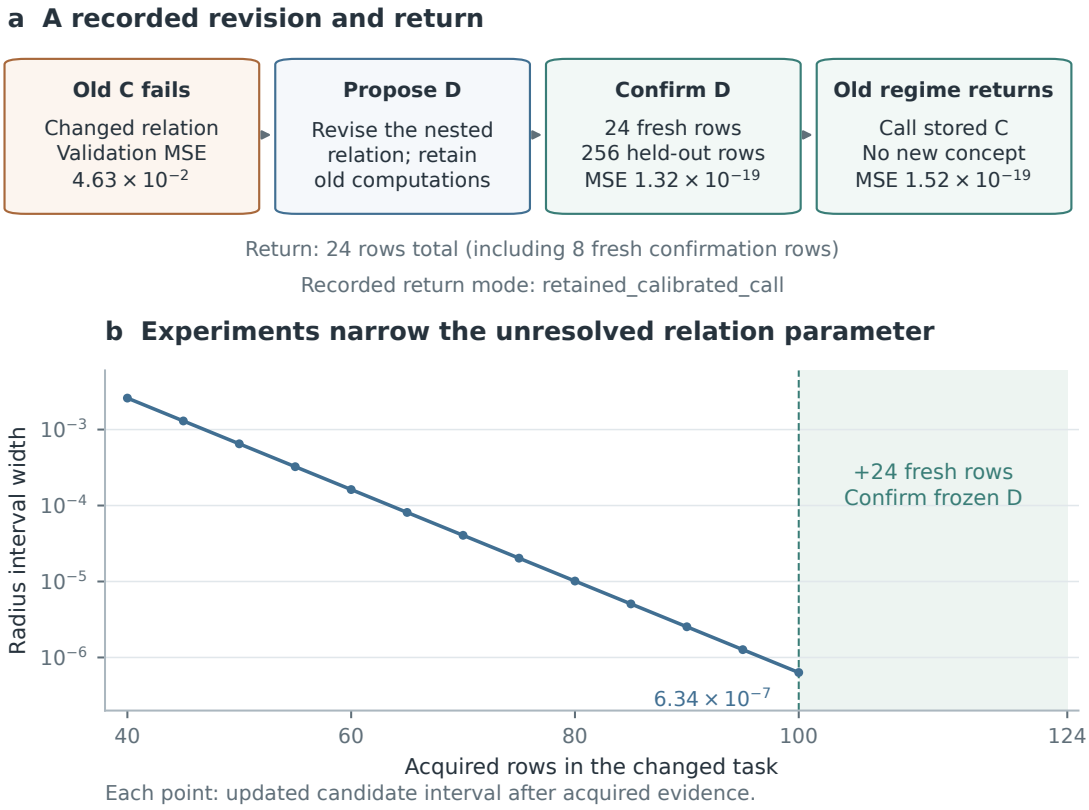


Figure H5. From a failed hypothesis to a confirmed revision and later reuse. a, In apparatus 812676, the acquired composition C fails on the changed task. A nested relation correction produces D, which passes fresh confirmation and independent evaluation. When the original condition returns, the agent directly calls C. **b**, Recorded parameter refinements reduce the width of the remaining relation-radius interval as observations accumulate. The shaded segment marks 24 further observations acquired after freezing D; they confirm the chosen program rather than refine the interval. Returning to C uses a separate task's 24 observations, including eight for fresh confirmation. Vector PDF.

Table H5. Online inquiry under matched access to evidence. Outcomes concern 120 continuations per arm, grouped in 30 independent source chains. The common source acquisition is shared by the intervention. Reuse counts actual calls to retained definitions after reassessment. Rewrites occur before the changed experiment is executed.

Researcher	Confirmed and independently correct	False confirmations	Retained-definition calls	Online method rewrites
Recursive, revisable method	120/120	0	60	47
Flat active research	120/120	0	60	0
Recursive, frozen method	120/120	0	60	0

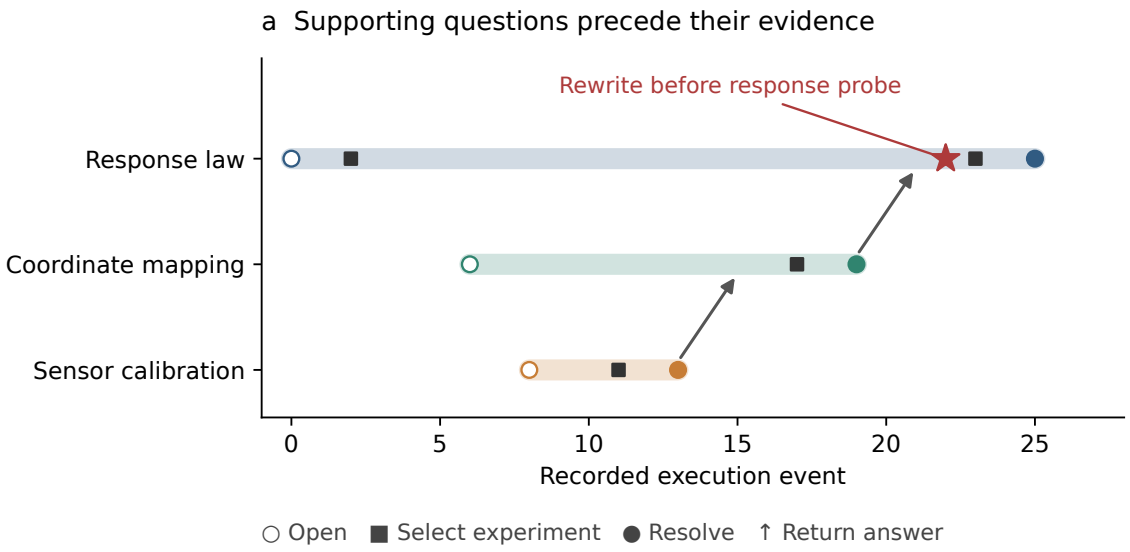
The 60 same-structure continuations in each arm use the retained response definition, including all 30 with a changed surface. The 60 changed-structure continuations identify a replacement instead. Recursive research rewrites a retained step on 23 same-structure/new-surface and 24 changed-structure/new-surface tasks. No rewrite occurs in either unchanged-surface condition. All four conditions pass in every arm, so this study establishes appropriate behavior within the finite language without an accuracy advantage over flat research or frozen procedures with fallback.

Figure H6 follows the first registered chain. The parent asks for a coordinate binding, whose child investigates calibration. After the coordinate answer identifies the second control as the driver, the retained response probe $(-0.6, -1)$ aliases the quadratic and absolute-value hypotheses: both predict 1. Before taking the next observation, the method changes the probe to $(-1, -0.6)$, which makes all four remaining laws predict different outcomes. Thus the supporting answer participates in choosing an informative parent experiment. A separate deterministic intervention test changes only the returned calibration evidence: the parent coordinate interpretation and response probe change, and fresh confirmation rejects the resulting incorrect branch. This intervention is a mechanism test, separate from the 30-chain performance cohort.

Resource totals describe a trade-off rather than the primary result: recursive, flat and frozen-method continuations use 852, 647 and 883 observations, respectively, and 388,496, 735,048 and 390,108 charged computations. The recursive organization acquires more observations than the flat control while scoring fewer candidate-action combinations. The complete protocol and results retain all registered chains and all three controls.

H.3.11 Does the framework improve a common-model Agent?

The completed comparison contains 30 source–target chains per arm, using the model and interface of Appendix H.2.8. Ordinary Code/ReAct, textual memory and Hegel share a 24-observation, 16-call and 300,000-token ceiling per task. The latter is checked before the next call. Each submitted predictor receives three fresh confirmation observations and separate 256-point terminal evaluation.



b Returned binding changes a useful test

Child answer: the second control is the driver

Candidate	Old probe	Revised probe
Linear	-1.00	-0.60
Quadratic	1.00	0.36
Absolute	1.00	0.60
Threshold	0.00	0.00

Old: (−0.6, −1.0) Revised: (−1.0, −0.6)

Old probe aliases quadratic and absolute responses.
Revised probe separates all four remaining laws.

Figure H6. Supporting evidence changes the experiment before execution. a, In chain 79101013, questions open before the observations that resolve them, and answers return to the parent. The star marks the method rewrite before the next response probe. **b**, Candidate predictions explain why that rewrite restores a distinction lost by the old step. The table is evaluated from the recorded candidate language and returned binding, not from a hidden mechanism label. Vector PDF.

Table H6. Complete common-model comparison. Each condition contains thirty registered source–target chains.

Knowledge condition	Source success	Registered target success
Ordinary Code/ReAct	29/30	29/30
Textual memory	30/30	29/30
Executable research knowledge (Hegel)	26/30	26/30

Every submitted predictor passes terminal evaluation. The failures concern completion: service interruption, exhausted tokens, or spending the observations needed for confirmation. Hegel executes 36 inherited experimental-program calls, including 15 without editing the inherited source, but that use does not produce an overall advantage. The complete experiment uses 894 calls and at least 20,395,181 reported tokens, with 70.3% of known input cached. The detailed failure breakdown and earlier service cohorts remain in Appendix F.1.

This finite laboratory offers four public response primitives that direct probes can identify. Ordinary code investigation is already near its ceiling. The result therefore answers the fourth question negatively on this cohort and motivates a harder mechanistic question: when short observations leave a supporting model ambiguous, does the agent acquire and use the evidence needed to revise it?

H.3.12 Can a supporting investigation change the parent explanation?

The temporal laboratory makes short histories ambiguous: finite-history and recurrent instruments agree on the legal three-step source probes, but can differ on 24-step targets. In a 12-pair offline study, all 24 targets pass fresh confirmation and terminal prediction. Finite cases admit history expansion; every recurrent case requires a measured instrument investigation. A four-step distinguishing probe selects the coordinate used in the parent response, whereas a three-lag quadratic refitted on the same paid observations fails all recurrent cases (MSE 0.00762–0.316).

A stronger control shares the instrument candidates, response fits, experiment repertoire and ceilings. On 30 further pairs, separate child confirmation and flat response validation both solve 60/60 targets. In recurrent cases, the flat controller uses 159 rather than 163 observations and 45 rather than 57 measurement units, while fitting three instrument-conditioned responses rather than one. Measured support can repair the representation; its separate organization as a child offers no accuracy gain in this comparison (Appendix F.2).

Real-model traces expose a further requirement: evidence must affect subsequent actions. In the first shared-operator pilot, Code and textual memory each solve 2/2 targets, while Hegel buys an informative measurement but completes neither revision. A development retest makes post-acquisition analysis executable: new meter evidence refutes the finite instrument at MSE 0.008481, supports a recurrent alternative at 7.12×10^{-19}, and enables parent confirmation at 1.18×10^{-17}. All three arms then solve 2/2 targets. A later matched advice-visibility pilot on four known targets gives 3/4 versus 2/4; its additional success actually executes the suggested distinction, assesses the instrument and revises the parent. Persistent advice on new apparatuses yields a tie, 4/6 attempted targets in each

condition. These studies establish successful measured support-to-parent updates and repeatable failure modes in using that support. Their full traces and costs are in Appendices F.3 and F.8.

H.3.13 Can a counterexample improve the research procedure itself?

Here the object of learning is an executable method, not just a fitted response. An author receives a source failure, edits the procedure, practices it on public source conditions, and freezes a successfully published program. Other source archives then test that program without language-model calls. Keeping the numerical operators fixed makes the revised control flow observable.

Three successive interventions address different procedural assumptions:

Table H7. Research procedures revised through source counterexamples. Each row has its own frozen cohort; counts are not pooled across rows.

Assumption exposed by a counterexample	**Required procedural revision**	**Frozen result**
A supported instrument implies a supported parent response	Retain the instrument but refit the response after parent refutation	Old 10/18; revised 18/18
All accumulated response records describe the current mechanism	Restrict fitting to current, instrument-supported records	Old 20/32; revised 32/32
Repairing the new cases preserves earlier capabilities	Test the prior prediction first, then retain or reopen its supporting instrument	Initial revision 36/44; corrected and refit-both control each 56/56

The first successful author makes one edit and completes three source practices in five calls. All eight response-only transfer failures are repaired while the ten earlier successes are preserved. Transfer observations fall from 2,928 to 2,544, measurement units remain 3,888, and computation rises from 183,048 to 200,712. This is a learned change in how failure is investigated with a measurable accuracy effect (Appendix F.6).

The scope revision follows a failed author phase that continues mixing obsolete and current records. Its successful program fits the response on the records supporting the current instrument. The selective revision then addresses the cost and overreplacement of this refit-both procedure. Its first version repairs all twelve new cases while breaking eight earlier ones; adding the observed failure to practice leads to preservation of all 56 target outcomes. The final program retains 20 instruments, reopens 32 and directly reuses four predictions. Against refit-both, charged computation falls 35.48%, measurements are unchanged, and observations increase by 384. Each row is a separate, disclosed development cohort; the table is not a pooled accuracy estimate. Appendices F.7–F.8 preserve all intermediate failures, authoring costs and transfer protocols.

H.3.14 Can the agent adopt and preserve an effective research method?

A working research operator need not be used. Live pilots that exposed a complete procedure and then presented it in feedback produced no invocations, including 24 unselected displayed options. Five author phases likewise failed to replace a fresh-loss cutoff with evidence-dependent scoped repair, despite a paid diagnostic showing why the cutoff was inadequate. Integrating the existing scoped-repair mechanism by hand yielded a usable program. When three later authors received that program, the old candidate and their source evidence, all three copied the supplied repair and passed all five fresh practices in two calls each. The ten unchanged transfer outcomes were reused once. This establishes adoption of supplied knowledge and keeps it distinct from invention (Appendix F.8).

A further response change occurs after an instrument-supported response has already passed validation. The supplied method now stops after fresh refutation, solving 10/12 targets. Three request-only authors publish no revision. With the same practice machinery triggered immediately after actual source edits, two of three authors publish new programs, each satisfying all six source conditions and independently executing successfully on the twelve frozen targets. The 24 target executions are two programs on the same twelve contracts, not 24 independent tasks. The 25 source practices include fifteen successes and ten failures; the condition costs 29 calls and 1,750,532 reported tokens. The first program uses 2,296 target observations versus the unchanged method's 2,010. The host initiates practice; the models write the changed procedures.

The third author supplies an instructive failure. An intermediate program passes the new case, but a later edit removes the reacquisition path and fails it. The successful source remains in the history yet is not published. Two subsequent three-author recall attempts, each limited to four calls, produce no publication or explicit restoration. This separates creating a useful procedure from keeping and selecting it.

We therefore test the submission-time retention rule in Appendix H.2.9. One of three author attempts submits a draft whose practice fails; the host then tests an older, supported source. That source retains its own prior confirmation, passes five missing practices and is published at 6/6. Frozen execution then passes 12/12 targets with zero false confirmations. The other attempts end in provider capacity failure and inspection-only call exhaustion. The successful recovery uses an LLM-authored method, while selection and completion are performed by the host. Appendix F.9 reports the complete costs and the sequential-control boundary. These results make host-supported preservation an executed part of research. The following intervention separately tests selection by the model.

H.3.15 Can the model recover a useful method without host selection?

We resume the failed-method state in twenty fixed author attempts: ten retain the practiced sources and ten remove the supported source while preserving the failed candidates and their actual outcomes. A direct evidence summary presents all available candidates without recommending one. Both conditions disable host fallback and share an eight-call, 350,000-token and 2,400-observation ceiling. At the starting point, the useful source has passed only case_5; publication still requires confirmation of its five remaining cases. These are repeated model decisions on one regression state. Future confirmation streams vary with

replica and practice history, so the groups share budgets rather than identical random streams (Appendix F.11).

Table H8. Autonomous recall and completion. Every rate uses all ten registered author attempts in its condition. Dashes indicate an unavailable source, rather than a failed retrieval test.

Outcome	Supported history retained	Supported history removed
Model selects the supported earlier source	10/10	—
Publication with the recalled source unchanged	7/10	—
Publication after a source revision	1/10	2/10
All successful publications	8/10	2/10
Final call-limit exhaustion	2/10	3/10
Final token-limit exhaustion	0/10	5/10

All ten retained-history authors select the useful source themselves. Seven publish it unchanged; one edits it after retrieval and publishes a revised program. The remaining two retrieve it but never submit before their call limit. All ten removal authors write revisions, of which two pass publication. Every published program satisfies all six source cases. Retention uses 53 model calls and 1,687,907 tokens, versus 70 calls and 3,717,879 tokens after removal, including all unsuccessful attempts.

The seven unchanged publications refer to one previously evaluated program and its existing 12/12 target outcomes once; this study executes no new transfer targets. The three revised publications establish source-case success only. This closes a local autonomy gap: the model can select and complete evidence for an earlier useful method without the host choosing it. The unfinished retrievals and repeated use of one historical failure state delimit the result.

H.3.16 Do learned hypotheses improve downstream optimization?

The decision study uses three-dimensional integer controls $x \in \{0, \dots, 100\}^3$, normalized as $z = x/100$, with two equally weighted objective outputs sharing the mean $F_1 = F_2 = 0.35 + (z - c)^T Q(z - c)$. The constraint follows the existing StatePolicyRZDT1 simulator. Each output receives independent noise with standard deviation 0.03. Three profiles keep an interacting quadratic fixed, change its center, or change both center and interactions. The latter changes begin at online observation 9, after the common 32-observation prefix. Centers, matrices and the change clock remain inside the simulator; research receives purchased measurements.

The completion-aware controller is frozen before three new seeds per profile, for nine cases. Every arm receives the same 32 paid initialization vectors and 32 further online calls. The controls are native SC-KG and the preceding Hegel controller. The tested extension plans fitting and future-validation work to complete objective research, and compares an already learned objective explanation with the deployed posterior. Final regret is the recommended objective minus the feasible, on-grid minimum 0.35; all three arms return feasible recommendations in 9/9 cases.

Table H9. Mean terminal regret in the SC-KG confirmation cohort. Each profile contains three new seeds. Earlier and completion-aware Hegel correspond to the frozen v0.52 and v0.53 sources identified in Appendix G.

Profile	Native SC-KG	Earlier Hegel	Completion-aware Hegel
Stable interactions	0.024937	0.013327	0.000177
Center change	0.122027	0.036303	0.003333
Center and interaction change	0.153037	0.069907	0.050323
All nine cases	0.100000	0.039846	0.017944

Completion-aware research improves all nine recommendations against native SC-KG, and improves five with four ties against the earlier Hegel controller. Future evidence supports deployment of an initial interaction explanation in three cases and a changed lower-order objective model in three cases. No post-change interaction model is installed, and three research scopes remain unfinished when optimization ends. The improvement therefore coexists with an unresolved part of the proposed revision process.

A separate, later nine-case confirmation compares a constraint-knowledge extension with its immediate predecessor and native SC-KG. Both Hegel conditions have mean regret 0.012751, versus 0.108191 for native SC-KG: eight wins and one loss, with every recommendation feasible. The extension ties its predecessor case by case. In three additional constraint-change cases it reduces constraint MSE by 37.71% without changing the final objective. These results separate a useful complete research-and-optimization system from the incremental decision value of another learned component. Appendix G also reports the earlier same-data null result and the subsequent deployment-feedback comparison.

We additionally rerun the nine Table H9 cases against the historical four-property configuration, using one archived SC-KG backend for all three arms. Mean regret is 0.100000 for native SC-KG, 0.129478 for the full structural-prior configuration, and 0.017944 for Hegel; every arm is feasible on 9/9 cases. Hegel improves all nine recommendations against each control. The full prior improves five cases, ties one and worsens three against native SC-KG. Its gate selects the learned coordinate representation at both initialization and completion in all nine cases. Thus this comparison tests the original full configuration with its selection rule. Appendix G.4 gives its shared-prefix design, timing and retrospective status.

Appendix I. From recurrent representation to transferable research

I.1 Shared task and the source investigation

This study connects three registered cohorts: source representation research, method authorship and transfer, and live continuation after interface repair. Every LLM stage requests `gpt-6-luna`, medium reasoning, through the same subscription-backed Codex provider. The public apparatus accepts resettable trajectories with scalar controls in $[-1, 1]$ and a maximum length of 24. Agents may buy response observations and optional meter readings, write ordinary Python, inspect their histories, and test predictors. The prediction inter-

face admits current and preceding controls; it does not supply privileged simulator state or future meter values.

Each source or target investigation starts with 24 independent three-step trajectories, totaling 72 response rows, without meter readings. Paired live arms receive the same controls and apparatus parameters, tools, archive and limits: 576 response rows, 96 model calls, 10 million reported tokens, 600 seconds of cumulative Python execution and 250,000 charged computation units. Each fresh confirmation buys 96 response rows. Final evaluation uses 512 rows, with trajectory length 24 and MSE threshold 10^{-10}. All research in a cohort stops before its terminal labels are generated. A failed scored confirmation becomes paid research evidence; an execution timeout without a score is not a predictive counterexample.

The source cohort contains two new recurrent parameter settings crossed with Code and Hegel. Both arms receive the complete earlier six-source anonymous archive, without successful files from the intervening representation-rescue runs. They also share conditional-component evaluation, binding and retention tools. Hegel adds explicit research questions and child-result bookkeeping. The final source outcomes are Hegel 1/2 and Code 0/2. The other Hegel episode's last confirmation times out; it has no new scored confirmation error. All four outcomes remain available.

In the successful source, Luna purchases 96 rows of meter readings and finds a recurrence of the form

$$m_t = am_{t-1} + bu_t + cu_{t-1}, \qquad z_t = (m_t, m_{t-1}, u_t, u_{t-1}), \qquad \hat{y}_t = q_2(z_t),$$

with reset initial conditions. Here q_2 is a full quadratic readout: an intercept, four linear terms and ten products, expressed after affine standardization in the source predictor. During composition the actor omits a previous-control feature, observes the resulting mismatch, restores that feature and validates the exported model. Fresh-confirmation MSE is 1.4215×10^{-30}; terminal MSE is 1.5208×10^{-30}. The actual predictor and research trace identify the authored computation. All four source episodes leave the optional component-binding and child-research interfaces unused; the successful model composes its intermediate mechanism within one Python program.

I.2 Learning and executing a parameter-identification procedure

One designated Luna author receives the complete four-investigation archive, including the three unsuccessful episodes. The author writes and practices a self-contained `select_actions` program using the existing method-publication interface. Practice covers two returns to source parameters and two additional parameter settings. The author allowance is 96 calls, 10 million reported tokens, 12,000 response rows, 3,600 Python seconds and four million charged computation units; task practice retains the target confirmation contract. Eight cumulative practices contain six successes and two failures. The final unchanged source passes all four required settings: one through unchanged predictor reuse and three through refitting. Publication takes seven model calls.

The published procedure evaluates archived predictors on current paid rows. If none fits, it jointly estimates three latent recurrence coefficients and fifteen quadratic-readout coefficients using ordinary nonlinear least squares. It exports the resulting predictor and requests confirmation. Each invocation receives the accumulated observations, so subsequent calls

can fit the same representation to evidence that refuted an earlier numerical instance. The source structure is learned from the investigations; its parameters are estimated separately for each target.

Six new-parameter target settings and the two live-control settings are recorded before authoring. Publication precedes target execution; consumers receive the original archive and the published method, without practice-fitted models or practice observations. There is one author attempt. Had it failed to publish, the planned target slots would have remained unavailable rather than receiving a hand-selected draft. The complete method summary retains author costs, all practice outcomes and every target.

All six frozen targets pass both confirmation and terminal evaluation. Their first fits interpolate the 72-row short prefix but fail the first 96-row long-path confirmation. Resuming the same program on the resulting 168 rows produces a revised predictor that passes the next 96-row check. Terminal MSE ranges from 7.2732×10^{-32} to 3.9552×10^{-31}. Each target acquires 264 response rows, no meter readings and no LLM calls. The frozen executor supplies the next program invocation after failure; the long trajectories come from the confirmation contract. The program's learned contribution is parameter identification and reuse of the enlarged evidence, rather than synthesis of the confirmation experiments.

I.3 Live continuation and the retained negative control

Live Code and Hegel receive the same archive and published procedure on the two designated targets. Both can inspect, execute, edit or ignore it. Their first method calls produce the same failed confirmation errors within each pair: 1.1621 and 1.2814. Original live Code solves both targets in ten total calls. Hegel solves one in five calls; on the other it stops reusing the method, tries different predictors, and finishes after 23 calls and 576 observations without confirmation. Its last scored confirmation error is 0.2743. This failure is preserved independently of the later repair.

The trace audit identifies an asymmetric state-update defect. Before confirming a submission, Hegel replaces its in-memory manifest with a temporary model-submission manifest. Without a separate shared manifest file, the next method view lacks the registered invocation limit and prior practice support. Code retains that information. The repair reuses an existing path-based metadata merge in the live view. The view also exposes the rejected predictor, newly paid evidence, original invocation and editable source; the actor remains responsible for choosing to continue, revise, switch or stop.

The development repeat restarts the same two targets from the original 72-row prefix. The scientific program, source archive, model and limits stay fixed, and no prior target result enters the agents' context. Both Hegel episodes choose a second original-method invocation immediately after the first failed confirmation. Each then passes the new confirmation and terminal evaluation, in two Luna calls and 264 response rows. Final MSEs are 2.2380×10^{-31} and 2.2773×10^{-31}. Neither rewrites the method. This is live reuse with updated fitted parameters; it involves no automatic live retry or intervention during the run.

Code passes the first target in four calls. On the second it examines an old predictor, compares finite-lag fits, acquires additional trajectories, and writes a fit with paid-data MSE 6.8495×10^{-30}. Its scratch program prints parameters but does not export a predictor. After buying more observations it invokes the original procedure on 360 rows; that call consumes

558.197 seconds and ends at the cumulative 600-second limit. There is no second candidate or confirmation. The saved event trace therefore distinguishes an incomplete execution from a new scientific refutation. The two Hegel traces are also retained for target 000 and target 003.

I.4 Resources and limits of inference

Table I1. Stage-specific research expenditure. Values include unsuccessful attempts, practice and initial paid prefixes. Tokens are provider-reported; rows and meter units count acquired responses and measurement charges. Python seconds measure cumulative execution, not elapsed study duration; charged computation is the runner's ledger. Source acquisition is counted once. Frozen targets and original live targets exclude authoring costs, which have their own row.

Stage / arm	Calls	Tokens	Rows	Meter	Compute	Python s
Source / Code	58	4,362,045	1,056	0	7,800	610.47
Source / Hegel	92	9,248,557	1,152	360	5,088	639.10
Method author and practice	7	1,942,281	1,632	0	4,872	256.35
Frozen targets	0	0	1,584	0	2,592	631.37
Original live / Code	10	2,293,307	528	0	1,536	216.68
Original live / Hegel	28	8,501,682	840	0	1,992	204.34
Development repeat / Code	13	3,483,126	624	0	936	866.45
Development repeat / Hegel	4	908,944	528	0	864	670.43

Together these stages cost 212 model calls, 30,739,942 reported tokens, 7,944 response rows, 360 meter units, 25,680 charged computation units and 4,095.20 Python seconds. They inherit an earlier six-source archive whose construction is outside these incremental totals. Recorded cached input tokens are 4,317,440 for the source cohort, 1,998,848 for method authoring plus original live execution, and 554,496 for the development repeat. Cache hits are already included in provider token usage, not subtracted from the reported counts. The experiments establish no amortized net saving after acquisition.

The source representation and numerical parameters need not be uniquely identifiable: latent rescaling and dependent quadratic features can preserve predictions. Six successful targets assess one author's method within the same apparatus family. The fixed interface and confirmation schedule remain supplied, and the source, authoring and consumer phases are separately initiated. The latest live repeat reuses two opened targets while changing metadata and feedback together, so model variability and the individual effects of those changes remain confounded. Its success cannot replace the thirty-chain complete-agent comparison or establish a recursive advantage. Parameter revision and method continuation are demonstrated here; autonomous selection of a new method family, independent method-learning replications, and transfer across distinct mechanisms remain separate questions.

References

Agarwal, A. K., and Yamada, M. (2026). Hypothesis-driven reasoning for large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(3), 1686–1693. doi:10.1609/aaai.v40i3.37146.

Alchourrón, C. E., Gärdenfors, P., and Makinson, D. (1985). On the logic of theory change: Partial meet contraction and revision functions. *The Journal of Symbolic Logic*, 50(2), 510–530. doi:10.2307/2274239.

Bengio, Y., Courville, A., and Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798–1828. doi:10.1109/TPAMI.2013.50.

Boyd, S., and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press. Author-hosted book.

de Kleer, J. (1986). An assumption-based TMS. *Artificial Intelligence*, 28(2), 127–162. doi:10.1016/0004-3702(86)90080-9.

Ellis, K., Wong, C., Nye, M., Sablé-Meyer, M., Morales, L., Hewitt, L., Cary, L., Solar-Lezama, A., and Tenenbaum, J. B. (2021). DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. *Proceedings of PLDI*. Conference record.

Hegel, G. W. F. (1892). *Lectures on the History of Philosophy*, Vol. I. Translated by E. S. Haldane. Kegan Paul, Trench, Trübner & Co. Introduction A.3; consulted in the 1955 reprint. Public-domain text.

Huang, K., Jin, Y., Li, R., Li, M. Y., Candès, E., and Leskovec, J. (2025). Automated hypothesis validation with agentic sequential falsifications. *Proceedings of ICML*, PMLR 267, 25372–25437. Proceedings.

Iacob, A., et al. (2026). The Red Queen Gödel Machine: Co-evolving agents and their evaluators. *arXiv:2606.26294*. Preprint.

Iyengar, G. N. (2005). Robust dynamic programming. *Mathematics of Operations Research*, 30(2), 257–280. doi:10.1287/moor.1040.0129.

Le Goues, C., Nguyen, T., Forrest, S., and Weimer, W. (2012). GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1), 54–72. doi:10.1109/TSE.2011.104.

Melrose, R. (2020). *Functional Analysis: Lecture Notes for 18.102, Spring 2020*. Massachusetts Institute of Technology. Chapter 1, sections 2, 4 and 5. Lecture notes.

Novikov, A., et al. (2025). AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*. Preprint.

Pearl, J. (2009). Causal inference in statistics: An overview. *Statistics Surveys*, 3, 96–146. doi:10.1214/09-SS057.

Piriyakulkij, W. T., Langenfeld, C., Le, T. A., and Ellis, K. (2024). Doing experiments and revising rules with natural language and probabilistic reasoning. *Advances in Neural Information Processing Systems*, 37, 53102–53137. Proceedings.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36. Proceedings.

Solar-Lezama, A., Tancau, L., Bodík, R., Seshia, S. A., and Saraswat, V. A. (2006). Combinatorial sketching for finite programs. *Proceedings of ASPLOS*, 404–415. Author publication page.

Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press. Publisher.

Varici, B., Acartürk, E., Shanmugam, K., and Tajer, A. (2024). General identifiability and achievability for causal representation learning. *Proceedings of AISTATS*, PMLR 238, 2314–2322. Proceedings.

Wang, G., et al. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv:2305.16291*. Preprint.
Wong, L., Ellis, K. M., Tenenbaum, J., and Andreas, J. (2021). Leveraging language to learn program abstractions and search heuristics. *Proceedings of ICML*, PMLR 139, 11193–11204. Proceedings.
Yamada, Y., Lange, R. T., Lu, C., Hu, S., Lu, C., Foerster, J., Clune, J., and Ha, D. (2025). The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv:2504.08066*. Preprint.
Zhang, J., Hu, S., Lu, C., Lange, R. T., and Clune, J. (2025). Darwin Gödel Machine: Open-ended evolution of self-improving agents. *arXiv:2505.22954*. Preprint.
Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J., and Huang, G. (2023). ExpeL: LLM agents are experiential learners. *arXiv:2308.10144*. Preprint.
Zhou, P., et al. (2024). SELF-DISCOVER: Large language models self-compose reasoning structures. *Advances in Neural Information Processing Systems*, 37. Proceedings.
Zsák, A. (2023). *Functional Analysis, Part III, Lent 2023*. University of Cambridge; notes taken by Leonard Tomczak. Sections 1.4 and 3.2. Course notes.

Artifact paths in the companion bundle

Local evidence links in this PDF point to this index. Extract the companion materials archive and open the listed paths from its project root.

[**A1**] paper/ARTIFACT_INDEX.md
[**A2**] src/hegel_machine/cognition/language.py
[**A3**] src/hegel_machine/cognition/memory.py
[**A4**] src/hegel_machine/cognition/research_programs.py
[**A5**] src/hegel_machine/cognition/unified_research.py
[**A6**] src/hegel_machine/cognition/unified_session.py
[**A7**] src/hegel_machine/cognition/unified_object_lab.py
[**A8**] src/hegel_machine/cognition/sc_kg_environment.py
[**A9**] src/hegel_machine/cognition/objective_research.py
[**A10**] artifacts/kg_op_paper_integration_20261007/evidence.json
[**A11**] artifacts/narrow_research_cycle_20261004_r2/protocol.json
[**A12**] artifacts/narrow_research_cycle_20261004_r2/summary.json
[**A13**] artifacts/two_source_cycle_20261004/protocol.json
[**A14**] artifacts/two_source_cycle_20261004/summary.json
[**A15**] artifacts/composed_local_cycle_20261004_r3/protocol.json
[**A16**] artifacts/composed_local_cycle_20261004_r3/summary.json
[**A17**] artifacts/paper_composed_comparison_20261005/protocol.json
[**A18**] artifacts/paper_composed_comparison_20261005/summary.json
[**A19**] artifacts/paper_scoped_retention_20261005/protocol.json
[**A20**] artifacts/paper_scoped_retention_20261005/summary.json
[**A21**] artifacts/paper_revision_choice_20261005/protocol.json
[**A22**] artifacts/paper_revision_choice_20261005/summary.json
[**A23**] artifacts/growth_paper_20261006_main_30/protocol.json
[**A24**] artifacts/growth_paper_20261006_main_30/summary.json
[**A25**] artifacts/growth_paper_20261006_main_30/derived_analysis.json
[**A26**] artifacts/workflow_bridge_20261006_main_12/results/protocol.json
[**A27**] artifacts/workflow_bridge_20261006_main_12/results/summary.json
[**A28**] artifacts/workflow_demand_20261006_main_12/protocol.json
[**A29**] artifacts/workflow_demand_20261006_main_12/results/summary.json
[**A30**] artifacts/workflow_budget_20261006_main_12/protocol.json
[**A31**] artifacts/workflow_budget_20261006_main_12/summary.json
[**A32**] artifacts/paid_revisit_20261006_main_12/protocol.json
[**A33**] artifacts/paid_revisit_20261006_main_12/summary.json
[**A34**] artifacts/new_product_value_20261006_posthoc_12/protocol.json
[**A35**] artifacts/new_product_value_20261006_posthoc_12/summary.json
[**A36**] artifacts/growth_paper_20261006_frozen_source
[**A37**] src/hegel_machine/cognition/research_growth.py
[**A38**] src/hegel_machine/cognition/growth_models.py
[**A39**] src/hegel_machine/cognition/growth_procedures.py
[**A40**] src/hegel_machine/cognition/growth_uncertainty.py
[**A41**] scripts/analyze_growth_paper_study.py

[**A42**] artifacts/composed_local_cycle_20261004_r3/diagnostic_26107_method_reuse.json

[**A43**] paper/RESEARCH_PLAN.md

[**A44**] artifacts/paper_retention_stopping_audit_20261007/cost_contract.json

[**A45**] artifacts/online_inquiry_20261005_main/source/src/hegel_machine/cognition/online_inquiry.py

[**A46**] artifacts/growth_paper_20261006_frozen_source/src/hegel_machine/cognition/research_growth.py

[**A47**] artifacts/growth_paper_20261006_frozen_source/src/hegel_machine/cognition/growth_models.py

[**A48**] artifacts/growth_paper_20261006_frozen_source/src/hegel_machine/cognition/growth_uncertainty.py

[**A49**] artifacts/online_inquiry_20261005_main/case_79101013/case.json

[**A50**] src/hegel_machine/cognition/instrument_hypotheses.py

[**A51**] src/hegel_machine/cognition/temporal_instrument_research.py

[**A52**] src/hegel_machine/cognition/temporal_response_experiments.py

[**A53**] artifacts/online_inquiry_luna_subscription_20261005_main_30/RUN_STATUS.md

[**A54**] artifacts/temporal_recursive_research_20261005/RUN_STATUS.md

[**A55**] artifacts/temporal_matched_repertoire_20261006/RUN_STATUS.md

[**A56**] artifacts/temporal_shared_operators_luna_20261006/RUN_STATUS.md

[**A57**] artifacts/temporal_evidence_continuation_luna_20261006/RUN_STATUS.md

[**A58**] artifacts/temporal_method_learning_retest_luna_20261006/RUN_STATUS.md

[**A59**] artifacts/temporal_method_practice_feedback_luna_20261006/RUN_STATUS.md

[**A60**] artifacts/temporal_method_dependency_feedback_luna_20261006/RUN_STATUS.md

[**A61**] artifacts/temporal_method_dependency_extended_luna_20261006/RUN_STATUS.md

[**A62**] artifacts/temporal_conditional_method_retest_luna_20261006/RUN_STATUS.md

[**A63**] artifacts/temporal_paired_candidate_replay_20261006/RUN_STATUS.md

[**A64**] artifacts/temporal_paired_online_repaired_luna_20261006/RUN_STATUS.md

[**A65**] artifacts/temporal_paired_coverage_continuation_luna_20261006/RUN_STATUS.md

[**A66**] artifacts/temporal_paired_state_resume_luna_20261006/summary.json

[**A67**] artifacts/temporal_paired_retention_luna_20261006/RUN_STATUS.md

[**A68**] artifacts/temporal_paired_submission_retest_luna_20261006/RUN_STATUS.md

[**A69**] artifacts/temporal_calibration_transfer_luna_20261006/RUN_STATUS.md

[**A70**] artifacts/temporal_counterexample_learning_luna_20261006/RUN_STATUS.md
[**A71**] artifacts/temporal_failure_selectivity_luna_20261006/RUN_STATUS.md
[**A72**] artifacts/temporal_evidence_scope_learning_luna_20261006/RUN_STATUS.md
[**A73**] artifacts/temporal_selective_revision_luna_20261006/RUN_STATUS.md
[**A74**] artifacts/temporal_selective_preservation_luna_20261006/RUN_STATUS.md
[**A75**] artifacts/temporal_historical_methods_luna_20261006/RUN_STATUS.md
[**A76**] artifacts/temporal_historical_actions_luna_20261006/RUN_STATUS.md
[**A77**] artifacts/callable_source_returns_20261006/RUN_STATUS.md
[**A78**] artifacts/legacy_session_luna_20261006/RUN_STATUS.md
[**A79**] docs/hegel_machine_candidate_retention_20261006.md
[**A80**] artifacts/candidate_retention_cycle_luna_20261006/RUN_STATUS.md
[**A81**] artifacts/python_refit_criteria_luna_20261006/RUN_STATUS.md
[**A82**] artifacts/python_research_routing_final_luna_20261006/RUN_STATUS.md
[**A83**] artifacts/python_research_advice_ablation_luna_20261006/RUN_STATUS.md
[**A84**] artifacts/python_pending_advice_luna_20261006/RUN_STATUS.md
[**A85**] artifacts/python_procedure_luna_20261006/RUN_STATUS.md
[**A86**] artifacts/python_procedure_offer_luna_20261006/RUN_STATUS.md
[**A87**] artifacts/python_method_candidate_replay_20261006/RUN_STATUS.md
[**A88**] artifacts/python_method_publication_luna_20261006/RUN_STATUS.md
[**A89**] artifacts/python_selective_method_completion_luna_20261006/RUN_STATUS.md
[**A90**] artifacts/python_scoped_method_comparison_luna_20261006/RUN_STATUS.md
[**A91**] artifacts/python_current_evidence_bridge_20261006/RUN_STATUS.md
[**A92**] artifacts/python_method_adoption_luna_20261006/RUN_STATUS.md
[**A93**] artifacts/python_method_edit_practice_luna_20261006/RUN_STATUS.md
[**A94**] artifacts/python_method_recall_luna_20261007/summary.json
[**A95**] artifacts/python_method_recall_compact_luna_20261007/summary.json
[**A96**] artifacts/python_retained_method_comparison_luna_20261007/RUN_STATUS.md
[**A97**] artifacts/python_autonomous_recall_luna_20261007/protocol.json
[**A98**] artifacts/python_autonomous_recall_luna_20261007/summary.json
[**A99**] artifacts/python_autonomous_recall_luna_20261007/derived_analysis.json
[**A100**] artifacts/python_recall_frozen_transfer_20261007/protocol.json
[**A101**] artifacts/python_recall_frozen_transfer_20261007/summary.json
[**A102**] artifacts/python_recall_frozen_transfer_20261007/brief.json
[**A103**] scripts/run_python_recall_frozen_transfer.py
[**A104**] artifacts/executable_archive_controls_20261007/evidence.json
[**A105**] artifacts/response_structure_transfer_20261009/RUN_STATUS.md

[**A106**] artifacts/response_matched_adoption_20261010/RUN_STATUS.md
[**A107**] artifacts/response_assignment_comparison_20261010/protocol.json
[**A108**] artifacts/response_assignment_comparison_20261010/summary.json
[**A109**] artifacts/response_assignment_comparison_20261010/brief.json
[**A110**] artifacts/response_assignment_comparison_20261010/paid_exclusion_replay/verified.json
[**A111**] artifacts/response_revision_native_20261010/protocol.json
[**A112**] artifacts/response_revision_native_20261010/summary.json
[**A113**] artifacts/response_revision_native_20261010/brief.json
[**A114**] artifacts/response_revision_native_20261010/analysis.json
[**A115**] artifacts/response_native_parent_return_20261010/RUN_STATUS.md
[**A116**] artifacts/response_native_two_decision_replay_20261010/RUN_STATUS.md
[**A117**] artifacts/response_parent_return_live_20261010/RUN_STATUS.md
[**A118**] artifacts/response_current_public_view_live_20261010/RUN_STATUS.md
[**A119**] artifacts/response_refuted_author_return_20261010/live/protocol.json
[**A120**] artifacts/response_refuted_author_return_20261010/live/summary.json
[**A121**] artifacts/response_refuted_author_return_20261010/live_analysis.json
[**A122**] artifacts/response_refuted_author_return_20261010/analyze_live.py
[**A123**] artifacts/response_refuted_author_return_20261010/RUN_STATUS.md
[**A124**] artifacts/response_research_connections_live_20261010/RUN_STATUS.md
[**A125**] artifacts/response_research_connections_live_20261010/live_analysis.json
[**A126**] artifacts/response_research_connections_live_20261010/analyze.py
[**A127**] artifacts/response_research_reopen_20261010/RUN_STATUS.md
[**A128**] artifacts/response_research_reopen_20261010/live_protocol.json
[**A129**] artifacts/response_research_reopen_20261010/live/summary.json
[**A130**] artifacts/response_research_reopen_20261010/live_analysis.json
[**A131**] artifacts/response_assignment_resume_20261011/RUN_STATUS.md
[**A132**] artifacts/response_assignment_resume_20261011/live_protocol.json
[**A133**] artifacts/response_assignment_resume_20261011/live/summary.json
[**A134**] artifacts/response_assignment_resume_20261011/live_analysis.json
[**A135**] artifacts/response_assignment_resume_20261011/analyze_live.py
[**A136**] artifacts/response_late_withdrawal_20261011/protocol.json
[**A137**] artifacts/response_late_withdrawal_20261011/summary.json
[**A138**] artifacts/response_late_withdrawal_20261011/preservation/summary.json
[**A139**] artifacts/response_late_withdrawal_20261011/stage_outcome_audit.json
[**A140**] artifacts/response_numeric_transfer_20261011/protocol.json

[**A141**] artifacts/response_numeric_transfer_20261011/forecast_summary.json
[**A142**] artifacts/response_numeric_transfer_20261011/summary.json
[**A143**] artifacts/response_numeric_transfer_20261011/outcome_audit.json
[**A144**] scripts/run_response_numeric_transfer.py
[**A145**] artifacts/response_transfer_stage_resume_20261011/protocol.json
[**A146**] artifacts/response_transfer_stage_resume_20261011/summary.json
[**A147**] artifacts/response_transfer_stage_resume_20261011/stage_outcome_audit.json
[**A148**] artifacts/response_exhausted_stage_20261011/protocol.json
[**A149**] artifacts/response_exhausted_stage_20261011/summary.json
[**A150**] artifacts/response_exhausted_stage_20261011/outcome_audit.json
[**A151**] artifacts/response_point_finalization_20261011/public_replay.json
[**A152**] artifacts/response_point_finalization_20261011/summary.json
[**A153**] artifacts/response_forecast_classes_20261011/protocol.json
[**A154**] artifacts/response_forecast_classes_20261011/outcome_audit.json
[**A155**] artifacts/response_trajectory_choice_20261011/protocol.json
[**A156**] artifacts/response_trajectory_choice_20261011/outcome_audit.json
[**A157**] artifacts/response_trajectory_choice_20261011/withdrawal.json
[**A158**] artifacts/response_target_consequences_20261011/protocol.json
[**A159**] artifacts/response_target_consequences_20261011/withdrawal.json
[**A160**] artifacts/response_target_consequences_20261011/summary.json
[**A161**] artifacts/response_intermediate_redecision_20261011/protocol.json
[**A162**] artifacts/response_intermediate_redecision_20261011/summary.json
[**A163**] artifacts/response_pending_band_revision_20261011/protocol.json
[**A164**] artifacts/response_pending_band_revision_20261011/summary.json
[**A165**] artifacts/response_condition_retention_20261011/protocol.json
[**A166**] artifacts/response_condition_retention_20261011/summary.json
[**A167**] artifacts/response_condition_retention_20261011/outcome_audit.json
[**A168**] artifacts/response_exhausted_stage_20261011/confirmation/summary.json
[**A169**] artifacts/response_revision_successors_20261011/protocol.json
[**A170**] artifacts/response_revision_successors_20261011/summary.json
[**A171**] artifacts/response_revision_successors_20261011/source_replay_preservation.json
[**A172**] artifacts/response_revision_successors_20261011/outcome_audit.json
[**A173**] scripts/run_response_revision_successors.py
[**A174**] artifacts/sc_kg_meta_prior_efficient_v030_20260910/summary.json
[**A175**] artifacts/sc_kg_optimizer_data_reuse_v030_20260910/summary.json
[**A176**] artifacts/sc_kg_goal_completion_20260911/fresh_confirmation/protocol.json
[**A177**] artifacts/sc_kg_goal_completion_20260911/completion_summary.json

[**A178**] artifacts/sc_kg_constraint_knowledge_20260912/revision4/report_brief.json

[**A179**] artifacts/cognitive_direction_reset_20260912/protocol.json

[**A180**] artifacts/cognitive_direction_reset_20260912/summary.json

[**A181**] artifacts/cognitive_direction_reset_20260912/result_interpretation.json

[**A182**] artifacts/sc_kg_structural_prior_comparison_20261007/protocol.json

[**A183**] scripts/run_sc_kg_structural_prior_comparison.py

[**A184**] artifacts/sc_kg_structural_prior_comparison_20261007/summary.json

[**A185**] artifacts/sc_kg_structural_prior_comparison_20261007/brief.json

[**A186**] paper/figures/composed_offline_comparison.pdf

[**A187**] paper/figures/scoped_retention_comparison.pdf

[**A188**] artifacts/paper_retention_stopping_audit_20261007/summary.json

[**A189**] scripts/audit_scoped_retention_stopping.py

[**A190**] paper/figures/joint_research_growth.pdf

[**A191**] artifacts/paper_revision_choice_20261005/case_812676_confirmation.json

[**A192**] artifacts/composed_local_cycle_20261004_r3/case_26003_development.json

[**A193**] paper/figures/hypothesis_revision_trace.pdf

[**A194**] paper/figures/online_support_inquiry.pdf

[**A195**] artifacts/online_inquiry_20261005_main/summary.json

[**A196**] artifacts/conditional_components_luna_20261011/protocol.json

[**A197**] artifacts/recurrent_method_learning_luna_20261012/protocol.json

[**A198**] artifacts/recurrent_method_continuation_luna_20261012/protocol.json

[**A199**] artifacts/conditional_components_luna_20261011/summary.json

[**A200**] artifacts/conditional_components_luna_20261011/case_000/hegel/model.py

[**A201**] artifacts/conditional_components_luna_20261011/case_000/hegel/training.json

[**A202**] artifacts/recurrent_method_learning_luna_20261012/author/shared/consolidated_method.py

[**A203**] artifacts/recurrent_method_learning_luna_20261012/summary.json

[**A204**] artifacts/recurrent_method_continuation_luna_20261012/target_003/code_react/scratch_007.py

[**A205**] artifacts/recurrent_method_continuation_luna_20261012/target_003/code_react/training.json

[**A206**] artifacts/recurrent_method_continuation_luna_20261012/target_000/hegel/training.json

[**A207**] artifacts/recurrent_method_continuation_luna_20261012/target_003/hegel/training.json